

Notebook

March 3, 2026

1 Mini Project 1

Date: 03/02/2026

Name: Mikkel Korsgaard Sørensen

Institution: ES AAU

Class: NDS9

Subject: Numerical Scientific Computing

Github Repo <https://github.com/NDS8-TCP/nsc-mikkel>

System specifications:

- *CPU:* AMD Ryzen 7 PRO 6850U (16) @ 4.77 GHz
- *GPU:* AMD Radeon 680M [Integrated]
- *RAM:* 32 GiB LPDDR5 @ 6400 MT/s
- *OS:* Debian GNU/Linux forky/sid (forky) x86_64

Note: I completed *almost* all the exercises during the studio sessions, but I chose to redo them according to the mini project requirements presented in Lecture 3, which is why the Git commits appear close together.

1.1 Lecture 1: Course Intro and Development Tools

1.1.1 ****Studio Session 1:** Setup Python Environment**

****Task:** Create and configure course environment**

****Note:** We recommend **mamba** for best performance!**

****Step 1:** Create environment (10 min)** 1. Choose your tool: mamba** or conda**

I have chosen to go with *mamba*

2. Create environment named nsc2026

Note: For sanity check, clear mamba cache so that we get (somewhat) deterministic output. This is technically not needed

```
[1]: !mamba clean --all --yes
```

```
Collect information..
```

```
Cleaning index cache..
```

```
Cleaning lock files..
```

Package file	Size
--------------	------

/opt/conda/pkg

asttokens-3.0.1-pyhd8ed1ab_0.conda	29kB
ca-certificates-2026.2.25-hbd8a1cb_0.conda	147kB
comm-0.2.3-pyhe01879c_0.conda	15kB
debugpy-1.8.20-py311hc665b79_0.conda	3MB
decorator-5.2.1-pyhd8ed1ab_0.conda	14kB
executing-2.2.1-pyhd8ed1ab_0.conda	31kB
ipykernel-7.2.0-pyha191276_1.conda	134kB
ipython-9.10.0-pyh53cf698_0.conda	647kB
ipython_pygments_lexers-1.1.1-pyhd8ed1ab_0.conda	14kB
jedi-0.19.2-pyhd8ed1ab_1.conda	844kB
jupyter_client-8.8.0-pyhcf101f3_0.conda	113kB
jupyter_core-5.9.1-pyhcf90fa1f_0.conda	66kB
libsodium-1.0.20-h4ab18f5_0.conda	206kB
matplotlib-inline-0.2.1-pyhd8ed1ab_0.conda	15kB
nest-asyncio-1.6.0-pyhd8ed1ab_1.conda	12kB
parso-0.8.6-pyhcf101f3_0.conda	82kB
pexpect-4.9.0-pyhd8ed1ab_1.conda	54kB
platformdirs-4.9.2-pyhcf101f3_0.conda	26kB
prompt-toolkit-3.0.52-pyha770c72_0.conda	274kB
ptyprocess-0.7.0-pyhd8ed1ab_1.conda	19kB
pure_eval-0.2.3-pyhd8ed1ab_1.conda	17kB
pyzmq-27.1.0-py311h57d2397_2.conda	384kB
stack_data-0.6.3-pyhd8ed1ab_1.conda	27kB
traitlets-5.14.3-pyhd8ed1ab_1.conda	110kB
wcwidth-0.6.0-pyhd8ed1ab_0.conda	72kB
zeromq-4.3.5-h387f397_9.conda	311kB

/root/.mamba/pkg

Total size:	6MB
-------------	-----

```
Cleaning tarballs..
```

Package folder	Size
----------------	------

/opt/conda/pkg

ca-certificates-2026.1.4-hbd8a1cb_0 287kB

/root/.mamba/pkg

Total size: 287kB

Cleaning packages..

warning libmamba This does not check for packages installed using symlinks back to the package cache.

[2]: !mamba create -n nsc2026 python=3.11 -y

```
[+] 0.0s
[+] 0.1s
[+] 0.2s
conda-forge/linux-64          59.2kB / 51.4MB @
329.4kB/s 0.1s
conda-forge/noarch            345.8kB / 24.9MB @
1.9MB/s 0.1s[+] 0.3s
conda-forge/linux-64          184.3kB / 51.4MB @
801.6kB/s 0.2s
conda-forge/noarch            927.6kB / 24.9MB @
4.0MB/s 0.2s[+] 0.4s
conda-forge/linux-64          593.6kB / 51.4MB @
1.6MB/s 0.3s
conda-forge/noarch            2.5MB / 24.9MB @
6.8MB/s 0.3s[+] 0.5s
conda-forge/linux-64          921.3kB / 51.4MB @
1.9MB/s 0.4s
conda-forge/noarch            3.6MB / 24.9MB @
7.7MB/s 0.4s[+] 0.6s
conda-forge/linux-64          1.2MB / 51.4MB @
2.1MB/s 0.5s
conda-forge/noarch            4.7MB / 24.9MB @
8.2MB/s 0.5s[+] 0.7s
conda-forge/linux-64          1.4MB / 51.4MB @
2.1MB/s 0.6s
conda-forge/noarch            5.4MB / 24.9MB @
7.9MB/s 0.6s[+] 0.8s
conda-forge/linux-64          1.7MB / 51.4MB @
2.3MB/s 0.7s
```

conda-forge/noarch	6.5MB / 24.9MB @
8.8MB/s 0.7s[+] 0.9s	
conda-forge/linux-64	2.2MB / 51.4MB @
2.5MB/s 0.8s	
conda-forge/noarch	8.0MB / 24.9MB @
9.2MB/s 0.8s[+] 1.0s	
conda-forge/linux-64	2.6MB / 51.4MB @
2.6MB/s 0.9s	
conda-forge/noarch	9.3MB / 24.9MB @
9.4MB/s 0.9s[+] 1.1s	
conda-forge/linux-64	2.9MB / 51.4MB @
2.7MB/s 1.0s	
conda-forge/noarch	10.4MB / 24.9MB @
9.5MB/s 1.0s[+] 1.2s	
conda-forge/linux-64	3.2MB / 51.4MB @
2.7MB/s 1.1s	
conda-forge/noarch	11.5MB / 24.9MB @
9.6MB/s 1.1s[+] 1.3s	
conda-forge/linux-64	3.6MB / 51.4MB @
2.7MB/s 1.2s	
conda-forge/noarch	12.5MB / 24.9MB @
9.6MB/s 1.2s[+] 1.4s	
conda-forge/linux-64	3.8MB / 51.4MB @
2.8MB/s 1.3s	
conda-forge/noarch	13.0MB / 24.9MB @
9.6MB/s 1.3s[+] 1.5s	
conda-forge/linux-64	4.1MB / 51.4MB @
2.8MB/s 1.4s	
conda-forge/noarch	14.1MB / 24.9MB @
9.7MB/s 1.4s[+] 1.6s	
conda-forge/linux-64	4.4MB / 51.4MB @
2.9MB/s 1.5s	
conda-forge/noarch	15.2MB / 24.9MB @
9.8MB/s 1.5s[+] 1.7s	
conda-forge/linux-64	4.8MB / 51.4MB @
2.9MB/s 1.6s	
conda-forge/noarch	16.2MB / 24.9MB @
9.8MB/s 1.6s[+] 1.8s	
conda-forge/linux-64	5.1MB / 51.4MB @
2.9MB/s 1.7s	
conda-forge/noarch	17.3MB / 24.9MB @
9.9MB/s 1.7s[+] 1.9s	
conda-forge/linux-64	5.6MB / 51.4MB @
3.0MB/s 1.8s	
conda-forge/noarch	18.4MB / 24.9MB @
9.9MB/s 1.8s[+] 2.0s	
conda-forge/linux-64	5.9MB / 51.4MB @
3.0MB/s 1.9s	

conda-forge/noarch	19.4MB / 24.9MB @
9.9MB/s 1.9s[+] 2.1s	
conda-forge/linux-64	6.3MB / 51.4MB @
3.1MB/s 2.0s	
conda-forge/noarch	20.4MB / 24.9MB @
9.9MB/s 2.0s[+] 2.2s	
conda-forge/linux-64	6.7MB / 51.4MB @
3.1MB/s 2.1s	
conda-forge/noarch	21.4MB / 24.9MB @
9.9MB/s 2.1s[+] 2.3s	
conda-forge/linux-64	7.1MB / 51.4MB @
3.2MB/s 2.2s	
conda-forge/noarch	22.5MB / 24.9MB @
9.9MB/s 2.2s[+] 2.4s	
conda-forge/linux-64	7.5MB / 51.4MB @
3.2MB/s 2.3s	
conda-forge/noarch	23.5MB / 24.9MB @
10.0MB/s 2.3s[+] 2.5s	
conda-forge/linux-64	7.9MB / 51.4MB @
3.2MB/s 2.4s	
conda-forge/noarch	24.5MB / 24.9MB @
9.8MB/s 2.4s[+] 2.6s	
conda-forge/linux-64	7.9MB / 51.4MB @
3.2MB/s 2.5s	
conda-forge/noarch	24.5MB / 24.9MB @
9.8MB/s 2.5sconda-forge/noarch	
24.9MB @ 9.8MB/s 2.5s	
[+] 2.7s	
conda-forge/linux-64	9.0MB / 51.4MB @
3.4MB/s 2.6s[+] 2.8s	
conda-forge/linux-64	9.6MB / 51.4MB @
3.5MB/s 2.7s[+] 2.9s	
conda-forge/linux-64	10.4MB / 51.4MB @
3.6MB/s 2.8s[+] 3.0s	
conda-forge/linux-64	11.4MB / 51.4MB @
3.8MB/s 2.9s[+] 3.1s	
conda-forge/linux-64	12.6MB / 51.4MB @
4.1MB/s 3.0s[+] 3.2s	
conda-forge/linux-64	13.9MB / 51.4MB @
4.4MB/s 3.1s[+] 3.3s	
conda-forge/linux-64	15.4MB / 51.4MB @
4.7MB/s 3.2s[+] 3.4s	
conda-forge/linux-64	16.8MB / 51.4MB @
5.0MB/s 3.3s[+] 3.5s	
conda-forge/linux-64	18.2MB / 51.4MB @
5.2MB/s 3.4s[+] 3.6s	
conda-forge/linux-64	19.6MB / 51.4MB @
5.5MB/s 3.5s[+] 3.7s	

conda-forge/linux-64	21.1MB / 51.4MB @
5.7MB/s 3.6s[+] 3.8s	
conda-forge/linux-64	22.5MB / 51.4MB @
6.0MB/s 3.7s[+] 3.9s	
conda-forge/linux-64	23.9MB / 51.4MB @
6.2MB/s 3.8s[+] 4.0s	
conda-forge/linux-64	25.3MB / 51.4MB @
6.4MB/s 3.9s[+] 4.1s	
conda-forge/linux-64	26.7MB / 51.4MB @
6.5MB/s 4.0s[+] 4.2s	
conda-forge/linux-64	28.1MB / 51.4MB @
6.7MB/s 4.1s[+] 4.3s	
conda-forge/linux-64	29.6MB / 51.4MB @
6.9MB/s 4.2s[+] 4.4s	
conda-forge/linux-64	30.5MB / 51.4MB @
6.9MB/s 4.3s[+] 4.5s	
conda-forge/linux-64	32.5MB / 51.4MB @
7.2MB/s 4.4s[+] 4.6s	
conda-forge/linux-64	33.1MB / 51.4MB @
7.3MB/s 4.5s[+] 4.7s	
conda-forge/linux-64	34.5MB / 51.4MB @
7.4MB/s 4.6s[+] 4.8s	
conda-forge/linux-64	35.9MB / 51.4MB @
7.6MB/s 4.7s[+] 4.9s	
conda-forge/linux-64	37.3MB / 51.4MB @
7.7MB/s 4.8s[+] 5.0s	
conda-forge/linux-64	38.2MB / 51.4MB @
7.7MB/s 4.9s[+] 5.1s	
conda-forge/linux-64	40.2MB / 51.4MB @
7.9MB/s 5.0s[+] 5.2s	
conda-forge/linux-64	41.0MB / 51.4MB @
7.9MB/s 5.1s[+] 5.3s	
conda-forge/linux-64	42.6MB / 51.4MB @
8.1MB/s 5.2s[+] 5.4s	
conda-forge/linux-64	44.1MB / 51.4MB @
8.2MB/s 5.3s[+] 5.5s	
conda-forge/linux-64	45.4MB / 51.4MB @
8.3MB/s 5.4s[+] 5.6s	
conda-forge/linux-64	46.7MB / 51.4MB @
8.4MB/s 5.5s[+] 5.7s	
conda-forge/linux-64	47.4MB / 51.4MB @
8.4MB/s 5.6s[+] 5.8s	
conda-forge/linux-64	48.8MB / 51.4MB @
8.5MB/s 5.7s[+] 5.9s	
conda-forge/linux-64	50.1MB / 51.4MB @
8.6MB/s 5.8s[+] 6.0s	
conda-forge/linux-64	51.4MB / 51.4MB @
8.6MB/s 5.9s[+] 6.1s	

```
conda-forge/linux-64          51.4MB / 51.4MB @
8.6MB/s 6.0sconda-forge/linux-64
51.4MB @ 8.6MB/s 6.1s
```

Transaction

Prefix: /opt/conda/envs/nsc2026

Updating specs:

- python=3.11

Package	Version	Build	Channel	Size
Install:				
+ <code>_openmp_mutex</code>	4.5	20_gnu	conda-forge	29kB
+ <code>bzip2</code>	1.0.8	hda65f42_9	conda-forge	260kB
+ <code>ca-certificates</code>	2026.2.25	hbd8a1cb_0	conda-forge	
Cached				
+ <code>icu</code>	78.2	h33c6efd_0	conda-forge	
Cached				
+ <code>ld_impl_linux-64</code>	2.45.1	default_hbd61a6d_101	conda-forge	
Cached				
+ <code>libexpat</code>	2.7.4	hecca717_0	conda-forge	77kB
+ <code>libffi</code>	3.5.2	h3435931_0	conda-forge	
Cached				
+ <code>libgcc</code>	15.2.0	he0feb66_18	conda-forge	1MB
+ <code>libgcc-ng</code>	15.2.0	h69a702a_18	conda-forge	28kB
+ <code>libgomp</code>	15.2.0	he0feb66_18	conda-forge	603kB
+ <code>liblzma</code>	5.8.2	hb03c661_0	conda-forge	
Cached				
+ <code>libnsl</code>	2.0.1	hb9d3cd8_1	conda-forge	
Cached				

+ <code>libsqlite</code>	3.51.2	hf4e2dac_0	conda-forge
Cached			
+ <code>libstdcxx</code>	15.2.0	h934c35e_18	conda-forge
6MB			
+ <code>libuuid</code>	2.41.3	h5347b49_0	conda-forge
Cached			
+ <code>libxcrypt</code>	4.4.36	hd590300_1	conda-forge
Cached			
+ <code>libzlib</code>	1.3.1	hb9d3cd8_2	conda-forge
Cached			
+ <code>ncurses</code>	6.5	h2d0b736_3	conda-forge
Cached			
+ <code>openssl</code>	3.6.1	h35e630c_1	conda-forge
Cached			
+ <code>packaging</code>	26.0	pyhcf101f3_0	conda-forge
Cached			
+ <code>pip</code>	26.0.1	pyh8b19718_0	conda-forge
Cached			
+ <code>python</code>	3.11.14	hd63d673_3_cpython	conda-forge
Cached			
+ <code>readline</code>	8.3	h853b02a_0	conda-forge
Cached			
+ <code>setuptools</code>	82.0.0	pyh332efcf_0	conda-forge
638kB			
+ <code>tk</code>	8.6.13	noxft_h366c992_103	conda-forge
Cached			
+ <code>tzdata</code>	2025c	hc9c84f9_1	conda-forge
Cached			
+ <code>wheel</code>	0.46.3	pyhd8ed1ab_0	conda-forge
Cached			
+ <code>zstd</code>	1.5.7	hb78ec9c_6	conda-forge
Cached			

Summary:

Install: 28 packages

Total download: 9MB


```

Transaction starting
[+] 0.0s
Downloading 0.0 B
0.0s
Extracting 0
0.0s[+] 0.1s
Downloading (5) 0.0 B bzip2
0.0s
Extracting 0
0.0s[+] 0.2s
Downloading (5) 404.7kB bzip2
0.1s
Extracting 0
0.0s[+] 0.3s
Downloading (5) 1.8MB bzip2
0.2s
Extracting 0
0.0ssetuptools
637.5kB @ 2.0MB/s 0.3s
libgomp 603.3kB @ 1.0MB/s 0.3s
bzip2 260.2kB @ 682.9kB/s 0.3s
libexpat 76.8kB @ ???.?MB/s 0.1s
libgcc 1.0MB @ 2.3MB/s 0.3s
[+] 0.4s
Downloading (3) 3.5MB _openmp_mutex
0.3s
Extracting (3) 2 bzip2
0.0s_openmp_mutex
28.9kB @ ???.?MB/s 0.1s
libgcc-ng 27.5kB @ ???.?MB/s 0.1s
[+] 0.5s
Downloading (1) 4.2MB libstdcxx
0.4s
Extracting 7
0.1s[+] 0.6s
Downloading (1) 4.9MB libstdcxx
0.5s
Extracting 7
0.1s[+] 0.7s
Downloading (1) 5.8MB libstdcxx
0.6s
Extracting 7
0.1s[+] 0.8s
Downloading (1) 6.9MB libstdcxx

```

```

0.7s
Extracting                                7
0.1slibstdcxx
5.9MB @ 6.5MB/s 0.8s
[+] 0.9s
Downloading                                8.5MB                                0.8s
Extracting (1)                            7 libstdcxx
0.1sLinking tzdata-2025c-hc9c84f9_1
Linking ca-certificates-2026.2.25-hbd8a1cb_0
Linking libgomp-15.2.0-he0feb66_18
Linking _openmp_mutex-4.5-20_gnu
Linking libgcc-15.2.0-he0feb66_18
Linking libstdcxx-15.2.0-h934c35e_18
Linking libffi-3.5.2-h3435931_0
Linking openssl-3.6.1-h35e630c_1
Linking libzlib-1.3.1-hb9d3cd8_2
Linking libgcc-ng-15.2.0-h69a702a_18
Linking libns1-2.0.1-hb9d3cd8_1
Linking libexpat-2.7.4-hecca717_0
Linking bzip2-1.0.8-hda65f42_9
Linking ncurses-6.5-h2d0b736_3
Linking libuuid-2.41.3-h5347b49_0
Linking liblzma-5.8.2-hb03c661_0
Linking icu-78.2-h33c6efd_0
Linking tk-8.6.13-noxft_h366c992_103
Linking zstd-1.5.7-hb78ec9c_6
Linking libxcrypt-4.4.36-hd590300_1
Linking readline-8.3-h853b02a_0
Linking libsqlite-3.51.2-hf4e2dac_0
Linking ld_impl_linux-64-2.45.1-default_hbd61a6d_101
Linking python-3.11.14-hd63d673_3_cpython
Linking packaging-26.0-pyhcf101f3_0
Linking setuptools-82.0.0-pyh332efcf_0
Linking wheel-0.46.3-pyhd8ed1ab_0
Linking pip-26.0.1-pyh8b19718_0

```

Transaction finished

To activate this environment, use:

```
mamba activate nsc2026
```

Or to execute a single command in this environment, use:

```
mamba run -n nsc2026 mycommand
```

3. Activate it

```
[3]: !mamba activate nsc2026
```

Your parent process name is Name: python.
If your shell is xonsh, please use "-s xonsh".

'mamba' is running as a subprocess and can't modify the parent shell.
Thus you must initialize your shell before using activate and deactivate.

To initialize the current bash shell, run:

```
$ eval "$(mamba shell hook --shell bash)"
```

and then activate or deactivate with:

```
$ mamba activate
```

To automatically initialize all future (bash) shells, run:

```
$ mamba shell init --shell bash --root-prefix=~/.local/share/mamba
```

If your shell was already initialized, reinitialize your shell with:

```
$ mamba shell reinit --shell bash
```

Otherwise, this may be an issue. In the meantime you can run commands. See:

```
$ mamba run --help
```

Supported shells are {bash, zsh, csh, posix, xonsh, cmd.exe, powershell, fish, nu}.

critical libmamba Shell not initialized

Because of jupyter notebooks limitations' on subshells, I instead chose to run via the `mamba run --name nsc2026`

4. Verify: python --version

```
[4]: !mamba run --name nsc2026 \n      python --version
```

Python 3.11.14

Command copied from slide 19

```
[5]: !python -c "import numpy; numpy.show_config()" | grep -iE\n      ↪"openblas|mk1|ACCELERATE"
```

```
openblas configuration: unknown
```

```
openblas configuration: unknown
```

We see output, so we should be optimised! Good to go.

******Step 2:** Install core packages (15 min)** 1. numpy, matplotlib, scipy**

```
[6]: !mamba install --name nsc2026 \n      numpy matplotlib scipy -y
```

conda-forge/linux-64

Using cache

conda-forge/noarch

Using cache

Pinned packages:

- python=3.11

Transaction

Prefix: /opt/conda/envs/nsc2026

Updating specs:

- numpy
- matplotlib
- scipy

Package Size	Version	Build	Channel
-----------------	---------	-------	---------

Install:

+ alsa-lib	1.2.15.3	hb03c661_0	
conda-forge	Cached		
+ brotli	1.2.0	hed03a55_1	
conda-forge	Cached		
+ brotli-bin	1.2.0	hb03c661_1	
conda-forge	Cached		
+ cairo	1.18.4	he90730b_1	
conda-forge	Cached		
+ contourpy	1.3.3	py311h724c32c_4	
conda-forge	Cached		
+ cycler	0.12.1	pyhcf101f3_2	
conda-forge	Cached		
+ cyrus-sasl	2.1.28	hac629b4_1	
conda-forge	210kB		
+ dbus	1.16.2	h24cb091_1	
conda-forge	Cached		
+ double-conversion	3.4.0	hecca717_0	
conda-forge	Cached		
+ font-ttf-dejavu-sans-mono	2.37	hab24e00_0	
conda-forge	Cached		
+ font-ttf-inconsolata	3.000	h77eed37_0	
conda-forge	Cached		
+ font-ttf-source-code-pro	2.038	h77eed37_0	

conda-forge	Cached		
+ font-ttf-ubuntu		0.83	h77eed37_3
conda-forge	Cached		
+ fontconfig		2.17.1	h27c8c51_0
conda-forge	271kB		
+ fonts-conda-ecosystem		1	0
conda-forge	Cached		
+ fonts-conda-forge		1	hc364b38_1
conda-forge	Cached		
+ fonttools		4.61.1	py311h3778330_0
conda-forge	Cached		
+ freetype		2.14.1	ha770c72_0
conda-forge	Cached		
+ graphite2		1.3.14	hecca717_2
conda-forge	Cached		
+ harfbuzz		12.3.2	h6083320_0
conda-forge	Cached		
+ keyutils		1.6.3	hb9d3cd8_0
conda-forge	Cached		
+ kiwisolver		1.4.9	py311h724c32c_2
conda-forge	Cached		
+ krb5		1.22.2	ha1258a1_0
conda-forge	1MB		
+ lcms2		2.18	h0c24ade_0
conda-forge	Cached		
+ lerc		4.0.0	h0aef613_1
conda-forge	Cached		
+ libblas		3.11.0	5_h4a7cf45_openblas
conda-forge	Cached		
+ libbrotlicommon		1.2.0	hb03c661_1
conda-forge	Cached		
+ libbrotlidec		1.2.0	hb03c661_1
conda-forge	Cached		
+ libbrotlienc		1.2.0	hb03c661_1
conda-forge	Cached		
+ libcbblas		3.11.0	5_h0358290_openblas
conda-forge	Cached		
+ libclang-cpp22.1		22.1.0	default_h99862b1_0
conda-forge	22MB		
+ libclang13		22.1.0	default_h746c552_0
conda-forge	13MB		
+ libcups		2.3.3	h7a8fb5f_6
conda-forge	5MB		
+ libdeflate		1.25	h17f619e_0
conda-forge	Cached		
+ libdrm		2.4.125	hb03c661_1
conda-forge	Cached		
+ libedit		3.1.20250104	pl5321h7949ede_0

conda-forge	Cached		
+ libegl		1.7.0	ha4b6fd6_2
conda-forge	Cached		
+ libfreetype		2.14.1	ha770c72_0
conda-forge	Cached		
+ libfreetype6		2.14.1	h73754d4_0
conda-forge	Cached		
+ libgfortran		15.2.0	h69a702a_18
conda-forge	28kB		
+ libgfortran5		15.2.0	h68bc16d_18
conda-forge	2MB		
+ libgl		1.7.0	ha4b6fd6_2
conda-forge	Cached		
+ libglib		2.86.4	h6548e54_1
conda-forge	4MB		
+ libglvnd		1.7.0	ha4b6fd6_2
conda-forge	Cached		
+ libglx		1.7.0	ha4b6fd6_2
conda-forge	Cached		
+ libiconv		1.18	h3b78370_2
conda-forge	Cached		
+ libjpeg-turbo		3.1.2	hb03c661_0
conda-forge	Cached		
+ liblapack		3.11.0	5_h47877c9_openblas
conda-forge	Cached		
+ libllvm22		22.1.0	hf7376ad_0
conda-forge	44MB		
+ libntlm		1.8	hb9d3cd8_0
conda-forge	Cached		
+ libopenblas		0.3.30	pthread_h94d23a6_4
conda-forge	Cached		
+ libopengl		1.7.0	ha4b6fd6_2
conda-forge	Cached		
+ libpciaccess		0.18	hb9d3cd8_0
conda-forge	Cached		
+ libpng		1.6.55	h421ea60_0
conda-forge	318kB		
+ libpq		18.3	h9abb657_0
conda-forge	3MB		
+ libstdcxx-ng		15.2.0	hdf11a46_18
conda-forge	28kB		
+ libtiff		4.7.1	h9d88235_1
conda-forge	Cached		
+ libvulkan-loader		1.4.341.0	h5279c79_0
conda-forge	Cached		
+ libwebp-base		1.6.0	hd42ef1d_0
conda-forge	Cached		
+ libxcb		1.17.0	h8a09558_0

conda-forge	Cached		
+ libxkbcommon		1.13.1	hca5e8e5_0
conda-forge	Cached		
+ libxml2		2.15.1	he237659_1
conda-forge	Cached		
+ libxml2-16		2.15.1	hca6bf5a_1
conda-forge	Cached		
+ libxslt		1.1.43	h711ed8c_1
conda-forge	Cached		
+ matplotlib		3.10.8	py311h38be061_0
conda-forge	Cached		
+ matplotlib-base		3.10.8	py311h0f3be63_0
conda-forge	Cached		
+ munkres		1.1.4	pyhd8ed1ab_1
conda-forge	Cached		
+ numpy		2.4.2	py311h2e04523_1
conda-forge	9MB		
+ openjpeg		2.5.4	h55fea9a_0
conda-forge	Cached		
+ openldap		2.6.10	hbde042b_1
conda-forge	786kB		
+ pcre2		10.47	haa7fec5_0
conda-forge	Cached		
+ pillow		12.1.1	py311hf88fc01_0
conda-forge	1MB		
+ pixman		0.46.4	h54a6638_1
conda-forge	Cached		
+ pthread-stubs		0.4	hb9d3cd8_1002
conda-forge	Cached		
+ pyparsing		3.3.2	pyhcf101f3_0
conda-forge	Cached		
+ pyside6		6.10.2	py311he4c1a5a_0
conda-forge	Cached		
+ python-dateutil		2.9.0.post0	pyhe01879c_2
conda-forge	Cached		
+ python_abi		3.11	8_cp311
conda-forge	Cached		
+ qhull		2020.2	h434a139_5
conda-forge	Cached		
+ qt6-main		6.10.2	h17e89b9_5
conda-forge	57MB		
+ scipy		1.17.1	py311hbe70eeb_0
conda-forge	17MB		
+ six		1.17.0	pyhe01879c_1
conda-forge	Cached		
+ tornado		6.5.4	py311h41d9c34_0
conda-forge	Cached		
+ unicodedata2		17.0.1	py311h49ec1c0_0

conda-forge	410kB		
+ wayland		1.24.0	hd6090a7_1
conda-forge	Cached		
+ xcb-util		0.4.1	h4f16b4b_2
conda-forge	Cached		
+ xcb-util-cursor		0.1.6	hb03c661_0
conda-forge	Cached		
+ xcb-util-image		0.4.0	hb711507_2
conda-forge	Cached		
+ xcb-util-keysyms		0.4.1	hb711507_0
conda-forge	Cached		
+ xcb-util-renderutil		0.3.10	hb711507_0
conda-forge	Cached		
+ xcb-util-wm		0.4.2	hb711507_0
conda-forge	Cached		
+ xkeyboard-config		2.47	hb03c661_0
conda-forge	399kB		
+ xorg-libice		1.1.2	hb9d3cd8_0
conda-forge	Cached		
+ xorg-libsm		1.2.6	he73a12e_0
conda-forge	Cached		
+ xorg-libx11		1.8.13	he1eb515_0
conda-forge	840kB		
+ xorg-libxau		1.0.12	hb03c661_1
conda-forge	Cached		
+ xorg-libxcomposite		0.4.7	hb03c661_0
conda-forge	Cached		
+ xorg-libxcursor		1.2.3	hb9d3cd8_0
conda-forge	Cached		
+ xorg-libxdamage		1.1.6	hb9d3cd8_0
conda-forge	Cached		
+ xorg-libxdmcp		1.1.5	hb03c661_1
conda-forge	Cached		
+ xorg-libxext		1.3.7	hb03c661_0
conda-forge	Cached		
+ xorg-libxfixed		6.0.2	hb03c661_0
conda-forge	Cached		
+ xorg-libxi		1.8.2	hb9d3cd8_0
conda-forge	Cached		
+ xorg-libxrandr		1.5.5	hb03c661_0
conda-forge	Cached		
+ xorg-libxrender		0.9.12	hb9d3cd8_0
conda-forge	Cached		
+ xorg-libxtst		1.2.5	hb9d3cd8_3
conda-forge	Cached		
+ xorg-libxxf86vm		1.1.7	hb03c661_0
conda-forge	Cached		
+ zlib-ng		2.3.3	hceb46e0_1

conda-forge **Cached**

Summary:

Install: 108 packages

Total download: 182MB

Transaction starting

[+] 0.0s	
Downloading	0.0 B
0.0s	
Extracting	0
0.0s[+] 0.1s	
Downloading (5)	0.0 B libclang-cpp22.1
0.0s	
Extracting	0
0.0s[+] 0.2s	
Downloading (5)	344.1kB libclang-cpp22.1
0.1s	
Extracting	0
0.0s[+] 0.3s	
Downloading (5)	1.6MB libclang-cpp22.1
0.2s	
Extracting	0
0.0s[+] 0.4s	
Downloading (5)	3.1MB libclang-cpp22.1
0.3s	
Extracting	0
0.0s[+] 0.5s	
Downloading (5)	4.5MB libclang13
0.4s	
Extracting	0
0.0s[+] 0.6s	
Downloading (5)	5.3MB libclang13
0.5s	
Extracting	0
0.0s[+] 0.7s	
Downloading (5)	7.3MB libclang13
0.6s	
Extracting	0
0.0s[+] 0.8s	
Downloading (5)	7.9MB libclang13

0.7s	
Extracting	0
0.0s[+] 0.9s	
Downloading (5)	9.6MB libllvm22
0.8s	
Extracting	0
0.0s[+] 1.0s	
Downloading (5)	11.6MB libllvm22
0.9s	
Extracting	0
0.0s[+] 1.1s	
Downloading (5)	13.0MB libllvm22
1.0s	
Extracting	0
0.0s[+] 1.2s	
Downloading (5)	13.7MB libllvm22
1.1s	
Extracting	0
0.0s[+] 1.3s	
Downloading (5)	15.3MB qt6-main
1.2s	
Extracting	0
0.0s[+] 1.4s	
Downloading (5)	17.2MB qt6-main
1.3s	
Extracting	0
0.0s[+] 1.5s	
Downloading (5)	18.6MB qt6-main
1.4s	
Extracting	0
0.0s[+] 1.6s	
Downloading (5)	19.3MB qt6-main
1.5s	
Extracting	0
0.0s[+] 1.7s	
Downloading (5)	21.4MB scipy
1.6s	
Extracting	0
0.0s[+] 1.8s	
Downloading (5)	22.9MB scipy
1.7s	
Extracting	0
0.0s[+] 1.9s	
Downloading (5)	24.2MB scipy
1.8s	
Extracting	0
0.0s[+] 2.0s	
Downloading (5)	25.7MB scipy

1.9s		
Extracting	0	
0.0s[+] 2.1s		
Downloading (5)	27.0MB	libclang-cpp22.1
2.0s		
Extracting	0	
0.0s[+] 2.2s		
Downloading (5)	28.0MB	libclang-cpp22.1
2.1s		
Extracting	0	
0.0s[+] 2.3s		
Downloading (5)	29.7MB	libclang-cpp22.1
2.2s		
Extracting	0	
0.0s[+] 2.4s		
Downloading (5)	31.0MB	libclang-cpp22.1
2.3s		
Extracting	0	
0.0slibclang13		
12.8MB @ 5.2MB/s 2.4s		
[+] 2.5s		
Downloading (5)	32.3MB	libllvm22
2.4s		
Extracting (1)	0	libclang13
0.0s[+] 2.6s		
Downloading (5)	33.6MB	libllvm22
2.5s		
Extracting	1	
0.1s[+] 2.7s		
Downloading (5)	34.3MB	libllvm22
2.6s		
Extracting	1	
0.1s[+] 2.8s		
Downloading (5)	35.9MB	libllvm22
2.7s		
Extracting	1	
0.1s[+] 2.9s		
Downloading (5)	37.2MB	numpy
2.8s		
Extracting	1	
0.1s[+] 3.0s		
Downloading (5)	38.3MB	numpy
2.9s		
Extracting	1	
0.1s[+] 3.1s		
Downloading (5)	39.5MB	numpy
3.0s		
Extracting	1	

0.1s[+] 3.2s		
Downloading (5)	40.9MB	numpy
3.1s		
Extracting	1	
0.1s[+] 3.3s		
Downloading (5)	42.0MB	qt6-main
3.2s		
Extracting	1	
0.1s[+] 3.4s		
Downloading (5)	43.2MB	qt6-main
3.3s		
Extracting	1	
0.1s[+] 3.5s		
Downloading (5)	44.5MB	qt6-main
3.4s		
Extracting	1	
0.1s[+] 3.6s		
Downloading (5)	45.9MB	qt6-main
3.5s		
Extracting	1	
0.1s[+] 3.7s		
Downloading (5)	46.9MB	scipy
3.6s		
Extracting	1	
0.1s[+] 3.8s		
Downloading (5)	48.2MB	scipy
3.7s		
Extracting	1	
0.1s[+] 3.9s		
Downloading (5)	49.6MB	scipy
3.8s		
Extracting	1	
0.1s[+] 4.0s		
Downloading (5)	50.7MB	scipy
3.9s		
Extracting	1	
0.1s[+] 4.1s		
Downloading (5)	52.0MB	libclang-cpp22.1
4.0s		
Extracting	1	
0.1s[+] 4.2s		
Downloading (5)	53.3MB	libclang-cpp22.1
4.1s		
Extracting	1	
0.1s[+] 4.3s		
Downloading (5)	54.5MB	libclang-cpp22.1
4.2s		
Extracting	1	

0.1s[+] 4.4s	
Downloading (5)	55.7MB libclang-cpp22.1
4.3s	
Extracting	1
0.1s[+] 4.5s	
Downloading (5)	56.8MB libllvm22
4.4s	
Extracting	1
0.1s[+] 4.6s	
Downloading (5)	57.9MB libllvm22
4.5s	
Extracting	1
0.1s[+] 4.7s	
Downloading (5)	59.3MB libllvm22
4.6s	
Extracting	1
0.1s[+] 4.8s	
Downloading (5)	60.6MB libllvm22
4.7s	
Extracting	1
0.1s[+] 4.9s	
Downloading (5)	61.9MB numpy
4.8s	
Extracting	1
0.1s[+] 5.0s	
Downloading (5)	63.1MB numpy
4.9s	
Extracting	1
0.1s[+] 5.1s	
Downloading (5)	64.4MB numpy
5.0s	
Extracting	1
0.1s[+] 5.2s	
Downloading (5)	65.5MB numpy
5.1s	
Extracting	1
0.1s[+] 5.3s	
Downloading (5)	66.7MB qt6-main
5.2s	
Extracting	1
0.1s[+] 5.4s	
Downloading (5)	68.0MB qt6-main
5.3s	
Extracting	1
0.1s[+] 5.5s	
Downloading (5)	69.3MB qt6-main
5.4s	
Extracting	1

```

0.1snumpy
9.4MB @ 3.1MB/s 3.1s
[+] 5.6s
Downloading (5) 70.6MB qt6-main
5.5s
Extracting (1) 1 numpy
0.1s[+] 5.7s
Downloading (5) 71.7MB scipy
5.6s
Extracting (1) 1 numpy
0.2s[+] 5.8s
Downloading (5) 72.9MB scipy
5.7s
Extracting 2
0.3s[+] 5.9s
Downloading (5) 74.1MB scipy
5.8s
Extracting 2
0.3slibclang-cpp22.1
21.7MB @ 3.6MB/s 5.9s
[+] 6.0s
Downloading (5) 75.6MB scipy
5.9s
Extracting (1) 2 libclang-
cpp22.1 0.3s[+] 6.1s
Downloading (5) 76.7MB libcups
6.0s
Extracting (1) 2 libclang-
cpp22.1 0.4s[+] 6.2s
Downloading (5) 78.0MB libcups
6.1s
Extracting (1) 2 libclang-
cpp22.1 0.5s[+] 6.3s
Downloading (5) 79.0MB libcups
6.2s
Extracting 3
0.6s[+] 6.4s
Downloading (5) 80.3MB libcups
6.3s
Extracting 3
0.6s[+] 6.5s
Downloading (5) 81.5MB libglib
6.4s
Extracting 3
0.6s[+] 6.6s
Downloading (5) 82.8MB libglib
6.5s
Extracting 3

```

```

0.6s[+] 6.7s
Downloading (5) 84.0MB libglib
6.6s
Extracting 3
0.6s[+] 6.8s
Downloading (5) 85.3MB libglib
6.7s
Extracting 3
0.6s[+] 6.9s
Downloading (5) 86.6MB libllvm22
6.8s
Extracting 3
0.6s[+] 7.0s
Downloading (5) 87.8MB libllvm22
6.9s
Extracting 3
0.6s[+] 7.1s
Downloading (5) 88.9MB libllvm22
7.0s
Extracting 3
0.6s[+] 7.2s
Downloading (5) 90.2MB libllvm22
7.1s
Extracting 3
0.6slibglib
4.4MB @ 3.3MB/s 1.3s
[+] 7.3s
Downloading (5) 91.5MB libpq
7.2s
Extracting (1) 3 libglib
0.6s[+] 7.4s
Downloading (5) 92.9MB libpq
7.3s
Extracting 4
0.7s[+] 7.5s
Downloading (5) 93.8MB libpq
7.4s
Extracting 4
0.7s[+] 7.6s
Downloading (5) 95.2MB libpq
7.5s
Extracting 4
0.7s[+] 7.7s
Downloading (5) 96.4MB qt6-main
7.6s
Extracting 4
0.7sscipy
17.1MB @ 2.2MB/s 7.7s

```

```

[+] 7.8s
Downloading (5) 97.8MB qt6-main
7.7s
Extracting (1) 4 scipy
0.7s[+] 7.9s
Downloading (5) 99.1MB qt6-main
7.8s
Extracting (1) 4 scipy
0.8s[+] 8.0s
Downloading (5) 100.2MB qt6-main
7.9s
Extracting (1) 4 scipy
0.9s[+] 8.1s
Downloading (5) 101.4MB libcup
8.0s
Extracting 5
1.0s[+] 8.2s
Downloading (5) 102.6MB libcup
8.1s
Extracting 5
1.0slibpq
2.9MB @ 3.1MB/s 0.9s
[+] 8.3s
Downloading (5) 103.9MB libcup
8.2s
Extracting (1) 5 libpq
1.0s[+] 8.4s
Downloading (5) 105.0MB libcup
8.3s
Extracting 6
1.1slibcup
4.5MB @ 1.6MB/s 2.9s
[+] 8.5s
Downloading (5) 106.2MB krb5
8.4s
Extracting 7
1.1s[+] 8.6s
Downloading (5) 107.3MB krb5
8.5s
Extracting 7
1.1s[+] 8.7s
Downloading (5) 108.4MB krb5
8.6s
Extracting 7
1.1s[+] 8.8s
Downloading (5) 109.8MB krb5
8.7s
Extracting 7

```



```

1.1skrb5
1.4MB @ 2.0MB/s 0.7s
[+] 8.9s
Downloading (5) 111.1MB libgfortran5
8.8s
Extracting (1) 7 krb5
1.1slibgfortran5
2.5MB @ 2.0MB/s 1.2s
[+] 9.0s
Downloading (5) 112.6MB libllvm22
8.9s
Extracting (1) 8 libgfortran5
1.2s[+] 9.1s
Downloading (5) 113.9MB libllvm22
9.0s
Extracting 9
1.3spillow
1.0MB @ 1.3MB/s 0.8s
[+] 9.2s
Downloading (5) 115.1MB libllvm22
9.1s
Extracting 9
1.3s[+] 9.3s
Downloading (5) 116.4MB libllvm22
9.2s
Extracting 10
1.3sxorg-libx11
839.7kB @ 1.6MB/s 0.5s
openldap 785.6kB @ 1.9MB/s 0.4s
[+] 9.4s
Downloading (5) 117.6MB libpng
9.3s
Extracting (2) 10 openldap
1.3s[+] 9.5s
Downloading (5) 118.8MB libpng
9.4s
Extracting 12
1.4sunicodedata2
409.7kB @ 1.3MB/s 0.3s
libpng 317.7kB @ 1.1MB/s 0.2s
[+] 9.6s
Downloading (5) 120.0MB cyrus-sasl
9.5s
Extracting 14
1.4sxkeyboard-config
399.3kB @ 1.3MB/s 0.3s
[+] 9.7s
Downloading (5) 121.0MB cyrus-sasl

```

```

9.6s
Extracting      (1)                14 xkeyboard-
config          1.4scyrus-sasl
210.1kB @    1.3MB/s  0.2s
fontconfig                                270.7kB @    1.4MB/s  0.2s
libstdcxx-ng                                27.6kB @    ???.?MB/s  0.1s
libgfortran                                27.5kB @    ???.?MB/s  0.1s
[+] 9.8s
Downloading (2)                122.4MB libllvm22
9.7s
Extracting (1)                18 libgfortran
1.5s[+] 9.9s
Downloading (2)                123.5MB libllvm22
9.8s
Extracting                19
1.6s[+] 10.0s
Downloading (2)                124.7MB libllvm22
9.9s
Extracting                19
1.6s[+] 10.1s
Downloading (2)                126.1MB libllvm22
10.0s
Extracting                19
1.6s[+] 10.2s
Downloading (2)                127.3MB qt6-main
10.1s
Extracting                19
1.6s[+] 10.3s
Downloading (2)                128.7MB qt6-main
10.2s
Extracting                19
1.6s[+] 10.4s
Downloading (2)                130.1MB qt6-main
10.3s
Extracting                19
1.6s[+] 10.5s
Downloading (2)                131.3MB qt6-main
10.4s
Extracting                19
1.6s[+] 10.6s
Downloading (2)                132.1MB libllvm22
10.5s
Extracting                19
1.6s[+] 10.7s
Downloading (2)                133.6MB libllvm22
10.6s
Extracting                19
1.6s[+] 10.8s

```

Downloading (2)	134.8MB libllvm22
10.7s	
Extracting	19
1.6s[+] 10.9s	
Downloading (2)	136.2MB libllvm22
10.8s	
Extracting	19
1.6s[+] 11.0s	
Downloading (2)	137.5MB qt6-main
10.9s	
Extracting	19
1.6s[+] 11.1s	
Downloading (2)	138.7MB qt6-main
11.0s	
Extracting	19
1.6s[+] 11.2s	
Downloading (2)	139.3MB qt6-main
11.1s	
Extracting	19
1.6s[+] 11.3s	
Downloading (2)	140.3MB qt6-main
11.2s	
Extracting	19
1.6slibllvm22	
44.2MB @ 3.9MB/s 11.3s	
[+] 11.4s	
Downloading (1)	141.0MB qt6-main
11.3s	
Extracting (1)	19 libllvm22
1.6s[+] 11.5s	
Downloading (1)	141.3MB qt6-main
11.4s	
Extracting (1)	19 libllvm22
1.7s[+] 11.6s	
Downloading (1)	141.8MB qt6-main
11.5s	
Extracting (1)	19 libllvm22
1.8s[+] 11.7s	
Downloading (1)	142.3MB qt6-main
11.6s	
Extracting (1)	19 libllvm22
1.9s[+] 11.8s	
Downloading (1)	142.9MB qt6-main
11.7s	
Extracting	20
2.0s[+] 11.9s	
Downloading (1)	143.7MB qt6-main
11.8s	

Extracting	20
2.0s[+] 12.0s	
Downloading (1)	144.7MB qt6-main
11.9s	
Extracting	20
2.0s[+] 12.1s	
Downloading (1)	146.0MB qt6-main
12.0s	
Extracting	20
2.0s[+] 12.2s	
Downloading (1)	147.4MB qt6-main
12.1s	
Extracting	20
2.0s[+] 12.3s	
Downloading (1)	148.8MB qt6-main
12.2s	
Extracting	20
2.0s[+] 12.4s	
Downloading (1)	150.2MB qt6-main
12.3s	
Extracting	20
2.0s[+] 12.5s	
Downloading (1)	151.6MB qt6-main
12.4s	
Extracting	20
2.0s[+] 12.6s	
Downloading (1)	153.0MB qt6-main
12.5s	
Extracting	20
2.0s[+] 12.7s	
Downloading (1)	154.5MB qt6-main
12.6s	
Extracting	20
2.0s[+] 12.8s	
Downloading (1)	155.9MB qt6-main
12.7s	
Extracting	20
2.0s[+] 12.9s	
Downloading (1)	157.3MB qt6-main
12.8s	
Extracting	20
2.0s[+] 13.0s	
Downloading (1)	158.7MB qt6-main
12.9s	
Extracting	20
2.0s[+] 13.1s	
Downloading (1)	160.1MB qt6-main
13.0s	

Extracting	20
2.0s[+] 13.2s	
Downloading (1)	161.5MB qt6-main
13.1s	
Extracting	20
2.0s[+] 13.3s	
Downloading (1)	163.0MB qt6-main
13.2s	
Extracting	20
2.0s[+] 13.4s	
Downloading (1)	164.4MB qt6-main
13.3s	
Extracting	20
2.0s[+] 13.5s	
Downloading (1)	165.8MB qt6-main
13.4s	
Extracting	20
2.0s[+] 13.6s	
Downloading (1)	166.1MB qt6-main
13.5s	
Extracting	20
2.0s[+] 13.7s	
Downloading (1)	168.3MB qt6-main
13.6s	
Extracting	20
2.0s[+] 13.8s	
Downloading (1)	169.7MB qt6-main
13.7s	
Extracting	20
2.0s[+] 13.9s	
Downloading (1)	171.2MB qt6-main
13.8s	
Extracting	20
2.0s[+] 14.0s	
Downloading (1)	172.6MB qt6-main
13.9s	
Extracting	20
2.0s[+] 14.1s	
Downloading (1)	174.0MB qt6-main
14.0s	
Extracting	20
2.0s[+] 14.2s	
Downloading (1)	175.4MB qt6-main
14.1s	
Extracting	20
2.0s[+] 14.3s	
Downloading (1)	176.9MB qt6-main
14.2s	

Extracting	20	
2.0s[+] 14.4s		
Downloading (1)	178.3MB	qt6-main
14.3s		
Extracting	20	
2.0s[+] 14.5s		
Downloading (1)	179.2MB	qt6-main
14.4s		
Extracting	20	
2.0s[+] 14.6s		
Downloading (1)	181.0MB	qt6-main
14.5s		
Extracting	20	
2.0sqt6-main		
56.6MB @ 3.9MB/s 14.6s		
[+] 14.7s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.0s[+] 14.8s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.1s[+] 14.9s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.2s[+] 15.0s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.3s[+] 15.1s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.4s[+] 15.2s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.5s[+] 15.3s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.6s[+] 15.4s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.7s[+] 15.5s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.8s[+] 15.6s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main
2.9s[+] 15.7s		
Downloading	181.7MB	14.6s
Extracting (1)	20	qt6-main

```

3.0s[+] 15.8s
Downloading                                181.7MB                14.6s
Extracting    (1)                          20 qt6-main
3.1s[+] 15.9s
Downloading                                181.7MB                14.6s
Extracting    (1)                          20 qt6-main
3.2s[+] 16.0s
Downloading                                181.7MB                14.6s
Extracting    (1)                          20 qt6-main
3.3sLinking libgfortran5-15.2.0-h68bc16d_18
Linking libjpeg-turbo-3.1.2-hb03c661_0
Linking libwebp-base-1.6.0-hd42ef1d_0
Linking zlib-ng-2.3.3-hceb46e0_1
Linking libstdcxx-ng-15.2.0-hdf11a46_18
Linking libiconv-1.18-h3b78370_2
Linking pthread-stubs-0.4-hb9d3cd8_1002
Linking xorg-libxau-1.0.12-hb03c661_1
Linking xorg-libxdmcp-1.1.5-hb03c661_1
Linking libpng-1.6.55-h421ea60_0
Linking libdeflate-1.25-h17f619e_0
Linking libbrotlicommon-1.2.0-hb03c661_1
Linking libglvnd-1.7.0-ha4b6fd6_2
Linking alsa-lib-1.2.15.3-hb03c661_0
Linking double-conversion-3.4.0-hecca717_0
Linking pcre2-10.47-haa7fec5_0
Linking wayland-1.24.0-hd6090a7_1
Linking xorg-libice-1.1.2-hb9d3cd8_0
Linking keyutils-1.6.3-hb9d3cd8_0
Linking libedit-3.1.20250104-pl5321h7949ede_0
Linking lerc-4.0.0-h0aef613_1
Linking graphite2-1.3.14-hecca717_2
Linking pixman-0.46.4-h54a6638_1
Linking libntlm-1.8-hb9d3cd8_0
Linking libpciaccess-0.18-hb9d3cd8_0
Linking libgfortran-15.2.0-h69a702a_18
Linking qhull-2020.2-h434a139_5
Linking libxml2-16-2.15.1-hca6bf5a_1
Linking libxcb-1.17.0-h8a09558_0
Linking libfreetype6-2.14.1-h73754d4_0
Linking libbrotlienc-1.2.0-hb03c661_1
Linking libbrotldec-1.2.0-hb03c661_1
Linking libopengl-1.7.0-ha4b6fd6_2
Linking libegl-1.7.0-ha4b6fd6_2
Linking libglib-2.86.4-h6548e54_1
Linking xorg-libsm-1.2.6-he73a12e_0
Linking krb5-1.22.2-ha1258a1_0
Linking libtiff-4.7.1-h9d88235_1
Linking libdrm-2.4.125-hb03c661_1

```

Linking libopenblas-0.3.30-pthreads_h94d23a6_4
Linking libxml2-2.15.1-he237659_1
Linking xcb-util-0.4.1-h4f16b4b_2
Linking xcb-util-wm-0.4.2-hb711507_0
Linking xcb-util-renderutil-0.3.10-hb711507_0
Linking xcb-util-keysyms-0.4.1-hb711507_0
Linking xorg-libx11-1.8.13-he1eb515_0
Linking libfreetype-2.14.1-ha770c72_0
Linking brotli-bin-1.2.0-hb03c661_1
Linking dbus-1.16.2-h24cb091_1
Linking cyrus-sasl-2.1.28-hac629b4_1
Linking libcups-2.3.3-h7a8fb5f_6
Linking openjpeg-2.5.4-h55fea9a_0
Linking lcms2-2.18-h0c24ade_0
Linking libblas-3.11.0-5_h4a7cf45_openblas
Linking libllvm22-22.1.0-hf7376ad_0
Linking libxslt-1.1.43-h711ed8c_1
Linking xcb-util-image-0.4.0-hb711507_2
Linking xkeyboard-config-2.47-hb03c661_0
Linking xorg-libxfixes-6.0.2-hb03c661_0
Linking libglx-1.7.0-ha4b6fd6_2
Linking xorg-libxext-1.3.7-hb03c661_0
Linking xorg-libxrender-0.9.12-hb9d3cd8_0
Linking fontconfig-2.17.1-h27c8c51_0
Linking freetype-2.14.1-ha770c72_0
Linking brotli-1.2.0-hed03a55_1
Linking openldap-2.6.10-hbde042b_1
Linking libcbblas-3.11.0-5_h0358290_openblas
Linking liblapack-3.11.0-5_h47877c9_openblas
Linking libclang-cpp22.1-22.1.0-default_h99862b1_0
Linking libclang13-22.1.0-default_h746c552_0
Linking xcb-util-cursor-0.1.6-hb03c661_0
Linking libxkbcommon-1.13.1-hca5e8e5_0
Linking xorg-libxcomposite-0.4.7-hb03c661_0
Linking libgl-1.7.0-ha4b6fd6_2
Linking xorg-libxi-1.8.2-hb9d3cd8_0
Linking xorg-libxxf86vm-1.1.7-hb03c661_0
Linking xorg-libxdamage-1.1.6-hb9d3cd8_0
Linking xorg-libxcursor-1.2.3-hb9d3cd8_0
Linking xorg-libxrandr-1.5.5-hb03c661_0
Linking libpq-18.3-h9abb657_0
Linking xorg-libxtst-1.2.5-hb9d3cd8_3
Linking libvulkan-loader-1.4.341.0-h5279c79_0
Linking python_abi-3.11-8_cp311
Linking cyciler-0.12.1-pyhcf101f3_2
Linking pyparsing-3.3.2-pyhcf101f3_0
Linking munkres-1.1.4-pyhd8ed1ab_1
Linking six-1.17.0-pyhe01879c_1


```

Linking font-ttf-dejavu-sans-mono-2.37-hab24e00_0
Linking font-ttf-ubuntu-0.83-h77eed37_3
Linking font-ttf-inconsolata-3.000-h77eed37_0
Linking font-ttf-source-code-pro-2.038-h77eed37_0
Linking python-dateutil-2.9.0.post0-pyhe01879c_2
Linking fonts-conda-forge-1-hc364b38_1
Linking fonts-conda-ecosystem-1-0
Linking unicodedata2-17.0.1-py311h49ec1c0_0
Linking tornado-6.5.4-py311h41d9c34_0
Linking pillow-12.1.1-py311hf88fc01_0
Linking kiwisolver-1.4.9-py311h724c32c_2
Linking numpy-2.4.2-py311h2e04523_1
Linking cairo-1.18.4-he90730b_1
Linking fonttools-4.61.1-py311h3778330_0
Linking contourpy-1.3.3-py311h724c32c_4
Linking scipy-1.17.1-py311hbe70eeb_0
Linking harfbuzz-12.3.2-h6083320_0
Linking matplotlib-base-3.10.8-py311h0f3be63_0
Linking qt6-main-6.10.2-h17e89b9_5
Linking pyside6-6.10.2-py311he4c1a5a_0
Linking matplotlib-3.10.8-py311h38be061_0

```

Transaction finished

2. numba (for JIT compilation)

```
[7]: !mamba install --name nsc2026 \N
      numba -y
```

```
conda-forge/linux-64
conda-forge/noarch
```

```
Using cache
Using cache
```

Pinned packages:

```
- python=3.11
```

Transaction

Prefix: /opt/conda/envs/nsc2026

Updating specs:

```
- numba
```

Package	Version	Build	Channel	Size
---------	---------	-------	---------	------

Install:

+ llvmlite	0.46.0	py311h41a00d4_0	conda-forge	Cached
+ numba	0.63.1	py311h3c884d5_0	conda-forge	Cached

Downgrade:

- numpy	2.4.2	py311h2e04523_1	conda-forge	Cached
+ numpy	2.3.5	py311h2e04523_1	conda-forge	Cached

Summary:

Install: 2 packages

Downgrade: 1 packages

Total download: 0 B

Transaction starting

[+] 0.0s

Unlinking numpy-2.4.2-py311h2e04523_1

Linking llvmlite-0.46.0-py311h41a00d4_0

Linking numpy-2.3.5-py311h2e04523_1

Linking numba-0.63.1-py311h3c884d5_0

Transaction finished

3. pytest, pytest-cov (for testing)

```
[8]: !mamba install --name nsc2026 \N  
      pytest pytest-cov -y
```

conda-forge/linux-64

Using cache

conda-forge/noarch

Using cache

Pinned packages:

- python=3.11

Transaction

Prefix: /opt/conda/envs/nsc2026

Updating specs:

- pytest
- pytest-cov

Package	Version	Build	Channel	Size
---------	---------	-------	---------	------

Install:

+ colorama	0.4.6	pyhd8ed1ab_1	conda-forge	
Cached				
+ coverage	7.13.4	py311h3778330_0	conda-forge	398kB
+ exceptiongroup	1.3.1	pyhd8ed1ab_0	conda-forge	
Cached				
+ iniconfig	2.3.0	pyhd8ed1ab_0	conda-forge	
Cached				
+ pluggy	1.6.0	pyhf9edf01_1	conda-forge	
Cached				
+ pygments	2.19.2	pyhd8ed1ab_0	conda-forge	
Cached				
+ pytest	9.0.2	pyhcf101f3_0	conda-forge	
Cached				
+ pytest-cov	7.0.0	pyhcf101f3_1	conda-forge	29kB
+ tomli	2.4.0	pyhcf101f3_0	conda-forge	
Cached				
+ typing_extensions	4.15.0	pyhcf101f3_0	conda-forge	
Cached				

Summary:

Install: 10 packages

Total download: 427kB

Transaction starting
[+] 0.0s

```

Downloading                                0.0 B
0.0s
Extracting                                0
0.0s[+] 0.1s
Downloading (2)                            0.0 B coverage
0.0s
Extracting                                0
0.0spytest-cov
29.0kB @ 101.5kB/s 0.1s
[+] 0.2s
Downloading (1)                            29.0kB coverage
0.1s
Extracting                                1
0.0scoverage
398.0kB @ 70.5kB/s 0.2s
Linking pluggy-1.6.0-pyhf9edf01_1
Linking colorama-0.4.6-pyhd8ed1ab_1
Linking iniconfig-2.3.0-pyhd8ed1ab_0
Linking pygments-2.19.2-pyhd8ed1ab_0
Linking tomli-2.4.0-pyhcf101f3_0
Linking typing_extensions-4.15.0-pyhcf101f3_0
Linking exceptiongroup-1.3.1-pyhd8ed1ab_0
Linking pytest-9.0.2-pyhcf101f3_0
Linking coverage-7.13.4-py311h3778330_0
Linking pytest-cov-7.0.0-pyhcf101f3_1

```

Transaction finished

4. dask, distributed (for parallel computing)

```
[9]: !mamba install --name nsc2026 \N
      dask -y
```

```
conda-forge/linux-64
conda-forge/noarch
```

```
Using cache
Using cache
```

Pinned packages:

```
- python=3.11
```

Transaction

```
Prefix: /opt/conda/envs/nsc2026
```

Updating specs:

```
- dask
```

Package Size	Version	Build	Channel
-----------------	---------	-------	---------

Install:

+ aws-c-auth	0.9.6	hb9c0fe4_1	
conda-forge 133kB			
+ aws-c-cal	0.9.13	h2c9d079_1	
conda-forge Cached			
+ aws-c-common	0.12.6	hb03c661_0	
conda-forge Cached			
+ aws-c-compression	0.3.2	h8b1a151_0	
conda-forge 22kB			
+ aws-c-event-stream	0.5.9	h841be55_2	
conda-forge 59kB			
+ aws-c-http	0.10.10	hf621c6d_0	
conda-forge 226kB			
+ aws-c-io	0.26.1	h3ca20c3_1	
conda-forge 182kB			
+ aws-c-mqtt	0.14.0	ha25ca29_1	
conda-forge 220kB			
+ aws-c-s3	0.11.5	h9b5df67_3	
conda-forge 151kB			
+ aws-c-sdkutils	0.2.4	h8b1a151_4	
conda-forge Cached			
+ aws-checksums	0.2.10	h8b1a151_0	
conda-forge 101kB			
+ aws-crt-cpp	0.37.3	hb153662_0	
conda-forge 410kB			
+ aws-sdk-cpp	1.11.747	h133b1ee_1	
conda-forge 4MB			
+ azure-core-cpp	1.16.2	h206d751_0	
conda-forge Cached			
+ azure-identity-cpp	1.13.3	hed0cdb0_1	
conda-forge Cached			
+ azure-storage-blobs-cpp	12.16.0	hdd73cc9_1	
conda-forge Cached			
+ azure-storage-common-cpp	12.12.0	ha7a2c86_1	
conda-forge Cached			
+ azure-storage-files-datalake-cpp	12.14.0	h52c5a47_1	
conda-forge Cached			
+ backports.zstd	1.3.0	py311h6b1f9c4_0	
conda-forge Cached			

+ bokeh		3.8.2	pyhd8ed1ab_0
conda-forge	Cached		
+ brotli-python		1.2.0	py311h66f275b_1
conda-forge	Cached		
+ c-ares		1.34.6	hb03c661_0
conda-forge	Cached		
+ click		8.3.1	pyh8f84b5b_1
conda-forge	Cached		
+ cloudpickle		3.1.2	pyhcf101f3_1
conda-forge	Cached		
+ cytoolz		1.1.0	py311h49ec1c0_2
conda-forge	624kB		
+ dask		2026.1.2	pyhcf101f3_1
conda-forge	11kB		
+ dask-core		2026.1.2	pyhcf101f3_0
conda-forge	Cached		
+ distributed		2026.1.2	pyhcf101f3_1
conda-forge	845kB		
+ fsspec		2026.2.0	pyhd8ed1ab_0
conda-forge	Cached		
+ gflags		2.2.2	h5888daf_1005
conda-forge	Cached		
+ glog		0.7.1	hbabe93e_0
conda-forge	Cached		
+ h2		4.3.0	pyhcf101f3_0
conda-forge	Cached		
+ hpack		4.1.0	pyhd8ed1ab_0
conda-forge	Cached		
+ hyperframe		6.1.0	pyhd8ed1ab_0
conda-forge	Cached		
+ importlib-metadata		8.7.0	pyhe01879c_1
conda-forge	Cached		
+ jinja2		3.1.6	pyhcf101f3_1
conda-forge	Cached		
+ libabseil		20260107.1	cxx17_h7b12aa8_0
conda-forge	1MB		
+ libarrow		23.0.1	h711ef25_3_cpu
conda-forge	6MB		
+ libarrow-acero		23.0.1	h635bf11_3_cpu
conda-forge	609kB		
+ libarrow-compute		23.0.1	h53684a4_3_cpu
conda-forge	3MB		
+ libarrow-dataset		23.0.1	h635bf11_3_cpu
conda-forge	608kB		
+ libarrow-substrait		23.0.1	hb4dd7c2_3_cpu
conda-forge	519kB		
+ libcrc32c		1.1.2	h9c3ff4c_0
conda-forge	Cached		

+ libcurl		8.18.0	hcf29cc6_1
conda-forge	464kB		
+ libev		4.33	hd590300_2
conda-forge	Cached		
+ libevent		2.1.12	hf998b51_1
conda-forge	Cached		
+ libgoogle-cloud		2.39.0	h9d11ab5_1
conda-forge	1MB		
+ libgoogle-cloud-storage		2.39.0	hdbdcf42_1
conda-forge	803kB		
+ libgrpc		1.78.0	h1d1128b_1
conda-forge	7MB		
+ libnghttp2		1.67.0	had1ee68_0
conda-forge	Cached		
+ libopentelemetry-cpp		1.21.0	h9692893_2
conda-forge	897kB		
+ libopentelemetry-cpp-headers		1.21.0	ha770c72_2
conda-forge	362kB		
+ libparquet		23.0.1	h7376487_3_cpu
conda-forge	1MB		
+ libprotobuf		6.33.5	h2b00c02_0
conda-forge	4MB		
+ libre2-11		2025.11.05	h0dc7533_1
conda-forge	213kB		
+ libssh2		1.11.1	hcf80075_0
conda-forge	Cached		
+ libthrift		0.22.0	h454ac66_1
conda-forge	Cached		
+ libutf8proc		2.11.3	hfe17d71_0
conda-forge	Cached		
+ locket		1.0.0	pyhd8ed1ab_0
conda-forge	Cached		
+ lz4		4.4.5	py311h1c460e0_1
conda-forge	Cached		
+ lz4-c		1.10.0	h5888daf_1
conda-forge	Cached		
+ markupsafe		3.0.3	py311h3778330_1
conda-forge	27kB		
+ msgpack-python		1.1.2	py311hdf67eae_1
conda-forge	Cached		
+ narwhals		2.17.0	pyhcf101f3_0
conda-forge	280kB		
+ nlohmann_json		3.12.0	h54a6638_1
conda-forge	Cached		
+ orc		2.2.2	hbb90d81_1
conda-forge	1MB		
+ pandas		3.0.1	py311h8032f78_0
conda-forge	15MB		

+ partd		1.4.2	pyhd8ed1ab_0
conda-forge	Cached		
+ prometheus-cpp		1.3.0	ha5d0236_0
conda-forge	Cached		
+ psutil		7.2.2	py311haee01d2_0
conda-forge	Cached		
+ pyarrow		23.0.1	py311h38be061_0
conda-forge	29kB		
+ pyarrow-core		23.0.1	py311h66caee6_0_cpu
conda-forge	5MB		
+ pysocks		1.7.1	pyha55dd90_7
conda-forge	Cached		
+ pyyaml		6.0.3	py311h3778330_1
conda-forge	Cached		
+ re2		2025.11.05	h5301d42_1
conda-forge	27kB		
+ s2n		1.7.0	ha63dd3a_1
conda-forge	398kB		
+ snappy		1.2.2	h03e3b7b_1
conda-forge	Cached		
+ sortedcontainers		2.4.0	pyhd8ed1ab_1
conda-forge	Cached		
+ tblib		3.2.2	pyhcf101f3_0
conda-forge	Cached		
+ toolz		1.1.0	pyhd8ed1ab_1
conda-forge	Cached		
+ urllib3		2.6.3	pyhd8ed1ab_0
conda-forge	Cached		
+ xyzservices		2025.11.0	pyhd8ed1ab_0
conda-forge	Cached		
+ yaml		0.2.5	h280c20c_3
conda-forge	Cached		
+ zict		3.0.0	pyhd8ed1ab_1
conda-forge	Cached		
+ zipp		3.23.0	pyhcf101f3_1
conda-forge	Cached		
+ zlib		1.3.1	hb9d3cd8_2
conda-forge	Cached		

Summary:

Install: 86 packages

Total download: 58MB


```

Transaction starting
[+] 0.0s
Downloading 0.0 B
0.0s
Extracting 0
0.0s[+] 0.1s
Downloading (5) 0.0 B libarrow
0.0s
Extracting 0
0.0s[+] 0.2s
Downloading (5) 220.2kB libarrow
0.1s
Extracting 0
0.0s[+] 0.3s
Downloading (5) 1.6MB libarrow
0.2s
Extracting 0
0.0s[+] 0.4s
Downloading (5) 1.8MB libarrow
0.3s
Extracting 0
0.0s[+] 0.5s
Downloading (5) 3.7MB libgrpc
0.4s
Extracting 0
0.0s[+] 0.6s
Downloading (5) 5.2MB libgrpc
0.5s
Extracting 0
0.0s[+] 0.7s
Downloading (5) 7.0MB libgrpc
0.6s
Extracting 0
0.0s[+] 0.8s
Downloading (5) 8.6MB libgrpc
0.7s
Extracting 0
0.0s[+] 0.9s
Downloading (5) 10.0MB libprotobuf
0.8s
Extracting 0
0.0s[+] 1.0s
Downloading (5) 11.4MB libprotobuf
0.9s
Extracting 0
0.0s[+] 1.1s

```

Downloading (5)	12.8MB	libprotobuf	
1.0s			
Extracting	0		
0.0s			
libarrow			
6.5MB @ 5.6MB/s			1.1s
libprotobuf			
[+] 1.2s			
Downloading (5)	14.2MB	aws-sdk-cpp	
1.1s			
Extracting (2)	0	libarrow	
0.0s			
[+] 1.3s			
Downloading (5)	15.7MB	aws-sdk-cpp	
1.2s			
Extracting	2		
0.1s			
[+] 1.4s			
Downloading (5)	16.7MB	aws-sdk-cpp	
1.3s			
Extracting	2		
0.1s			
[+] 1.5s			
Downloading (5)	18.5MB	aws-sdk-cpp	
1.4s			
Extracting	2		
0.1s			
[+] 1.6s			
Downloading (5)	19.9MB	libarrow-compute	
1.5s			
Extracting	2		
0.1s			
[+] 1.7s			
Downloading (5)	21.1MB	libarrow-compute	
1.6s			
Extracting	2		
0.1s			
aws-sdk-cpp			
3.6MB @ 6.0MB/s			0.6s
[+] 1.8s			
Downloading (5)	22.6MB	libarrow-compute	
1.7s			
Extracting (1)	2	aws-sdk-cpp	
0.1s			
[+] 1.9s			
Downloading (5)	24.0MB	libarrow-compute	
1.8s			
Extracting (1)	2	aws-sdk-cpp	
0.2s			
[+] 2.0s			
Downloading (5)	24.6MB	libgrpc	
1.9s			
Extracting	3		
0.3s			
libparquet			
1.4MB @ 4.6MB/s			0.3s
[+] 2.1s			
Downloading (5)	26.5MB	libgrpc	

```

2.0s
Extracting                                4
0.3slibarrow-compute
3.0MB @ 3.2MB/s 0.9s
[+] 2.2s
Downloading (5)                          28.1MB libgrpc
2.1s
Extracting (1)                            4 libarrow-
compute 0.3s[+] 2.3s
Downloading (5)                          29.0MB libgrpc
2.2s
Extracting                                5
0.4slibabseil
1.4MB @ 4.3MB/s 0.3s
[+] 2.4s
Downloading (5)                          30.6MB orc
2.3s
Extracting (1)                            5 libabseil
0.4s[+] 2.5s
Downloading (5)                          31.6MB orc
2.4s
Extracting                                6
0.5sorc
1.3MB @ 2.6MB/s 0.5s
[+] 2.6s
Downloading (4)                          32.9MB libgoogle-cloud
2.5s
Extracting                                6
0.5slibgoogle-cloud
1.3MB @ 4.3MB/s 0.3s
[+] 2.7s
Downloading (5)                          34.2MB distributed
2.6s
Extracting                                8
0.5s[+] 2.8s
Downloading (5)                          35.4MB distributed
2.7s
Extracting                                8
0.5sdistributed
844.8kB @ 2.5MB/s 0.3s
[+] 2.9s
Downloading (5)                          36.5MB libgoogle-cloud-
storage 2.8s
Extracting (1)                            8 distributed
0.5slibopentelemetry-cpp
896.6kB @ 2.4MB/s 0.3s
[+] 3.0s
Downloading (5)                          38.1MB libgoogle-cloud-

```

```

storage      2.9s
Extracting                                10
0.6s[+] 3.1s
Downloading (5)                          39.4MB libgoogle-cloud-
storage      3.0s
Extracting                                10
0.6s[+] 3.2s
Downloading (5)                          40.6MB libgoogle-cloud-
storage      3.1s
Extracting                                10
0.6scopytoolz
623.8kB @ 1.7MB/s 0.3s
[+] 3.3s
Downloading (5)                          41.8MB libgrpc
3.2s
Extracting                                11
0.6slibgrpc
7.0MB @ 2.1MB/s 3.3s
libgoogle-cloud-storage                  803.5kB @ 1.7MB/s 0.5s
[+] 3.4s
Downloading (5)                          43.1MB libarrow-acero
3.3s
Extracting (2)                            11 libgoogle-
cloud-storage      0.6slibarrow-acero
609.4kB @ 2.3MB/s 0.3s
[+] 3.5s
Downloading (5)                          44.2MB libarrow-dataset
3.4s
Extracting (1)                            13 libarrow-
acero      0.7slibarrow-substrait
518.5kB @ 1.9MB/s 0.2s
[+] 3.6s
Downloading (5)                          45.4MB libarrow-dataset
3.5s
Extracting                                15
0.8s[+] 3.7s
Downloading (5)                          46.4MB libarrow-dataset
3.6s
Extracting                                15
0.8slibcurl
463.6kB @ 1.2MB/s 0.2s
aws-crt-cpp                             410.1kB @ 2.5MB/s 0.2s
[+] 3.8s
Downloading (5)                          47.7MB libarrow-dataset
3.7s
Extracting                                17
0.8ss2n
397.8kB @ 1.5MB/s 0.1s

```

libarrow-dataset	608.1kB @	1.2MB/s	0.5s
libopentelemetry-cpp-headers	362.2kB @	1.9MB/s	0.2s
[+] 3.9s			
Downloading (5)	49.0MB	aws-c-http	
3.8s			
Extracting (1)	19		
libopentelemetry-cpp-headers	0.8snarwhals		
280.4kB @	2.0MB/s	0.1s	
aws-c-http	225.7kB @	1.4MB/s	0.1s
[+] 4.0s			
Downloading (5)	50.2MB	aws-c-io	
3.9s			
Extracting (2)	20	aws-c-http	
0.9slibre2-11			
213.1kB @	???.?MB/s	0.1s	
aws-c-mqtt	220.0kB @	600.6kB/s	0.2s
[+] 4.1s			
Downloading (5)	51.4MB	aws-c-io	
4.0s			
Extracting (1)	23	aws-c-mqtt	
1.0saws-c-io			
181.6kB @	849.2kB/s	0.1s	
aws-c-s3	151.4kB @	???.?MB/s	0.1s
aws-c-auth	133.5kB @	1.2MB/s	0.1s
[+] 4.2s			
Downloading (5)	52.5MB	aws-c-event-stream	
4.1s			
Extracting (1)	26	aws-c-auth	
1.1saws-checksums			
101.4kB @	685.1kB/s	0.1s	
aws-c-event-stream	58.8kB @	???.?MB/s	0.1s
pyarrow	28.6kB @	???.?MB/s	0.1s
re2	27.5kB @	???.?MB/s	0.1s
markupsafe	26.8kB @	???.?MB/s	0.1s
[+] 4.3s			
Downloading (4)	53.9MB	aws-c-compression	
4.2s			
Extracting (3)	29	markupsafe	
1.2saws-c-compression			
22.3kB @	???.?MB/s	0.1s	
dask	11.4kB @	???.?MB/s	0.1s
[+] 4.4s			
Downloading (2)	54.3MB	pandas	
4.3s			
Extracting	34		
1.3s[+] 4.5s			
Downloading (2)	55.2MB	pandas	
4.4s			

```

Extracting                                34
1.3spyarrow-core
5.3MB @ 1.1MB/s 4.5s
[+] 4.6s
Downloading (1)                          56.5MB pandas
4.5s
Extracting (1)                            34 pyarrow-core
1.3s[+] 4.7s
Downloading (1)                          57.0MB pandas
4.6s
Extracting (1)                            34 pyarrow-core
1.4spandas
15.1MB @ 3.2MB/s 4.7s
[+] 4.8s
Downloading                                57.8MB                4.7s
Extracting (1)                            35 pandas
1.5s[+] 4.9s
Downloading                                57.8MB                4.7s
Extracting (1)                            35 pandas
1.6s[+] 5.0s
Downloading                                57.8MB                4.7s
Extracting (1)                            35 pandas
1.7s[+] 5.1s
Downloading                                57.8MB                4.7s
Extracting (1)                            35 pandas
1.8sLinking click-8.3.1-pyh8f84b5b_1
Linking cloudpickle-3.1.2-pyhcf101f3_1
Linking fsspec-2026.2.0-pyhd8ed1ab_0
Linking toolz-1.1.0-pyhd8ed1ab_1
Linking locket-1.0.0-pyhd8ed1ab_0
Linking narwhals-2.17.0-pyhcf101f3_0
Linking xyzservices-2025.11.0-pyhd8ed1ab_0
Linking zipp-3.23.0-pyhcf101f3_1
Linking sortedcontainers-2.4.0-pyhd8ed1ab_1
Linking tblib-3.2.2-pyhcf101f3_0
Linking zict-3.0.0-pyhd8ed1ab_1
Linking hpack-4.1.0-pyhd8ed1ab_0
Linking hyperframe-6.1.0-pyhd8ed1ab_0
Linking pysocks-1.7.1-pyha55dd90_7
Linking partd-1.4.2-pyhd8ed1ab_0
Linking importlib-metadata-8.7.0-pyhe01879c_1
Linking h2-4.3.0-pyhcf101f3_0
Linking pandas-3.0.1-py311h8032f78_0
Linking yaml-0.2.5-h280c20c_3
Linking libabseil-20260107.1-cxx17_h7b12aa8_0
Linking aws-c-common-0.12.6-hb03c661_0
Linking libutf8proc-2.11.3-hfe17d71_0
Linking libopentelemetry-cpp-headers-1.21.0-ha770c72_2

```

Linking zlib-1.3.1-hb9d3cd8_2
Linking libcrc32c-1.1.2-h9c3ff4c_0
Linking lz4-c-1.10.0-h5888daf_1
Linking snappy-1.2.2-h03e3b7b_1
Linking c-ares-1.34.6-hb03c661_0
Linking nlohmann_json-3.12.0-h54a6638_1
Linking libev-4.33-hd590300_2
Linking gflags-2.2.2-h5888daf_1005
Linking s2n-1.7.0-ha63dd3a_1
Linking markupsafe-3.0.3-py311h3778330_1
Linking msgpack-python-1.1.2-py311hdf67eae_1
Linking psutil-7.2.2-py311haee01d2_0
Linking brotli-python-1.2.0-py311h66f275b_1
Linking libevent-2.1.12-hf998b51_1
Linking backports.zstd-1.3.0-py311h6b1f9c4_0
Linking libssh2-1.11.1-hcf80075_0
Linking cytoolz-1.1.0-py311h49ec1c0_2
Linking pyyaml-6.0.3-py311h3778330_1
Linking libre2-11-2025.11.05-h0dc7533_1
Linking libprotobuf-6.33.5-h2b00c02_0
Linking aws-checksums-0.2.10-h8b1a151_0
Linking aws-c-cal-0.9.13-h2c9d079_1
Linking aws-c-sdkutils-0.2.4-h8b1a151_4
Linking aws-c-compression-0.3.2-h8b1a151_0
Linking lz4-4.4.5-py311h1c460e0_1
Linking libnghttp2-1.67.0-had1ee68_0
Linking glog-0.7.1-hbabe93e_0
Linking libthrift-0.22.0-h454ac66_1
Linking re2-2025.11.05-h5301d42_1
Linking orc-2.2.2-hbb90d81_1
Linking aws-c-io-0.26.1-h3ca20c3_1
Linking libcurl-8.18.0-hcf29cc6_1
Linking libgrpc-1.78.0-h1d1128b_1
Linking aws-c-event-stream-0.5.9-h841be55_2
Linking aws-c-http-0.10.10-hf621c6d_0
Linking prometheus-cpp-1.3.0-ha5d0236_0
Linking azure-core-cpp-1.16.2-h206d751_0
Linking libgoogle-cloud-2.39.0-h9d11ab5_1
Linking aws-c-mqtt-0.14.0-ha25ca29_1
Linking aws-c-auth-0.9.6-hb9c0fe4_1
Linking libopentelemetry-cpp-1.21.0-h9692893_2
Linking azure-storage-common-cpp-12.12.0-ha7a2c86_1
Linking azure-identity-cpp-1.13.3-hed0cdb0_1
Linking libgoogle-cloud-storage-2.39.0-hdbdcf42_1
Linking aws-c-s3-0.11.5-h9b5df67_3
Linking azure-storage-blobs-cpp-12.16.0-hdd73cc9_1
Linking aws-crt-cpp-0.37.3-hb153662_0
Linking azure-storage-files-datalake-cpp-12.14.0-h52c5a47_1

```

Linking aws-sdk-cpp-1.11.747-h133b1ee_1
Linking libarrow-23.0.1-h711ef25_3_cpu
Linking libarrow-compute-23.0.1-h53684a4_3_cpu
Linking libparquet-23.0.1-h7376487_3_cpu
Linking libarrow-acero-23.0.1-h635bf11_3_cpu
Linking pyarrow-core-23.0.1-py311h66caee6_0_cpu
Linking libarrow-dataset-23.0.1-h635bf11_3_cpu
Linking libarrow-substrait-23.0.1-hb4dd7c2_3_cpu
Linking pyarrow-23.0.1-py311h38be061_0
Linking jinja2-3.1.6-pyhcf101f3_1
Linking urllib3-2.6.3-pyhd8ed1ab_0
Linking dask-core-2026.1.2-pyhcf101f3_0
Linking bokeh-3.8.2-pyhd8ed1ab_0
Linking distributed-2026.1.2-pyhcf101f3_1
Linking dask-2026.1.2-pyhcf101f3_1

```

Transaction finished

5. Handle any installation errors (ask for help!)

One error: mamba expects to be part of the shell environment. One can choose to use the `--name` option instead.

******Step 3:** Test installation (10 min)** 1. Create test script: test_install.py**

```
[10]: !touch test_install.py
```

2. Import all packages, print versions

```
[11]: %pycat test_install.py

import numpy
import matplotlib
import scipy
import numba
import pytest
import dask

print(f"{numpy.__version__}")
print(f"{matplotlib.__version__}")
print(f"{scipy.__version__}")
print(f"{numba.__version__}")
print(f"{pytest.__version__}")
print(f"{dask.__version__}")

```

3. Run script, verify no errors

```
[12]: !mamba run --name nsc2026 \
      python test_install.py
```



```
Traceback (most recent call last):
  File "/ncs/test_install.py", line 2, in <module>
    import matplotlib
  File "/opt/conda/envs/nsc2026/lib/python3.11/site-
packages/matplotlib/__init__.py", line 1299, in <module>
    rcParams['backend'] = os.environ.get('MPLBACKEND')
    ~~~~~~^~~~~~
  File "/opt/conda/envs/nsc2026/lib/python3.11/site-
packages/matplotlib/__init__.py", line 774, in __setitem__
    raise ValueError(f"Key {key}: {ve}") from None
ValueError: Key backend: 'module://matplotlib_inline.backend_inline' is not a
valid value for backend; supported values are ['gtk3agg', 'gtk3cairo',
'gtk4agg', 'gtk4cairo', 'macosx', 'nbagg', 'notebook', 'qtagg', 'qtcairo',
'qt5agg', 'qt5cairo', 'tkagg', 'tkcairo', 'webagg', 'wx', 'wxagg', 'wxcairo',
'agg', 'cairo', 'pdf', 'pgf', 'ps', 'svg', 'template']
```

[Matplotlib Docs](#)

We only get this error because we are using a very simple shell inside a notebook. The command would work in any other normal shell. Therefore, tell Matplotlib to use a *very* simple backend such as `agg`

```
[13]: !MPLBACKEND=agg \
mamba run --name nsc2026 \
python test_install.py
```

```
numpy.__version__='2.3.5'
matplotlib.__version__='3.10.8'
scipy.__version__='1.17.1'
numba.__version__='0.63.1'
pytest.__version__='9.0.2'
dask.__version__='2026.1.2'
```

4. Commit to Git later!

Noted!

******Step 4:** Export environment (5 min)**** 1. Save for reproducibility:

Noted!

2. `mamba env export > environment.yml`

```
[14]: !mamba env --name nsc2026 export > environment.yml
```

```
[15]: !head --lines 10 environment.yml
```

```
name: nsc2026
channels:
  - conda-forge
dependencies:
  - _openmp_mutex=4.5=20_gnu
```

- alsa-lib=1.2.15.3=hb03c661_0
- aws-c-auth=0.9.6=hb9c0fe4_1
- aws-c-cal=0.9.13=h2c9d079_1
- aws-c-common=0.12.6=hb03c661_0
- aws-c-compression=0.3.2=h8b1a151_0

Note that I am going to use the environment that I have configured via my own devcontainer instead. In other words, I will be using 'nsc' (included in the devcontainer) over 'nsc2026'

```
[16]: !diff --side-by-side --suppress-common-lines environment.yml environment.yml
      | head --lines 20
```

```
name: nsc2026                                | name: nsc
  - _openmp_mutex=4.5=20_gnu                  | -
_libgcc_mutex=0.1=conda_forge                > -
                                              |
_openmp_mutex=4.5=2_gnu                      | - aws-c-
  - aws-c-auth=0.9.6=hb9c0fe4_1              | - aws-c-
auth=0.9.3=hef928c7_0                        | - aws-c-
  - aws-c-compression=0.3.2=h8b1a151_0       | - aws-c-event-
compression=0.3.1=h8b1a151_9                | - aws-c-
  - aws-c-event-stream=0.5.9=h841be55_2      | - aws-c-
stream=0.5.7=h28f887f_1                     | - aws-c-
  - aws-c-http=0.10.10=hf621c6d_0            | - aws-c-
http=0.10.7=ha8fc4e3_5                      | - aws-c-
  - aws-c-io=0.26.1=h3ca20c3_1               | - aws-c-
io=0.23.3=hda65f42_5                       | - aws-c-
  - aws-c-mqtt=0.14.0=ha25ca29_1             | - aws-
mqtt=0.13.3=hc63082f_11                    | - aws-
  - aws-c-s3=0.11.5=h9b5df67_3               | - aws-
c-s3=0.11.3=h06ab39a_1                     | - aws-
  - aws-checksums=0.2.10=h8b1a151_0          | - aws-crt-
checksums=0.2.7=h8b1a151_5                  | - aws-sdk-
  - aws-crt-cpp=0.37.3=hb153662_0            | -
cpp=0.35.4=h8824e59_0                      | -
  - aws-sdk-cpp=1.11.747=h133b1ee_1          | -
cpp=1.11.606=h20b40b1_10                   | -
  - bzip2=1.0.8=hda65f42_9                   | -
bzip2=1.0.8=hda65f42_8                     | - ca-
  - ca-certificates=2026.2.25=hbd8a1cb_0     | - ca-
certificates=2026.1.4=hbd8a1cb_0            <
  - coverage=7.13.4=py311h3778330_0         | - cyrus-
  - cyrus-sasl=2.1.28=hac629b4_1              | -
sasl=2.1.28=hd9c7081_0                     | -
  - cytoolz=1.1.0=py311h49ec1c0_2            | -
cytoolz=1.1.0=py311h49ec1c0_1              | -
  - dask=2026.1.2=pyhcf101f3_1               | -
dask=2026.1.2=pyhcf101f3_0                 | -
  - distributed=2026.1.2=pyhcf101f3_1        | -
```

distributed=2026.1.2=pyhcf101f3_0

yaml.cc

Note that the `.yaml` is the official file extension of yaml files not `.yml`, but there are no functional differences.

3. (or `conda env export > environment.yml`)

I am not using conda, so I will not be running this.

4. Save this file to commit to Git later!

Noted! It is already on disk.

```
[17]: !ls environment.yml
```

environment.yml

1.1.2 ****Studio Session 2:** Git Hands-On (50 min)**

Note: that since some of these steps can only be done once, I have just copied the command and output from my own machine.

****Task:** Create Git repository for mini-project**

****Step 1:** Initialize repository (5 min)** 1. Create folder: `mandelbrot-nsc`

I am already working in the right folder, but otherwise, use `mkdir mandelbrot-nsc`.

2. `cd mandelbrot-nsc`

Again, I am already in the right directory, so no need for `cd`.

3. `git init`

The default branch has changed from “master” to “main” on Github but not git, so we must ourselves name the branch “main”.

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git init --initial-branch=main
```

Initialized empty Git repository in `/home/mikkel/Sync/AAU/Semester 8/github/nsc-mikkel/.git/`

4. Verify: `git status`

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git status
```

On branch main

No commits yet

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
.devcontainer/
.gitignore
environment.yml
```

```
environment.yml
101_exercises.ipynb
test_install.py
```

nothing added to commit but untracked files present (use "git add" to track)

******Step 2:** Copy environment file (5 min)**** 1. Copy environment.yml from Python session to this folder

2. Verify it's there: ls

It is already here. No need to copy.

```
[18]: !ls environment.yml
```

```
environment.yml
```

******Step 3:** First commit (10 min)**** Note: that since this can only be done once, I have just copied the command and output from my own machine.

1. git add environment.yml

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git add environment.yml
```

Note no output so success.

2. git commit -m "Add Python environment specification"

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git commit -m "Add Python environment specification"
[main (root-commit) 42cc8c4] Add Python environment specification
1 file changed, 240 insertions(+)
create mode 100644 environment.yml
```

3. git log to see commit

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log
commit 42cc8c4471cfb70f0ecb1d88a75e3f633d414d78 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 20:52:34 2026 +0100
```

```
Add Python environment specification
```

******Step 4:** Second commit - Python skeleton (10min)**** 1. Create mandelbrot.py (example on previous slide)

```
[19]: !touch mandelbrot.py
```

```
[20]: %pycat mandelbrot.py
```

```
# Template from slide 30, lecture 01
"""
Mandelbrot Set Generator

Author: Mikkel Korsgaard Sørensen
Course: Numerical Scientific Computing 2026
"""
```

```
def f(x):
    """
    Example function.

    Parameters
    -----
    x : float
        Input value

    Returns
    -----
    float
        Output value
    """
    # TODO: Implement the algorithm
    pass
```

2. git add mandelbrot.py

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git add mandelbrot.py
```

3. git commit -m "Add mandelbrot skeleton"

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git commit -m "Add mandelbrot skeleton"
[main f9c732b] Add mandelbrot skeleton
1 file changed, 25 insertions(+)
create mode 100644 mandelbrot.py
```

4. git log - see both commits!

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log
commit f9c732b983dc6ece73b506af8b72faac75f2e26b (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 20:56:21 2026 +0100
```

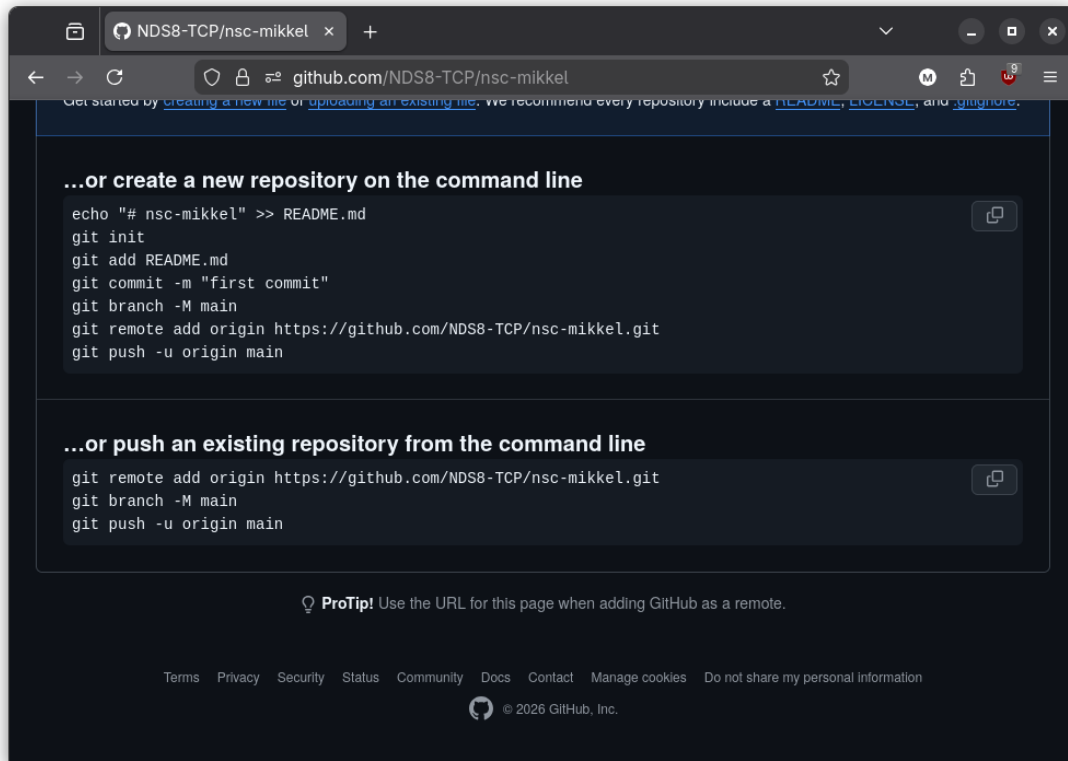
Add mandelbrot skeleton

```
commit 42cc8c4471cfb70f0ecb1d88a75e3f633d414d78
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
```

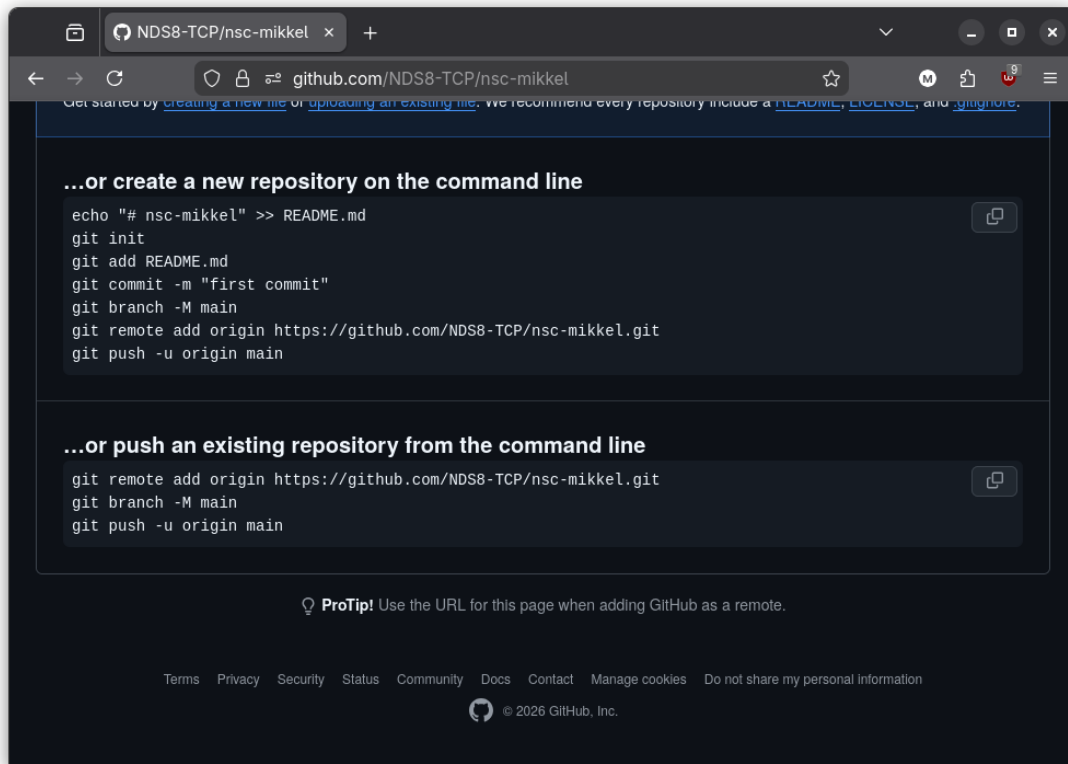
Date: Mon Mar 2 20:52:34 2026 +0100

Add Python environment specification

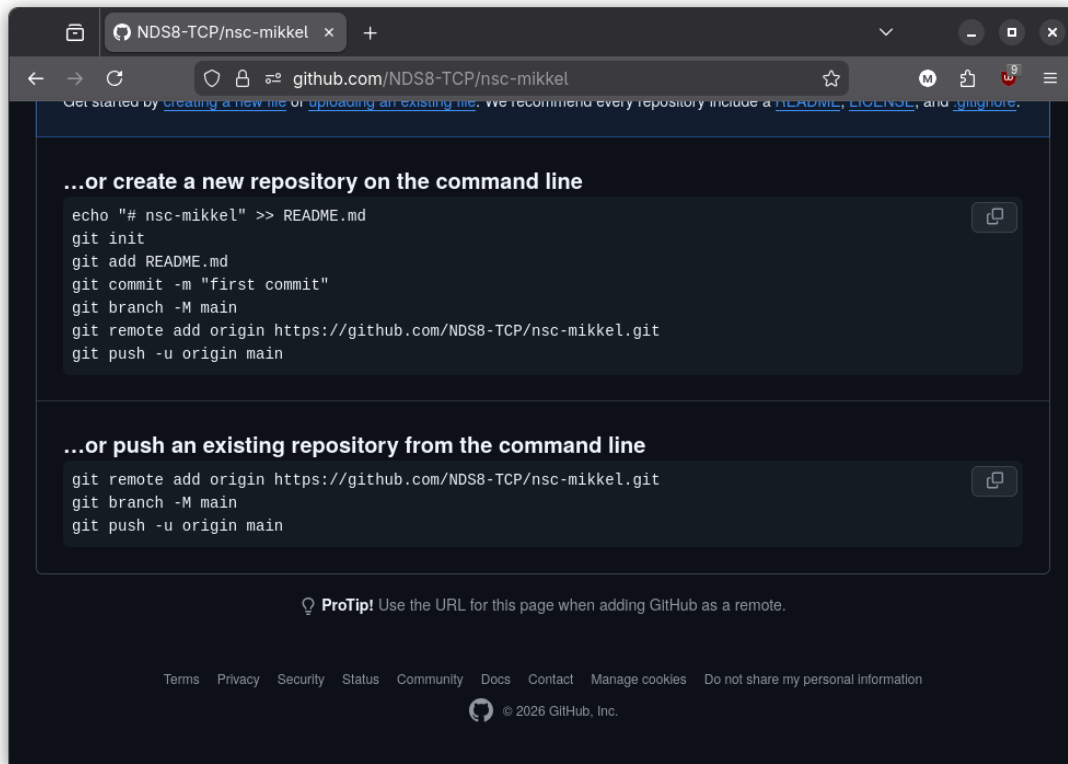
****Step 5:** Push to GitHub (15 min)** 1. Go to [GitHub.com](https://github.com)



2. Create new repository: mandelbrot-nsc



3. GitHub shows commands - copy them!****



```
git remote add origin https://github.com/NDS8-TCP/nsc-mikkel.git
git branch -M main
git push -u origin main
```

4. Paste in terminal (from mandelbrot-nsc/):

- git remote add origin <URL>

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
```

```
$ git remote add origin https://github.com/NDS8-TCP/nsc-mikkel.git
```

- git branch -M main

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
```

```
$ git branch -M main
```

- git push -u origin main

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
```

```
$ git push -u origin main
```

```
Username for 'https://github.com': ^C
```

Since I have not configured ssh authentication with github.com, I will be using vscode instead of pushing and pulling.

****Step 6:** Practice the cycle (5 min)** 1. Edit mandelbrot.py (add a comment)

```
[21]: %pycat mandelbrot.py
```



```

# Template from slide 30, lecture 01
"""
Mandelbrot Set Generator

Author: Mikkel Korsgaard Sørensen
Course: Numerical Scientific Computing 2026
"""
# This is a comment

def f(x):
    """
    Example function.

    Parameters
    -----
    x : float
        Input value

    Returns
    -----
    float
        Output value
    """
    # TODO: Implement the algorithm
    pass

```

2. git add mandelbrot.py

```

(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git add mandelbrot.py

```

3. git commit -m "Add TODO comment"

```

(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git commit -m "Add TODO comment"
[main a00914d] Add TODO comment
1 file changed, 1 insertion(+), 1 deletion(-)

```

4. git push

```

(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git push
Username for 'https://github.com': ^C

```

Again, I will be using VSCode for pulling and pushing needs.

1.1.3 *****Studio Session 3:** Begin Implementation (60 min)**

*****Task:** Implement naive Mandelbrot (or get started)**

*****Step 1:** Understand the algorithm (10 min)**

- Review pseudocode on previous slide

I would consider it an *embarrassingly parallel* problem, since this is basically an image processing task that can be computed in parallel per point/pixel in the image/grid. Only the *max_iter* is serial and dependent on the previous values of z for computing z^2

- Discuss with neighbor: "What does each part do?"

Done. We came to the overall same conclusion.

- Ask questions if confused!

Noted!

****Step 2:** Implement mandelbrot point function (20 min)**

- Takes single complex number c
- Returns iteration count

```
[22]: %pycat mandelbrot.py

# Template from slide 30, lecture 01
"""
Mandelbrot Set Generator

Author: Mikkel Korsgaard Sørensen
Course: Numerical Scientific Computing 2026
"""
# This is a comment

def f(x):
    """
    Example function.

    Parameters
    -----
    x : float
        Input value

    Returns
    -----
    float
        Output value
    """
    # TODO: Implement the algorithm
    pass

def mandelbrot_point(c: complex, max_iter: int) -> int:
    z = 0j
```

```

for n in range(max_iter):
    if abs(z) > 2:
        return n
    z = z**2 + c

return max_iter

```

- Test with known points (e.g., $c = 0$ should be max iter)

```

[23]: from mandelbrot import mandelbrot_point

assert mandelbrot_point(c=0j, max_iter=100) == 100

```

- ****Commit to Git** with descriptive message**

```

(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git add mandelbrot.py
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git commit -m "Add mandelbrot point implementation"
[main 19089dd] Add mandelbrot point implementation
1 file changed, 10 insertions(+)
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 19089ddc6145a529cdd1986e6297c7c2ed12e320 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 21:12:18 2026 +0100

```

Add mandelbrot point implementation

****Step 3:** Implement compute mandelbrot grid version (20 min)**

- Create mesh of complex numbers (Hint:** use `numpy.linspace` and nested loops)**
- Loop over all points, call `mandelbrot point` for each
- Return 2D array of iteration counts

```

[24]: %pycat mandelbrot.py

# Template from slide 30, lecture 01
"""
Mandelbrot Set Generator

Author: Mikkel Korsgaard Sørensen
Course: Numerical Scientific Computing 2026
"""

import numpy as np
# This is a comment

def f(x):
    """

```

```

Example function.

Parameters
-----
x : float
    Input value

Returns
-----
float
    Output value
"""
# TODO: Implement the algorithm
pass


def mandelbrot_point(c: complex, max_iter: int) -> int:
    z = 0j
    for n in range(max_iter):
        if abs(z) > 2:
            return n
        z = z**2 + c

    return max_iter


def compute_mandelbrot(
    x_min: float,
    x_max: float,
    y_min: float,
    y_max: float,
    width: int,
    height: int,
    max_iter: int
) -> list[list[int]]:
    x = np.linspace(x_min, x_max, width)
    y = np.linspace(y_min, y_max, height)

    img = []

    for i in range(height):
        img_row = []
        for j in range(width):
            img_row.append(mandelbrot_point(x[j] + 1j * y[i],
max_iter))
        img.append(img_row)

    return img

```

- ****Start with small grid (e.g., 100×100) for testing****

[25]: `from mandelbrot import compute_mandelbrot`

```
image = compute_mandelbrot(
    x_min=-2,
    x_max=1,
    y_min=-1.5,
    y_max=1.5,
    width=100,
    height=100,
    max_iter=100
)
```

- ****Commit to Git** with descriptive message**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git add mandelbrot.py
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git commit -m "Add mandelbrot compute python"
[main 1676717] Add mandelbrot compute python
1 file changed, 24 insertions(+)
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 1676717c24bb27ebf3d97175a6a9042df0bff777 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 21:16:51 2026 +0100
```

Add mandelbrot compute python

****Step 4:** Measure execution time (10 min)**

- Use `time.time()` or `timeit` to measure performance

I have chosen to do both.

Using `time.time()`

[26]: *# Mostly copied from slide 38*

```
import time

start = time.time()
image = compute_mandelbrot(-2, 1, -1.5, 1.5, 100, 100, 100)
end = time.time()
elapsed_time = end - start

print(f"Naïve Python (100x100): {elapsed_time:.3f}s")
```

Naïve Python (100x100): 0.061s

Using `timeit`

```
[27]: from timeit import timeit

elapsed_time = timeit(
    lambda: compute_mandelbrot(
        x_min=-2,
        x_max=1,
        y_min=-1.5,
        y_max=1.5,
        width=100,
        height=100,
        max_iter=100
    ),
    number=1
)

print(f"Naïve Python (100x100): {elapsed_time:.3f}s")
```

Naïve Python (100x100): 0.059s

- Try grid size 1024×1024 for meaningful baseline

```
[28]: import time

start = time.time()
image = compute_mandelbrot(
    x_min=-2,
    x_max=1,
    y_min=-1.5,
    y_max=1.5,
    width=1024,
    height=1024,
    max_iter=100
)
end = time.time()
elapsed_time = end - start

print(f"Naïve Python (1024x1024): {elapsed_time:.3f}s")
```

Naïve Python (1024x1024): 4.268s

- ****Commit to Git** with descriptive message**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git commit -m "Add notebook lecture 1, performance measure"
[main 47a2fe5] Add notebook lecture 1, performance measure
1 file changed, 3240 insertions(+)
create mode 100644 101_exercises.ipynb
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 47a2fe526ad000e42197f7df4bd82562713c54ee (HEAD -> main)
```

Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 21:20:00 2026 +0100

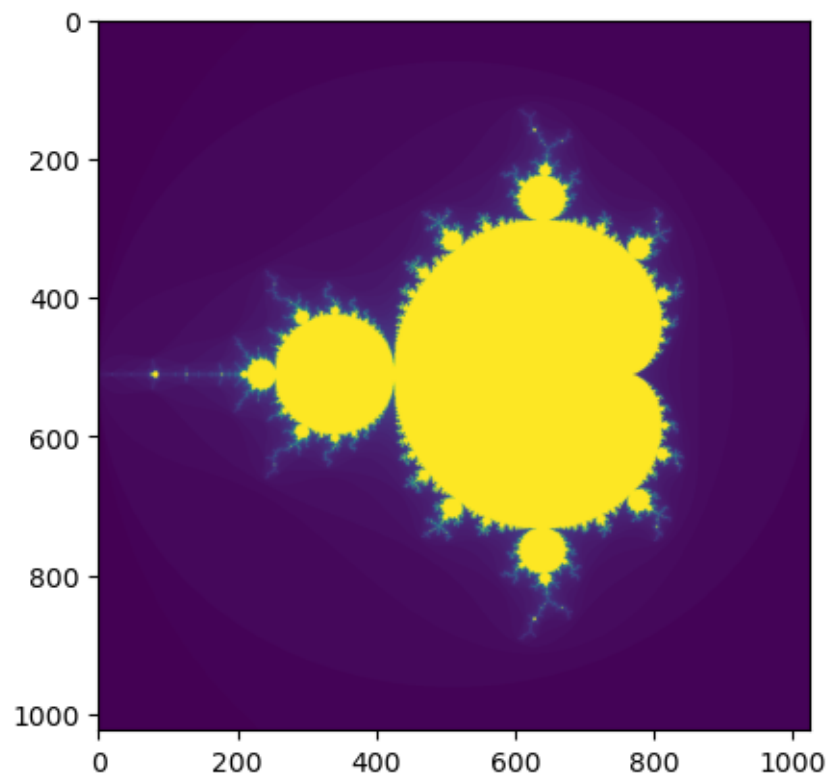
Add notebook lecture 1, performance measure

****Step 5:** Visualize the result (10 min)**

- Use `plt.imshow()` to display as image

```
[29]: import numpy as np
import matplotlib.pyplot as plt

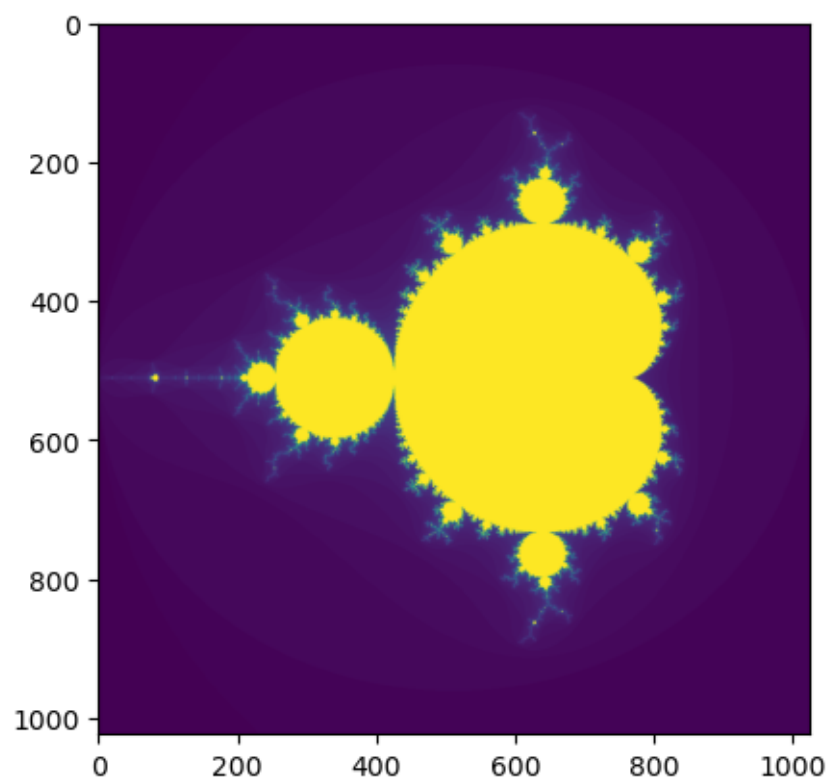
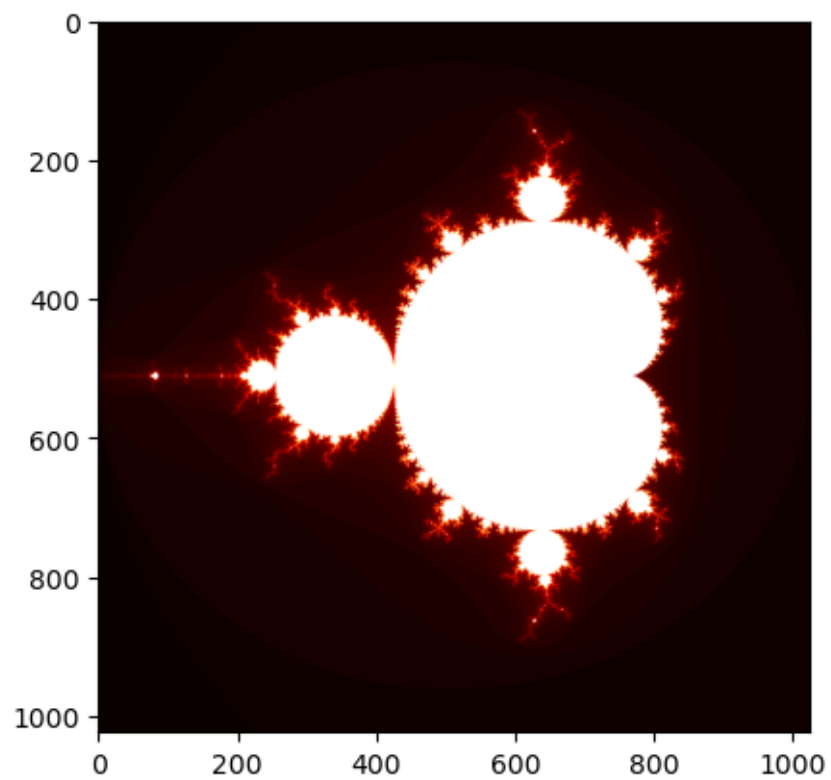
plt.imshow(np.array(image))
plt.show()
```

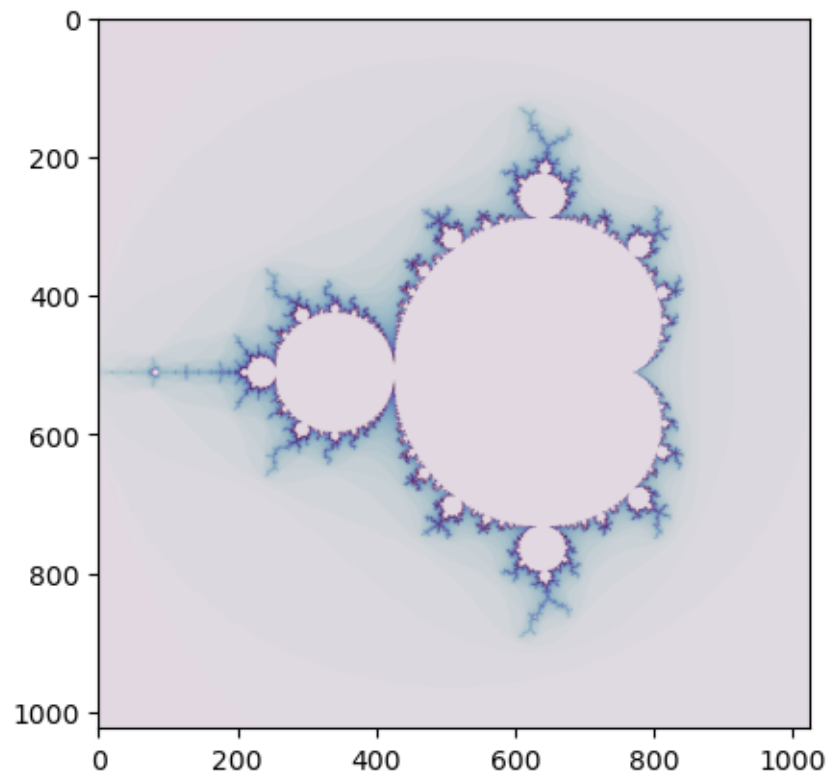


- Try colormaps: hot, viridis, twilight

```
[30]: color_maps = ["hot", "viridis", "twilight"]

for color_map in color_maps:
    plt.imshow(np.array(image), cmap=color_map)
    plt.show()
```





- Add colorbar, title. Save with `plt.savefig()`

```
[31]: for color_map in color_maps:
    plt.figure()
    plt.imshow(np.array(image), cmap=color_map)
    plt.colorbar()
    plt.title(color_map)
    plt.savefig(color_map)
    plt.close()
```

1.1.4 ****Peer Sharing:** Compare approaches (30 min)**

****Activity:** Form pairs or groups of 3**

****Step 1:** Share your progress (10 min per person)**

- Show your code (even if incomplete!)

Done! Note that I did not manage to complete in the studio session, since I was working on using an OCI container for development. I did share my code with the person next to me.

- Run and display Mandelbrot image

See studio session 3. There is a guide on how to run and display the mandelbrot images.

- **Explain your approach**
- I used the code from the slides as inspiration.
- Tried to make the implementation as slow as possible using python's list appending.
- **What worked? What was challenging?**

I did not find it particularly challenging. It just worked for me.

******Step 2:** Compare implementations (10 min)****

- **Different variable names?**

I compared my implementation to my peer next to me. Almost the same variable names, but still some slight differences, such as `width_resolution` vs `width`.

- **Different loop structures?**

Somewhat. In the `mandelbrot_point`, I swapped the computation of `z = z**2 + c` to *after* the convergence test so that I could get my `max_iter` to match with the previous exercises.

- **Different convergence tests?**

No. We both used Python's `abs()` method.

- **Any clever optimizations already?**

No. Not yet.

******Step 3:** Help each other (optional)****

- **If one person finished, help the other**
- **Debug issues together**
- **Share tips and tricks**

Noted, and done! I helped the peer next to me.

1.2 Lecture 2: Computer Architecture and Memory

After EACH milestone: 1. Share with neighbor (compare approaches) 2. Take a 10-minute break (self-managed) 3. Continue to next milestone

How it works: 1. After each successful optimization, click “Add entry” in the Performance Tracker on Moodle 2. Fill in your implementation type, resolution, and median runtime 3. Browse others’ entries — are you in the right ballpark?

******Benchmarking standard: Report the median of ≥ 3 runs using `time.perf_counter()`. Default resolution: 1024×1024 .****

******Today's task:**** After Milestone 2, log your **naive baseline** and your NumPy vectorized result. Your naive entry is the most important one — it's your starting point!**

1.2.1 ****Milestone 1:** Basic arrays & meshgrid (Target: 15 min)**

****Your Task:** Create the complex grid C for the Mandelbrot set**

- Create 1D arrays with linspace

```
[1]: import numpy as np

# Copied from slide 25
x = np.linspace(-2, 1, 1024)    # 1024 x - values
y = np.linspace(-1.5, 1.5, 1024) # 1024 y - values
```

- Create 2D grid with meshgrid

```
[2]: # Copied from slide 25

X, Y = np.meshgrid(x, y)
# X , Y are now 1024 x1024 arrays
```

- Combine into complex array C

```
[3]: # Copied from slide 25

C = X + 1j * Y # Note: 1j is imaginary unit in Python
# C is now 1024x1024 complex array
```

- Verify shape and dtype

```
[4]: print(f"Shape: {C.shape}")
      print(f"Type: {C.dtype}")
```

Shape: (1024, 1024)

Type: complex128

****Done?** Commit → share with neighbor → break → Milestone 2**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git add 102_exercises.ipynb
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git commit -m "102: milestone 1 done"
[main 806b5d0] 102: milestone 1 done
1 file changed, 489 insertions(+)
create mode 100644 102_exercises.ipynb
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 806b5d0e5cc8edf89cabd0725f15f1fbd2729107 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 21:34:41 2026 +0100
```

102: milestone 1 done

1.2.2 ****Milestone 2:** Full vectorized Mandelbrot (Target: 40 min)**

****Your Task:** Replace Python loops with NumPy operations**

****Naive approach has 3 nested loops:**** 1. **Loop over rows (i)** 2. **Loop over columns (j)**
→ selects pixel/point c 3. **Loop over iterations (n)** → computes $z = z^2 + c$ until escape

Hints: `np.zeros like(C)`, `np.zeros(C.shape, dtype=int)`

****NumPy approach — eliminate loops 1 & 2:****

- Initialize Z and M arrays (same shape as C)
- Keep only loop 3 (iterations), operate on *all* pixels at once
- Boolean mask: `mask = np.abs(Z) <= 2`
- Update only unescaped points: `Z[mask] = Z[mask]**2 + C[mask]`
- Increment iteration count: `M[mask] += 1`

```
[5]: %pycat mandelbrot.py
```

```
# Template from slide 30, lecture 01
"""
Mandelbrot Set Generator

Author: Mikkel Korsgaard Sørensen
Course: Numerical Scientific Computing 2026
"""
import numpy as np
# This is a comment

def f(x):
    """
    Example function.

    Parameters
    -----
    x : float
        Input value

    Returns
    -----
    float
        Output value
    """
    # TODO: Implement the algorithm
    pass

def mandelbrot_point(c: complex, max_iter: int) -> int:
    z = 0j
    for n in range(max_iter):
```

```

        if abs(z) > 2:
            return n
        z = z**2 + c

    return max_iter

def compute_mandelbrot(
    x_min: float,
    x_max: float,
    y_min: float,
    y_max: float,
    width: int,
    height: int,
    max_iter: int
) -> list[list[int]]:
    x = np.linspace(x_min, x_max, width)
    y = np.linspace(y_min, y_max, height)

    img = []

    for i in range(height):
        img_row = []
        for j in range(width):
            img_row.append(mandelbrot_point(x[j] + 1j * y[i],
max_iter))
        img.append(img_row)

    return img

def compute_mandelbrot_numpy(
    x_min: float,
    x_max: float,
    y_min: float,
    y_max: float,
    width: int,
    height: int,
    max_iter: int
):
    # Inspired by slide 30, lecture 02
    x = np.linspace(x_min, x_max, width)
    y = np.linspace(y_min, y_max, height)

    X, Y = np.meshgrid(x, y)

    C = X + 1j*Y

```

```

Z = np.zeros_like(C)
M = np.zeros_like(C, dtype=int)
for _ in range(max_iter):
    mask = np.abs(Z) <= 2
    Z[mask] = Z[mask]**2 + C[mask]
    M[mask] += 1

return M

```

****Done? Commit → log in Performance Tracker → share with neighbor**

```

(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 069bb1e23fb1d6e9aa03a710ad0af096ad5b78e1 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 20:39:57 2026 +0000

```

l02: milestone 2 done

Validating Your Results

```
[6]: from mandelbrot import compute_mandelbrot, compute_mandelbrot_numpy
```

```

conf = {
    "x_min": -2,
    "x_max": 1,
    "y_min": -1.5,
    "y_max": 1.5,
    "width": 1024,
    "height": 1024,
    "max_iter": 100
}
naive_result = compute_mandelbrot(**conf)
numpy_result = compute_mandelbrot_numpy(**conf)

# From slide 32
# CORRECT - Use np.allclose():
if np.allclose(naive_result, numpy_result):
    print("Results match!")
else:
    print("Results differ!")

# Check where they differ :
diff = np.abs(naive_result - numpy_result)
print(f"Max difference: {diff.max()}")
print(f"Different pixels: {(diff > 0).sum()}")

```

```

Results match!
Max difference: 0
Different pixels: 0

```

After Milestone 2, log your naive baseline and your NumPy vectorized result.

```
[7]: # From slide 29
import time
import statistics

def benchmark(func, *args, n_runs=3):
    """Time func, return median of n_runs."""
    times = []
    for _ in range(n_runs):
        t0 = time.perf_counter()
        result = func(*args)
        times.append(time.perf_counter() - t0)

    median_t = statistics.median(times)
    print(f"Median: {median_t:.4f}s"
          f"(min={min(times):.4f}, max={max(times):.4f})")
    return median_t, result
```

```
[8]: from mandelbrot import compute_mandelbrot as naive_python
from mandelbrot import compute_mandelbrot_numpy as numpy_python

# Mostly copied from slide 29
t1, M1 = benchmark(naive_python, -2, 1, -1.5, 1.5, 1024, 1024, 100)
t2, M2 = benchmark(numpy_python, -2, 1, -1.5, 1.5, 1024, 1024, 100)

print(f"Naïve Python (median): {t1:.3f}s")
print(f"Numpy Python (median): {t2:.3f}s. Speedup of {t1/t2:.2f}x")
```

```
Median: 4.0968s(min=4.0420, max=4.1096)
Median: 0.5983s(min=0.5888, max=0.6060)
Naïve Python (median): 4.097s
Numpy Python (median): 0.598s. Speedup of 6.85x
```

1.2.3 ****Milestone 3:** Memory access pattern analysis (Target: 15 min)**

****Your Task:** Measure the effect of memory layout on performance**

1. Create a large square** array: $A = \text{np.random.rand}(N, N)$ with $N = 10000$ **

```
[9]: N = 10_000
A = np.random.rand(N, N)
```

2. Write a function that computes row sums** by looping over rows: - for i in $\text{range}(N)$:
 $s = \text{np.sum}(A[i, :])$ **

```
[10]: def row_sum(A, N):
    for i in range(N):
        s = np.sum(A[i, :])
```

3. Write a function that computes column sums** by looping over columns: - for `j` in `range(N)`: `s = np.sum(A[:, j])**`

```
[11]: def column_sum(A, N):  
      for j in range(N):  
          s = np.sum(A[:, j])
```

4. Time both. Both loops run N times — which is faster and why?

```
[12]: t_row, _ = benchmark(row_sum, A, N)  
      t_col, _ = benchmark(column_sum, A, N)  
  
      print(f"Row sum (median): {t_row:.3f}s")  
      print(f"Column sum (median): {t_col:.3f}s. Slowdown of {t_col/t_row:.2f}x")
```

```
Median: 0.0789s(min=0.0770, max=0.0830)  
Median: 0.1532s(min=0.1522, max=0.1564)  
Row sum (median): 0.079s  
Column sum (median): 0.153s. Slowdown of 1.94x
```

5. Now try with `A_f = np.asfortranarray(A)` (column-major). What changes?

```
[13]: A_f = np.asfortranarray(A)  
  
      t_row, _ = benchmark(row_sum, A_f, N)  
      t_col, _ = benchmark(column_sum, A_f, N)  
  
      print(f"Row sum (median): {t_row:.3f}s")  
      print(f"Column sum (median): {t_col:.3f}s. Speedup of {t_row/t_col:.2f}x")
```

```
Median: 0.1543s(min=0.1512, max=0.1742)  
Median: 0.0689s(min=0.0648, max=0.0699)  
Row sum (median): 0.154s  
Column sum (median): 0.069s. Speedup of 2.24x
```

****Done? Commit → share results with neighbor → Milestone 4**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]  
$ git log -n 1  
commit 396437f33a71bec972e10695b289c652c6c016de (HEAD -> main)  
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>  
Date: Mon Mar 2 20:44:24 2026 +0000
```

102: milestone 3 done

1.2.4 **Milestone 4:** Problem size scaling (Target: 15 min)****

****Your Task:** How does Mandelbrot runtime scale with grid size?*

1. Run your vectorized Mandelbrot for grid sizes: 256, 512, 1024, 2048, 4096

See the code below, combined with timing results.

2. Record runtime for each (use timeit or manual timing)

I am using the provided benchmark function from the slides.

```
[14]: from mandelbrot import compute_mandelbrot_numpy as vectorized_mandelbrot

grids = [256, 512, 1024, 2048, 4096]

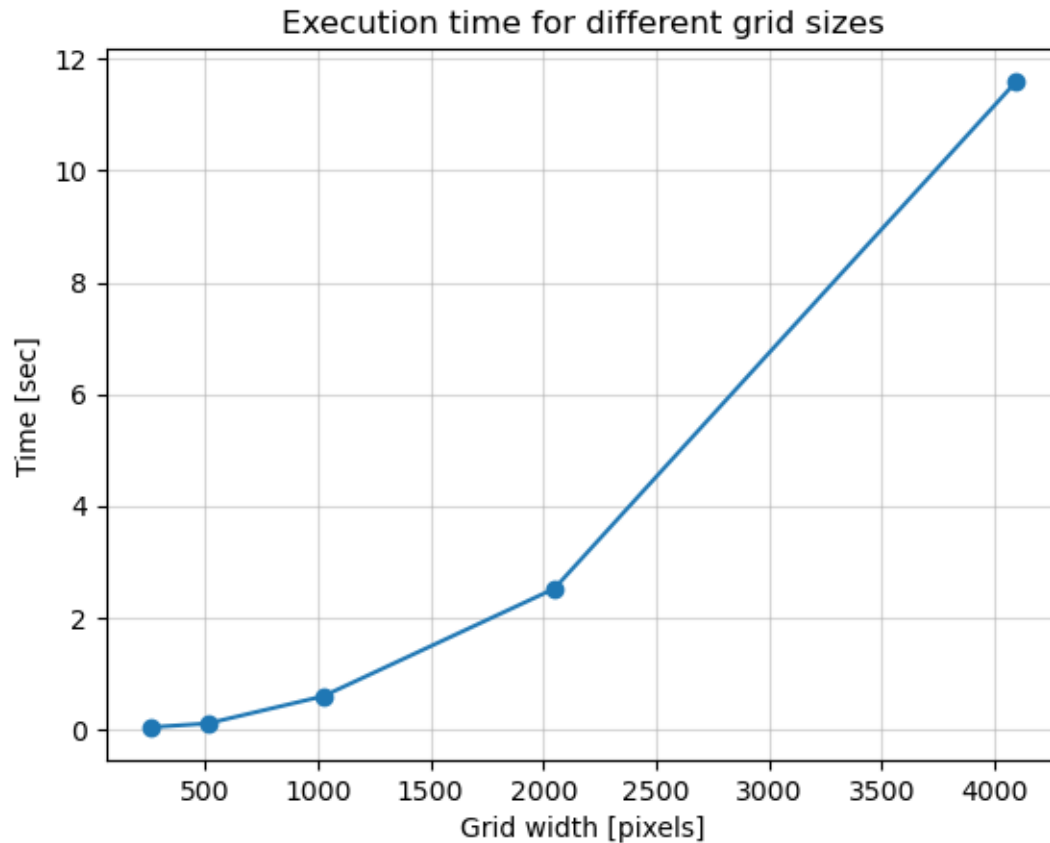
times = []
for g in grids:
    print(f"computing for: {g}")
    t, _ = benchmark(vectorized_mandelbrot, -2, 1, -1.5, 1.5, g, g, 100)
    times.append(t)
```

```
computing for: 256
Median: 0.0330s(min=0.0329, max=0.0366)
computing for: 512
Median: 0.1012s(min=0.0908, max=0.1289)
computing for: 1024
Median: 0.5891s(min=0.5742, max=0.5978)
computing for: 2048
Median: 2.5158s(min=2.4961, max=2.5170)
computing for: 4096
Median: 11.6031s(min=11.4277, max=11.6575)
```

3. Plot: grid size vs. runtime

```
[15]: import matplotlib.pyplot as plt

plt.figure()
plt.plot(grids, times, marker="o")
plt.title("Execution time for different grid sizes")
plt.xlabel("Grid width [pixels]")
plt.ylabel("Time [sec]")
plt.grid(alpha=0.5)
plt.show()
```



4. Predict:** If 1024×1024 takes X seconds, what should 2048×2048 take? - (Hint: $4\times$ the pixels — do you get $4\times$ the time?）**

$$1024^2 = 1,048,576$$

$$2048^2 = 4,194,304$$

Assuming that the compute time and grid pixels are linear, then quadrupling the number of pixels results in four times the compute time.

Questions to consider:

- Is the scaling linear in number of pixels?

It appears so, yes.

- At what size does the working set exceed your L3 cache?

```
(mikkel@pella)-[~]
$ lscpu | grep L3
L3 cache:                                16 MiB (1 instance)
```

Assuming that the memory footprint per pixel is about $C, Z, M = 16B + 16B + 4B = 36B$ (complex128, complex128, int32), then it would be possible to fit:

$$\frac{16 \times (1024^2)B}{36B} \approx 466,000 \text{ pixels}$$

Which is a width of $\sqrt{466,000} \approx 682$ pixels. Anything more than that would exceed the L3 cache.

- **Do you see a “knee” in the scaling curve where performance degrades?**

No, I do not.

****Done?**** Commit → share plot with neighbor → try extensions**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 43d25d86dc95ed5b82e7417742e145dce7f7e83b (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 20:47:40 2026 +0000
```

102: milestone 4 done

1.2.5 ****Extensions** Memory profiling, region exploration (no time limit)**

****Extension 1: Memory Profiling****

- **Install:** `mamba install memory_profiler`

For sanity check, clean mamba before install.

[16]: `!mamba clean --all --yes`

Collect information..

Cleaning index cache..

Cleaning lock files..

Package file	Size
--------------	------

/opt/conda/pkgs

_openmp_mutex-4.5-20_gnu.conda	29kB
aws-c-auth-0.9.6-hb9c0fe4_1.conda	133kB
aws-c-compression-0.3.2-h8b1a151_0.conda	22kB
aws-c-event-stream-0.5.9-h841be55_2.conda	59kB
aws-c-http-0.10.10-hf621c6d_0.conda	226kB
aws-c-io-0.26.1-h3ca20c3_1.conda	182kB
aws-c-mqtt-0.14.0-ha25ca29_1.conda	220kB
aws-c-s3-0.11.5-h9b5df67_3.conda	151kB
aws-checksums-0.2.10-h8b1a151_0.conda	101kB
aws-crt-cpp-0.37.3-hb153662_0.conda	410kB
aws-sdk-cpp-1.11.747-h133b1ee_1.conda	4MB

bzip2-1.0.8-hda65f42_9.conda	260kB
coverage-7.13.4-py311h3778330_0.conda	398kB
cyrus-sasl-2.1.28-hac629b4_1.conda	210kB
cytoolz-1.1.0-py311h49ec1c0_2.conda	624kB
dask-2026.1.2-pyhcf101f3_1.conda	11kB
distributed-2026.1.2-pyhcf101f3_1.conda	845kB
fontconfig-2.17.1-h27c8c51_0.conda	271kB
krb5-1.22.2-ha1258a1_0.conda	1MB
libabseil-20260107.1-cxx17_h7b12aa8_0.conda	1MB
libarrow-23.0.1-h711ef25_3_cpu.conda	6MB
libarrow-acero-23.0.1-h635bf11_3_cpu.conda	609kB
libarrow-compute-23.0.1-h53684a4_3_cpu.conda	3MB
libarrow-dataset-23.0.1-h635bf11_3_cpu.conda	608kB
libarrow-substrait-23.0.1-hb4dd7c2_3_cpu.conda	519kB
libclang-cpp22.1-22.1.0-default_h99862b1_0.conda	22MB
libclang13-22.1.0-default_h746c552_0.conda	13MB
libcups-2.3.3-h7a8fb5f_6.conda	5MB
libcurl-8.18.0-hcf29cc6_1.conda	464kB
libexpat-2.7.4-hecca717_0.conda	77kB
libgcc-15.2.0-he0feb66_18.conda	1MB
libgcc-ng-15.2.0-h69a702a_18.conda	28kB
libgfortran-15.2.0-h69a702a_18.conda	28kB
libgfortran5-15.2.0-h68bc16d_18.conda	2MB
libglib-2.86.4-h6548e54_1.conda	4MB
libgomp-15.2.0-he0feb66_18.conda	603kB
libgoogle-cloud-2.39.0-h9d11ab5_1.conda	1MB
libgoogle-cloud-storage-2.39.0-hdbdcf42_1.conda	803kB
libgrpc-1.78.0-h1d1128b_1.conda	7MB
libllvm22-22.1.0-hf7376ad_0.conda	44MB
libopentelemetry-cpp-1.21.0-h9692893_2.conda	897kB
libopentelemetry-cpp-headers-1.21.0-ha770c72_2.conda	362kB
libparquet-23.0.1-h7376487_3_cpu.conda	1MB
libpng-1.6.55-h421ea60_0.conda	318kB
libpq-18.3-h9abb657_0.conda	3MB
libprotobuf-6.33.5-h2b00c02_0.conda	4MB
libre2-11-2025.11.05-h0dc7533_1.conda	213kB
libstdcxx-15.2.0-h934c35e_18.conda	6MB
libstdcxx-ng-15.2.0-hdf11a46_18.conda	28kB
markupsafe-3.0.3-py311h3778330_1.conda	27kB
narwhals-2.17.0-pyhcf101f3_0.conda	280kB
numpy-2.4.2-py311h2e04523_1.conda	9MB
openldap-2.6.10-hbde042b_1.conda	786kB
orc-2.2.2-hbb90d81_1.conda	1MB
pandas-3.0.1-py311h8032f78_0.conda	15MB
pillow-12.1.1-py311hf88fc01_0.conda	1MB
pyarrow-23.0.1-py311h38be061_0.conda	29kB
pyarrow-core-23.0.1-py311h66caee6_0_cpu.conda	5MB
pytest-cov-7.0.0-pyhcf101f3_1.conda	29kB

qt6-main-6.10.2-h17e89b9_5.conda	57MB
re2-2025.11.05-h5301d42_1.conda	27kB
s2n-1.7.0-ha63dd3a_1.conda	398kB
scipy-1.17.1-py311hbe70eeb_0.conda	17MB
setuptools-82.0.0-pyh332efcf_0.conda	638kB
unicodedata2-17.0.1-py311h49ec1c0_0.conda	410kB
xkeyboard-config-2.47-hb03c661_0.conda	399kB
xorg-libx11-1.8.13-he1eb515_0.conda	840kB

/root/.mamba/pkgs

Total size:	249MB
Cleaning tarballs..	
Package folder	Size

/opt/conda/pkgs

numpy-2.4.2-py311h2e04523_1	42MB
-----------------------------	------

/root/.mamba/pkgs

Total size:	42MB
Cleaning packages..	
warning libmamba This does not check for packages installed using symlinks back to the package cache.	

```
[17]: !mamba install --name nsc2026 memory_profiler -y
```

```
[+] 0.0s
[+] 0.1s
conda-forge/linux-64          0.0 B /  ???.?MB
@  ???.?MB/s  0.0s
conda-forge/noarch            0.0 B
/  ???.?MB @  ???.?MB/s  0.0s[+] 0.2s
conda-forge/linux-64          0.0 B /  ???.?MB
@  ???.?MB/s  0.1s
conda-forge/noarch            0.0 B
```

```

/   ???.?MB @   ???.?MB/s   0.1s[+] 0.3s
conda-forge/linux-64           188.5kB /   51.4MB @
704.7kB/s   0.2s
conda-forge/noarch             265.2kB /   24.9MB @
991.6kB/s   0.2s[+] 0.4s
conda-forge/linux-64           412.6kB /   51.4MB @
1.3MB/s   0.3s
conda-forge/noarch             756.7kB /   24.9MB @
2.4MB/s   0.3s[+] 0.5s
conda-forge/linux-64           1.3MB /   51.4MB @
2.7MB/s   0.4s
conda-forge/noarch             1.9MB /   24.9MB @
4.1MB/s   0.4s[+] 0.6s
conda-forge/linux-64           1.8MB /   51.4MB @
3.1MB/s   0.5s
conda-forge/noarch             2.8MB /   24.9MB @
5.0MB/s   0.5s[+] 0.7s
conda-forge/linux-64           2.3MB /   51.4MB @
3.5MB/s   0.6s
conda-forge/noarch             3.7MB /   24.9MB @
5.6MB/s   0.6s[+] 0.8s
conda-forge/linux-64           2.9MB /   51.4MB @
3.7MB/s   0.7s
conda-forge/noarch             4.6MB /   24.9MB @
6.0MB/s   0.7s[+] 0.9s
conda-forge/linux-64           3.4MB /   51.4MB @
4.0MB/s   0.8s
conda-forge/noarch             5.5MB /   24.9MB @
6.3MB/s   0.8s[+] 1.0s
conda-forge/linux-64           4.0MB /   51.4MB @
4.1MB/s   0.9s
conda-forge/noarch             6.3MB /   24.9MB @
6.5MB/s   0.9s[+] 1.1s
conda-forge/linux-64           4.5MB /   51.4MB @
4.3MB/s   1.0s
conda-forge/noarch             7.2MB /   24.9MB @
6.7MB/s   1.0s[+] 1.2s
conda-forge/linux-64           5.1MB /   51.4MB @
4.4MB/s   1.1s
conda-forge/noarch             8.1MB /   24.9MB @
6.9MB/s   1.1s[+] 1.3s
conda-forge/linux-64           5.6MB /   51.4MB @
4.4MB/s   1.2s
conda-forge/noarch             9.0MB /   24.9MB @
7.1MB/s   1.2s[+] 1.4s
conda-forge/linux-64           6.2MB /   51.4MB @
4.5MB/s   1.3s
conda-forge/noarch             9.8MB /   24.9MB @

```

7.2MB/s 1.3s[+] 1.5s	
conda-forge/linux-64	6.7MB / 51.4MB @
4.6MB/s 1.4s	
conda-forge/noarch	10.7MB / 24.9MB @
7.3MB/s 1.4s[+] 1.6s	
conda-forge/linux-64	7.3MB / 51.4MB @
4.6MB/s 1.5s	
conda-forge/noarch	11.6MB / 24.9MB @
7.4MB/s 1.5s[+] 1.7s	
conda-forge/linux-64	7.8MB / 51.4MB @
4.7MB/s 1.6s	
conda-forge/noarch	12.4MB / 24.9MB @
7.4MB/s 1.6s[+] 1.8s	
conda-forge/linux-64	8.4MB / 51.4MB @
4.8MB/s 1.7s	
conda-forge/noarch	13.3MB / 24.9MB @
7.5MB/s 1.7s[+] 1.9s	
conda-forge/linux-64	9.0MB / 51.4MB @
4.8MB/s 1.8s	
conda-forge/noarch	14.1MB / 24.9MB @
7.5MB/s 1.8s[+] 2.0s	
conda-forge/linux-64	9.6MB / 51.4MB @
4.8MB/s 1.9s	
conda-forge/noarch	15.0MB / 24.9MB @
7.6MB/s 1.9s[+] 2.1s	
conda-forge/linux-64	10.2MB / 51.4MB @
4.9MB/s 2.0s	
conda-forge/noarch	15.8MB / 24.9MB @
7.6MB/s 2.0s[+] 2.2s	
conda-forge/linux-64	10.7MB / 51.4MB @
4.9MB/s 2.1s	
conda-forge/noarch	16.7MB / 24.9MB @
7.7MB/s 2.1s[+] 2.3s	
conda-forge/linux-64	11.3MB / 51.4MB @
5.0MB/s 2.2s	
conda-forge/noarch	17.5MB / 24.9MB @
7.7MB/s 2.2s[+] 2.4s	
conda-forge/linux-64	11.9MB / 51.4MB @
5.0MB/s 2.3s	
conda-forge/noarch	18.4MB / 24.9MB @
7.7MB/s 2.3s[+] 2.5s	
conda-forge/linux-64	12.4MB / 51.4MB @
5.0MB/s 2.4s	
conda-forge/noarch	19.2MB / 24.9MB @
7.8MB/s 2.4s[+] 2.6s	
conda-forge/linux-64	13.0MB / 51.4MB @
5.0MB/s 2.5s	
conda-forge/noarch	20.1MB / 24.9MB @

7.8MB/s 2.5s[+] 2.7s	
conda-forge/linux-64	13.3MB / 51.4MB @
5.0MB/s 2.6s	
conda-forge/noarch	20.5MB / 24.9MB @
7.6MB/s 2.6s[+] 2.8s	
conda-forge/linux-64	13.6MB / 51.4MB @
5.0MB/s 2.7s	
conda-forge/noarch	20.7MB / 24.9MB @
7.6MB/s 2.7s[+] 2.9s	
conda-forge/linux-64	14.6MB / 51.4MB @
5.1MB/s 2.8s	
conda-forge/noarch	22.4MB / 24.9MB @
7.8MB/s 2.8s[+] 3.0s	
conda-forge/linux-64	15.1MB / 51.4MB @
5.1MB/s 2.9s	
conda-forge/noarch	23.2MB / 24.9MB @
7.8MB/s 2.9s[+] 3.1s	
conda-forge/linux-64	15.7MB / 51.4MB @
5.1MB/s 3.0s	
conda-forge/noarch	24.2MB / 24.9MB @
7.9MB/s 3.0s[+] 3.2s	
conda-forge/linux-64	15.8MB / 51.4MB @
5.0MB/s 3.1s	
conda-forge/noarch	24.3MB / 24.9MB @
7.8MB/s 3.1sconda-forge/noarch	
24.9MB @ 7.8MB/s 3.2s	
[+] 3.3s	
conda-forge/linux-64	16.4MB / 51.4MB @
5.0MB/s 3.2s[+] 3.4s	
conda-forge/linux-64	18.0MB / 51.4MB @
5.3MB/s 3.3s[+] 3.5s	
conda-forge/linux-64	19.2MB / 51.4MB @
5.5MB/s 3.4s[+] 3.6s	
conda-forge/linux-64	20.6MB / 51.4MB @
5.7MB/s 3.5s[+] 3.7s	
conda-forge/linux-64	22.0MB / 51.4MB @
6.0MB/s 3.6s[+] 3.8s	
conda-forge/linux-64	23.5MB / 51.4MB @
6.2MB/s 3.7s[+] 3.9s	
conda-forge/linux-64	24.9MB / 51.4MB @
6.4MB/s 3.8s[+] 4.0s	
conda-forge/linux-64	25.9MB / 51.4MB @
6.5MB/s 3.9s[+] 4.1s	
conda-forge/linux-64	26.9MB / 51.4MB @
6.6MB/s 4.0s[+] 4.2s	
conda-forge/linux-64	28.4MB / 51.4MB @
6.8MB/s 4.1s[+] 4.3s	
conda-forge/linux-64	29.1MB / 51.4MB @

6.9MB/s 4.2s[+] 4.4s	
conda-forge/linux-64	30.6MB / 51.4MB @
7.0MB/s 4.3s[+] 4.5s	
conda-forge/linux-64	32.3MB / 51.4MB @
7.2MB/s 4.4s[+] 4.6s	
conda-forge/linux-64	33.5MB / 51.4MB @
7.3MB/s 4.5s[+] 4.7s	
conda-forge/linux-64	35.0MB / 51.4MB @
7.5MB/s 4.6s[+] 4.8s	
conda-forge/linux-64	36.1MB / 51.4MB @
7.5MB/s 4.7s[+] 4.9s	
conda-forge/linux-64	36.9MB / 51.4MB @
7.6MB/s 4.8s[+] 5.0s	
conda-forge/linux-64	38.4MB / 51.4MB @
7.7MB/s 4.9s[+] 5.1s	
conda-forge/linux-64	39.8MB / 51.4MB @
7.8MB/s 5.0s[+] 5.2s	
conda-forge/linux-64	41.0MB / 51.4MB @
7.9MB/s 5.1s[+] 5.3s	
conda-forge/linux-64	41.8MB / 51.4MB @
7.9MB/s 5.2s[+] 5.4s	
conda-forge/linux-64	43.1MB / 51.4MB @
8.0MB/s 5.3s[+] 5.5s	
conda-forge/linux-64	44.5MB / 51.4MB @
8.1MB/s 5.4s[+] 5.6s	
conda-forge/linux-64	45.8MB / 51.4MB @
8.2MB/s 5.5s[+] 5.7s	
conda-forge/linux-64	45.8MB / 51.4MB @
8.2MB/s 5.6s[+] 5.8s	
conda-forge/linux-64	47.0MB / 51.4MB @
8.2MB/s 5.7s[+] 5.9s	
conda-forge/linux-64	48.3MB / 51.4MB @
8.2MB/s 5.8s[+] 6.0s	
conda-forge/linux-64	49.7MB / 51.4MB @
8.3MB/s 5.9s[+] 6.1s	
conda-forge/linux-64	51.1MB / 51.4MB @
8.4MB/s 6.0s[+] 6.2s	
conda-forge/linux-64	51.1MB / 51.4MB @
8.4MB/s 6.1sconda-forge/linux-64	
51.4MB @ 8.4MB/s 6.2s	
[+] 6.3s	

Pinned packages:

- python=3.11

Transaction

Prefix: /opt/conda/envs/nsc2026

Updating specs:

- memory_profiler

Package	Version	Build	Channel	Size
---------	---------	-------	---------	------

Install:

+ memory_profiler	0.61.0	pyhcf101f3_1	conda-forge	
-------------------	--------	--------------	-------------	--

Cached

Summary:

Install: 1 packages

Total download: 0 B

Transaction starting

[+] 0.0s

Linking memory_profiler-0.61.0-pyhcf101f3_1

Transaction finished

I have manually added it as a dependency in the `environment.yaml` file.

```
[18]: !cat environment.yaml | grep memory_profiler
```

```
- memory_profiler=0.61.0=pyhcf101f3_1
```

- **Compare peak memory usage: naive vs NumPy**

Please check the docs on how to run and use https://github.com/pythonprofilers/memory_profiler.

```
[19]: %pycat 102_memory_profiling.py
```

```
def main() -> None:
    import matplotlib.pyplot as plt
    from memory_profiler import
memory_usage
    from mandelbrot import
```

```

compute_mandelbrot as naive_python
    from mandelbrot import
compute_mandelbrot_numpy as numpy_python

    # Measured in MiB
    naive_mem_usage: list[float] = memory_usage(
        (naive_python, [-2, 1, -1.5,
1.5, 1024, 1024, 100]))
    numpy_mem_usage: list[float] = memory_usage(
        (numpy_python, [-2, 1, -1.5,
1.5, 1024, 1024, 100]))

    plt.plot(naive_mem_usage, label="Naïve")
    plt.plot(numpy_mem_usage, label="Numpy")
    plt.title("Memory usage for 1024x1024 mandelbrot")
    plt.ylabel("Memory usage [MiB]")
    plt.xlabel("Time [sample index]")
    plt.legend()
    plt.grid(alpha=0.5)
    plt.ticklabel_format(style="plain") # prevent
scientific notation
    plt.savefig("l01_memory_profile.png")

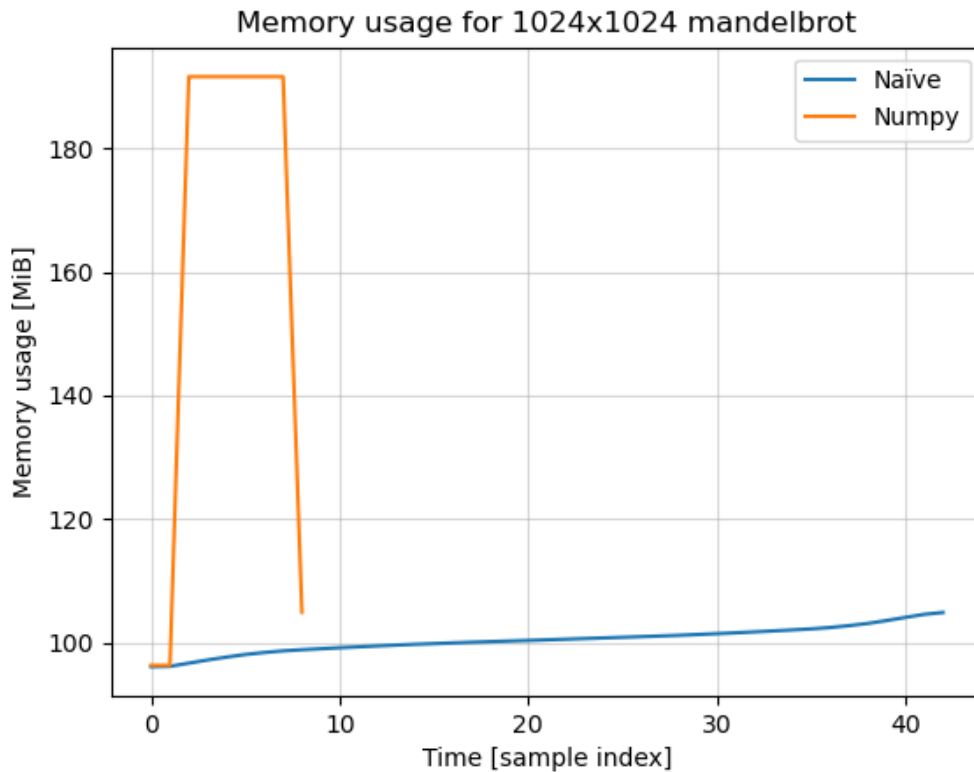
if __name__ == "__main__":
    main()

```

```
[20]: !python l02_memory_profiling.py
```

```
[21]: from IPython.display import Image
      Image(filename="l02_memory_profile.png")
```

```
[21]:
```



- Why does the vectorized version use more memory?

Numpy operates on *all* the pixels, while the naive only works on those that need it.

We can also see that python's `.append()` in the for loop results in growing memory allocation over time.

****Extension 2: Explore Different Mandelbrot Regions****

- Zoom into the boundary region (more iterations needed)

I do not know of “the boundary region”, so I have chosen the Seahorse Valley. (See slide number 49 for the parameters)

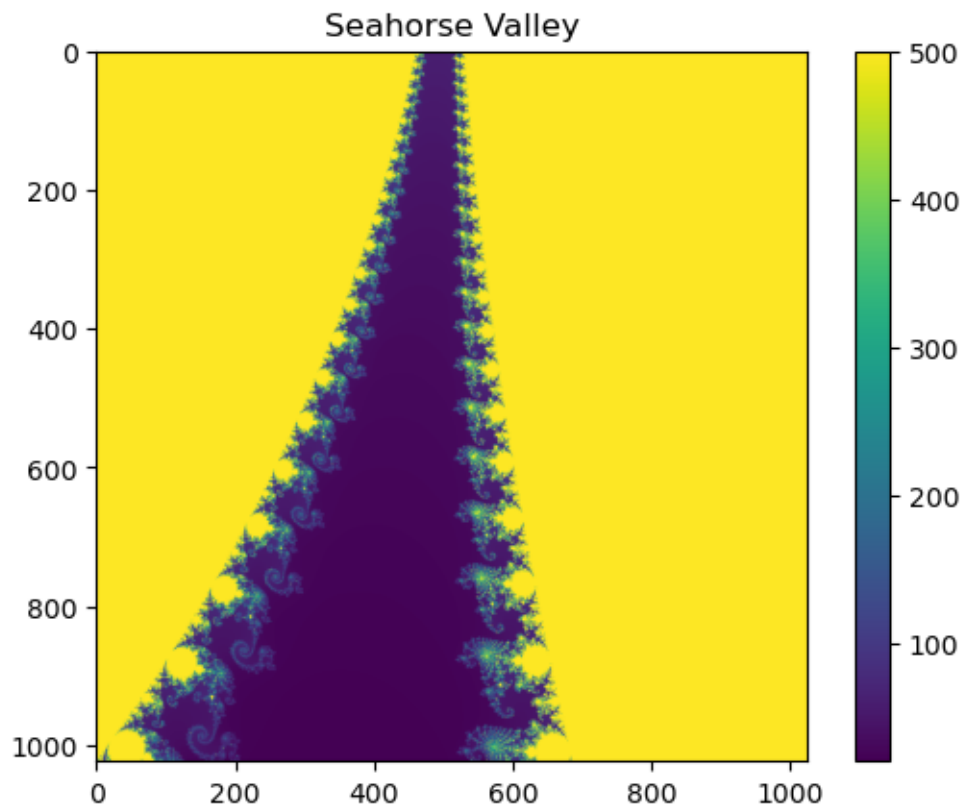
```
[23]: from mandelbrot import compute_mandelbrot_numpy

image = compute_mandelbrot_numpy(-0.8, -0.7, 0.05, 0.15, 1024, 1024, 500)
```

```
[24]: import matplotlib.pyplot as plt

plt.figure()
plt.title("Seahorse Valley")
plt.imshow(image)
```

```
plt.colorbar()
plt.show()
```



- Does runtime change? Why? (Hint: the mask)

Change compared to what? I am unsure what is meant here, so I am comparing the default region/parameters.

```
[25]: # Testing with seahorse valley

# using max_iter = 500 instead of 100
t_default, _ = benchmark(numpy_python,
                        -2, 1, -1.5, 1.5, 1024, 1024, 500)
t_seahorse, _ = benchmark(numpy_python, -0.8, -0.7,
                        0.05, 0.15, 1024, 1024, 500)

if t_default == t_seahorse:
    print("Runtime did NOT change.")
else:
    print("Runtime changed!")

print(f"Default region (median): {t_default:.3f}s")
```

```
print(f"Seahorse valley (median): {t_seahorse:.3f}s")
```

```
Median: 2.6163s(min=2.6084, max=2.7536)
```

```
Median: 6.2525s(min=6.0034, max=6.4473)
```

```
Runtime changed!
```

```
Default region (median): 2.616s
```

```
Seahorse valley (median): 6.253s
```

From the output, the runtime does change, likely due to the Seahorse Spiral region having more iterations on average per pixel than the default region.

- **Try the famous named regions:** Seahorse Valley, Elephant Valley, **or a** Deep Seahorse Spiral****

```
[26]: regions = {
    "standard": {
        "x_min": -2.0,
        "x_max": 1.0,
        "y_min": -1.5,
        "y_max": 1.5,
        "max_iter": 100
    },
    "seahorse_valley": {
        "x_min": -0.8,
        "x_max": -0.7,
        "y_min": 0.05,
        "y_max": 0.15,
        "max_iter": 500
    },
    "elephant_valley": {
        "x_min": 0.175,
        "x_max": 0.375,
        "y_min": -0.1,
        "y_max": 0.1,
        "max_iter": 500
    },
    "deep_seahorse_spiral": {
        "x_min": -0.7487667139,
        "x_max": -0.7487667078,
        "y_min": 0.1236408449,
        "y_max": 0.1236408510,
        "max_iter": 2000
    }
}
```

```
[27]: # Default image resolution
width = height = 512

for name, conf in regions.items():
```

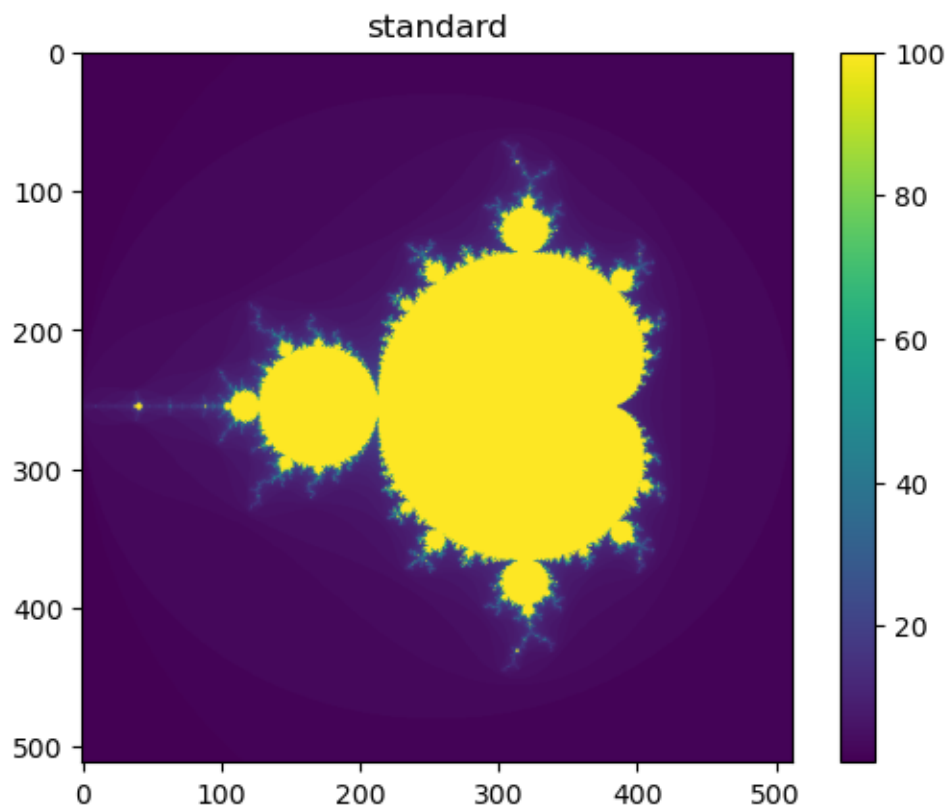
```

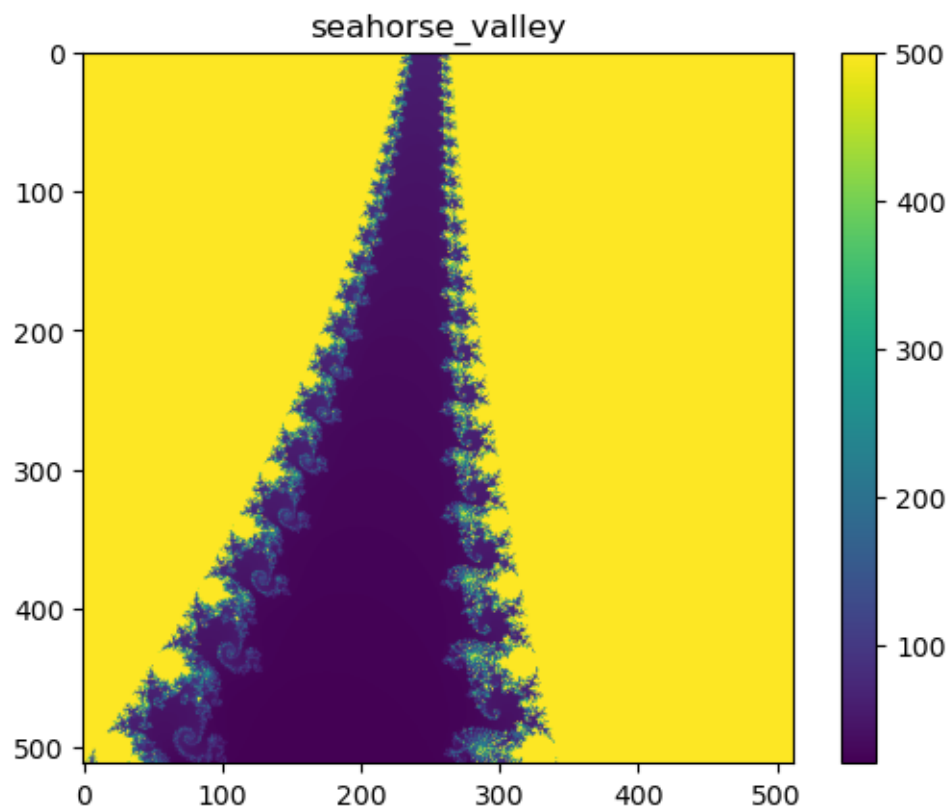
# Add in the image resolution
full_conf = {
    **conf,
    "width": width,
    "height": height,
}

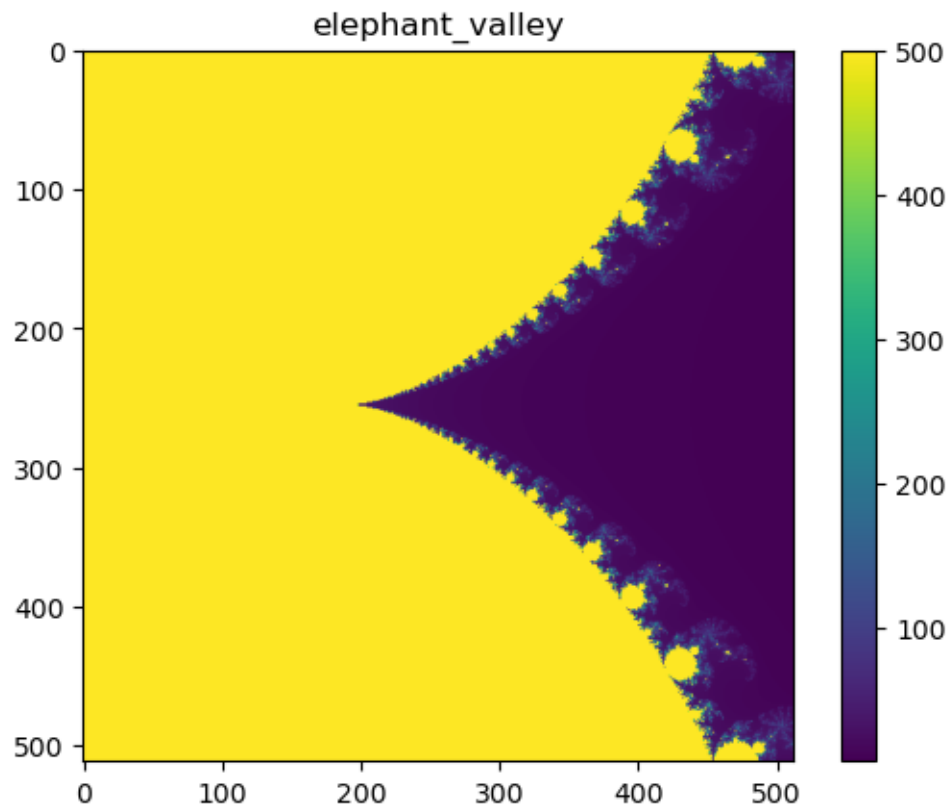
image = compute_mandelbrot_numpy(**full_conf)

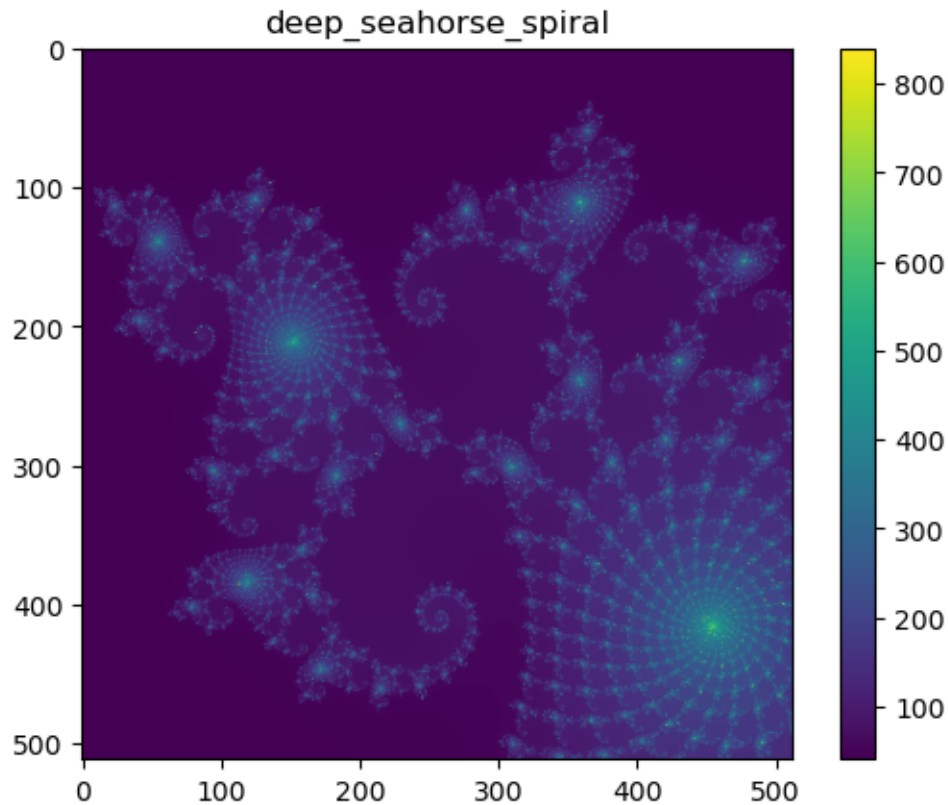
# plot
plt.figure()
plt.imshow(image)
plt.colorbar()
plt.title(name)
plt.show()
plt.close()

```









- *Coordinates for these regions are on the last backup slide*

Noted!

****Extension 3: Help Others!****

- Teaching deepens your understanding

Noted, done!

1.3 Lecture 3: Optimization and Numba

1.3.1 ****Milestone 1:** Function-level profiling with cProfile**

Use cProfile on both versions:

```
[1]: # From slide 27 (my function are named differently including max_iter)

import cProfile
import pstats
from mandelbrot import compute_mandelbrot as mandelbrot_naive
from mandelbrot import compute_mandelbrot_numpy as mandelbrot_numpy
```

```

cProfile.run('mandelbrot_naive(-2, 1, -1.5, 1.5, 512, 512, 100)',
             'naive_profile.prof')

cProfile.run('mandelbrot_numpy(-2, 1, -1.5, 1.5, 512, 512, 100)',
             'numpy_profile.prof')

for name in ('naive_profile.prof', 'numpy_profile.prof'):
    stats = pstats.Stats(name)
    stats.sort_stats('cumulative')
    stats.print_stats(10)

```

Mon Mar 2 21:15:47 2026 naive_profile.prof

6223626 function calls in 1.951 seconds

Ordered by: cumulative time

List reduced from 20 to 10 due to restriction <10>

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	1.951	1.951	{built-in method builtins.exec}
1	0.000	0.000	1.951	1.951	<string>:1(<module>)
1	0.435	0.435	1.950	1.950	
/ncs/mandelbrot.py:39(compute_mandelbrot)					
262144	1.197	0.000	1.501	0.000	
/ncs/mandelbrot.py:29(mandelbrot_point)					
5698796	0.304	0.000	0.304	0.000	{built-in method builtins.abs}
262656	0.014	0.000	0.014	0.000	{method 'append' of 'list' objects}
2	0.000	0.000	0.000	0.000	
/opt/conda/envs/nsc/lib/python3.11/site-packages/numpy/_core/function_base.py:26(linspace)					
2	0.000	0.000	0.000	0.000	
/opt/conda/envs/nsc/lib/python3.11/site-packages/numpy/_core/fromnumeric.py:3523(ndim)					
2	0.000	0.000	0.000	0.000	{built-in method numpy.arange}
2	0.000	0.000	0.000	0.000	
/opt/conda/envs/nsc/lib/python3.11/site-packages/numpy/_core/numeric.py:1964(isscalar)					

Mon Mar 2 21:15:47 2026 numpy_profile.prof

81 function calls in 0.104 seconds

Ordered by: cumulative time

List reduced from 42 to 10 due to restriction <10>

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
--------	---------	---------	---------	---------	---------------------------

```

1      0.000      0.000      0.104      0.104 {built-in method builtins.exec}
1      0.001      0.001      0.104      0.104 <string>:1(<module>)
1      0.099      0.099      0.103      0.103
/ncs/mandelbrot.py:62(compute_mandelbrot_numpy)
2      0.002      0.001      0.002      0.001
/opt/conda/envs/nsc/lib/python3.11/site-
packages/numpy/_core/numeric.py:98(zeros_like)
1      0.000      0.000      0.001      0.001
/opt/conda/envs/nsc/lib/python3.11/site-
packages/numpy/lib/_function_base_impl.py:5134(meshgrid)
3      0.000      0.000      0.001      0.000
/opt/conda/envs/nsc/lib/python3.11/site-
packages/numpy/lib/_function_base_impl.py:5280(<genexpr>)
2      0.001      0.001      0.001      0.001 {method 'copy' of 'numpy.ndarray'
objects}
2      0.000      0.000      0.000      0.000
/opt/conda/envs/nsc/lib/python3.11/site-
packages/numpy/_core/function_base.py:26(linspace)
1      0.000      0.000      0.000      0.000
/opt/conda/envs/nsc/lib/python3.11/site-
packages/numpy/lib/_stride_tricks_impl.py:481(broadcast_arrays)
1      0.000      0.000      0.000      0.000
/opt/conda/envs/nsc/lib/python3.11/site-
packages/numpy/lib/_stride_tricks_impl.py:546(<listcomp>)

```

Questions to answer: (slide 27)

- Which function takes most total time?

Naïve: `mandelbrot_point`, which is to be expected.

Numpy: `/ncs/mandelbrot.py:61(compute_mandelbrot_numpy)` which is the function itself.

- Are there functions called surprisingly many times?

Yes. `mandelbrot_point`, `append` are called tens of thousands of times, with `abs` being called 10x more often. That is a lot of function calls!

- How does the NumPy profile compare to naive?

From the output, it appears that numpy is able to compute most of the work inside of itself, without python being able “to see” it i.e., number of function calls. I am guessing that this is also due to the binary mask.

****Done?**** Add the cProfile output for both versions and your answers to the questions above to your mini-report → commit****

```

(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 3701704954893c917622f4ffc45c6e5343908271 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>

```

Date: Mon Mar 2 21:17:26 2026 +0000

103: milestone 1, half done

Questions to answer: (slide 28)

1. Which function takes most total time?

Please see the previous answers above.

2. Are there functions called surprisingly many times?

Please see the previous answers above.

3. How does NumPy profile compare to naive?

Please see the previous answers above.

4. Where does NumPy spend its time?

/ncs/mandelbrot.py which is the function itself.

Document your findings! Add a summary table to your mini-report.

1.3.2 ****Milestone 2:** Line-level profiling with line_profiler**

Add @profile decorator and run line profiler:

```
[2]: import numpy as np
from memory_profiler import profile

# from slide 29

@profile # Add this decorator
def mandelbrot_naive_profile(xmin, xmax, ymin, ymax, width, height,
    ↪max_iter=100):
    x = np.linspace(xmin, xmax, width)
    y = np.linspace(ymin, ymax, height)
    result = np.zeros((height, width), dtype=int)
    for i in range(height):
        for j in range(width):
            c = x[j] + 1j * y[i]
            z = 0
            for n in range(max_iter):
                if abs(z) > 2:
                    result[i, j] = n
                    break
            z = z*z + c
        else:
            result[i, j] = max_iter

    return result
```

```
kernprof -l -v mandelbrot_script.py # or use Spyder: Run -> Profile (F10)
```

Before we can run kernprof, we need to install it from the line_profiler package.

```
[3]: !mamba clean --all --yes
```

```
Collect information..  
Cleaning index cache..  
Cleaning lock files..  
Cleaning tarballs..  
Cleaning packages..
```

```
[4]: !mamba install --name nsc line_profiler -y
```

```
[+] 0.0s  
[+] 0.1s  
conda-forge/linux-64          0.0 B  
/  ???.?MB @  ???.?MB/s  0.0s  
conda-forge/noarch           0.0 B /  ???.?MB  
@  ???.?MB/s  0.0s[+] 0.2s  
conda-forge/linux-64         401.4kB /  51.4MB @  
2.6MB/s  0.1s  
conda-forge/noarch           0.0 B /  ???.?MB  
@  ???.?MB/s  0.1s[+] 0.3s  
conda-forge/linux-64         1.6MB /  51.4MB @  
6.2MB/s  0.2s  
conda-forge/noarch           210.6kB /  24.9MB @  
813.9kB/s  0.2s[+] 0.4s  
conda-forge/linux-64         2.2MB /  51.4MB @  
6.3MB/s  0.3s  
conda-forge/noarch           303.1kB /  24.9MB @  
866.5kB/s  0.3s[+] 0.5s  
conda-forge/linux-64         3.8MB /  51.4MB @  
8.3MB/s  0.4s  
conda-forge/noarch           772.5kB /  24.9MB @  
1.7MB/s  0.4s[+] 0.6s  
conda-forge/linux-64         4.9MB /  51.4MB @  
8.9MB/s  0.5s  
conda-forge/noarch           1.1MB /  24.9MB @  
1.9MB/s  0.5s[+] 0.7s  
conda-forge/linux-64         6.0MB /  51.4MB @  
9.2MB/s  0.6s  
conda-forge/noarch           1.4MB /  24.9MB @  
2.1MB/s  0.6s[+] 0.8s  
conda-forge/linux-64         7.1MB /  51.4MB @  
9.4MB/s  0.7s  
conda-forge/noarch           1.7MB /  24.9MB @  
2.3MB/s  0.7s[+] 0.9s  
conda-forge/linux-64         8.1MB /  51.4MB @
```

9.5MB/s 0.8s	
conda-forge/noarch	2.1MB / 24.9MB @
2.5MB/s 0.8s[+] 1.0s	
conda-forge/linux-64	9.2MB / 51.4MB @
9.6MB/s 0.9s	
conda-forge/noarch	2.5MB / 24.9MB @
2.6MB/s 0.9s[+] 1.1s	
conda-forge/linux-64	10.2MB / 51.4MB @
9.7MB/s 1.0s	
conda-forge/noarch	2.9MB / 24.9MB @
2.7MB/s 1.0s[+] 1.2s	
conda-forge/linux-64	11.3MB / 51.4MB @
9.7MB/s 1.1s	
conda-forge/noarch	3.2MB / 24.9MB @
2.8MB/s 1.1s[+] 1.3s	
conda-forge/linux-64	12.3MB / 51.4MB @
9.8MB/s 1.2s	
conda-forge/noarch	3.6MB / 24.9MB @
2.9MB/s 1.2s[+] 1.4s	
conda-forge/linux-64	13.3MB / 51.4MB @
9.8MB/s 1.3s	
conda-forge/noarch	4.0MB / 24.9MB @
3.0MB/s 1.3s[+] 1.5s	
conda-forge/linux-64	14.4MB / 51.4MB @
9.8MB/s 1.4s	
conda-forge/noarch	4.4MB / 24.9MB @
3.0MB/s 1.4s[+] 1.6s	
conda-forge/linux-64	15.4MB / 51.4MB @
9.8MB/s 1.5s	
conda-forge/noarch	4.9MB / 24.9MB @
3.1MB/s 1.5s[+] 1.7s	
conda-forge/linux-64	16.3MB / 51.4MB @
9.8MB/s 1.6s	
conda-forge/noarch	5.3MB / 24.9MB @
3.2MB/s 1.6s[+] 1.8s	
conda-forge/linux-64	17.3MB / 51.4MB @
9.8MB/s 1.7s	
conda-forge/noarch	5.8MB / 24.9MB @
3.3MB/s 1.7s[+] 1.9s	
conda-forge/linux-64	18.2MB / 51.4MB @
9.8MB/s 1.8s	
conda-forge/noarch	6.3MB / 24.9MB @
3.4MB/s 1.8s[+] 2.0s	
conda-forge/linux-64	19.2MB / 51.4MB @
9.8MB/s 1.9s	
conda-forge/noarch	6.7MB / 24.9MB @
3.4MB/s 1.9s[+] 2.1s	
conda-forge/linux-64	20.1MB / 51.4MB @

9.8MB/s 2.0s	
conda-forge/noarch	7.2MB / 24.9MB @
3.5MB/s 2.0s[+] 2.2s	
conda-forge/linux-64	21.1MB / 51.4MB @
9.7MB/s 2.1s	
conda-forge/noarch	7.7MB / 24.9MB @
3.5MB/s 2.1s[+] 2.3s	
conda-forge/linux-64	22.1MB / 51.4MB @
9.7MB/s 2.2s	
conda-forge/noarch	8.1MB / 24.9MB @
3.6MB/s 2.2s[+] 2.4s	
conda-forge/linux-64	23.0MB / 51.4MB @
9.7MB/s 2.3s	
conda-forge/noarch	8.6MB / 24.9MB @
3.6MB/s 2.3s[+] 2.5s	
conda-forge/linux-64	24.0MB / 51.4MB @
9.7MB/s 2.4s	
conda-forge/noarch	9.1MB / 24.9MB @
3.7MB/s 2.4s[+] 2.6s	
conda-forge/linux-64	25.2MB / 51.4MB @
9.8MB/s 2.5s	
conda-forge/noarch	9.2MB / 24.9MB @
3.6MB/s 2.5s[+] 2.7s	
conda-forge/linux-64	26.3MB / 51.4MB @
9.8MB/s 2.6s	
conda-forge/noarch	9.5MB / 24.9MB @
3.6MB/s 2.6s[+] 2.8s	
conda-forge/linux-64	27.3MB / 51.4MB @
9.8MB/s 2.7s	
conda-forge/noarch	9.9MB / 24.9MB @
3.6MB/s 2.7s[+] 2.9s	
conda-forge/linux-64	28.3MB / 51.4MB @
9.8MB/s 2.8s	
conda-forge/noarch	10.4MB / 24.9MB @
3.6MB/s 2.8s[+] 3.0s	
conda-forge/linux-64	29.3MB / 51.4MB @
9.8MB/s 2.9s	
conda-forge/noarch	10.8MB / 24.9MB @
3.6MB/s 2.9s[+] 3.1s	
conda-forge/linux-64	30.1MB / 51.4MB @
9.8MB/s 3.0s	
conda-forge/noarch	11.1MB / 24.9MB @
3.6MB/s 3.0s[+] 3.2s	
conda-forge/linux-64	31.4MB / 51.4MB @
9.8MB/s 3.1s	
conda-forge/noarch	11.5MB / 24.9MB @
3.6MB/s 3.1s[+] 3.3s	
conda-forge/linux-64	31.8MB / 51.4MB @

9.7MB/s 3.2s	
conda-forge/noarch	11.7MB / 24.9MB @
3.6MB/s 3.2s[+] 3.4s	
conda-forge/linux-64	32.8MB / 51.4MB @
9.7MB/s 3.3s	
conda-forge/noarch	11.9MB / 24.9MB @
3.5MB/s 3.3s[+] 3.5s	
conda-forge/linux-64	33.8MB / 51.4MB @
9.7MB/s 3.4s	
conda-forge/noarch	12.5MB / 24.9MB @
3.6MB/s 3.4s[+] 3.6s	
conda-forge/linux-64	34.8MB / 51.4MB @
9.7MB/s 3.5s	
conda-forge/noarch	12.7MB / 24.9MB @
3.6MB/s 3.5s[+] 3.7s	
conda-forge/linux-64	35.7MB / 51.4MB @
9.7MB/s 3.6s	
conda-forge/noarch	13.3MB / 24.9MB @
3.6MB/s 3.6s[+] 3.8s	
conda-forge/linux-64	36.6MB / 51.4MB @
9.7MB/s 3.7s	
conda-forge/noarch	13.8MB / 24.9MB @
3.6MB/s 3.7s[+] 3.9s	
conda-forge/linux-64	37.4MB / 51.4MB @
9.6MB/s 3.8s	
conda-forge/noarch	14.1MB / 24.9MB @
3.6MB/s 3.8s[+] 4.0s	
conda-forge/linux-64	38.4MB / 51.4MB @
9.6MB/s 3.9s	
conda-forge/noarch	14.5MB / 24.9MB @
3.6MB/s 3.9s[+] 4.1s	
conda-forge/linux-64	39.3MB / 51.4MB @
9.6MB/s 4.0s	
conda-forge/noarch	14.9MB / 24.9MB @
3.6MB/s 4.0s[+] 4.2s	
conda-forge/linux-64	40.2MB / 51.4MB @
9.6MB/s 4.1s	
conda-forge/noarch	15.2MB / 24.9MB @
3.6MB/s 4.1s[+] 4.3s	
conda-forge/linux-64	40.7MB / 51.4MB @
9.6MB/s 4.2s	
conda-forge/noarch	15.4MB / 24.9MB @
3.6MB/s 4.2s[+] 4.4s	
conda-forge/linux-64	41.5MB / 51.4MB @
9.5MB/s 4.3s	
conda-forge/noarch	15.8MB / 24.9MB @
3.6MB/s 4.3s[+] 4.5s	
conda-forge/linux-64	42.4MB / 51.4MB @

9.5MB/s	4.4s		
conda-forge/noarch		16.2MB /	24.9MB @
3.6MB/s	4.4s[+] 4.6s		
conda-forge/linux-64		43.5MB /	51.4MB @
9.5MB/s	4.5s		
conda-forge/noarch		16.6MB /	24.9MB @
3.6MB/s	4.5s[+] 4.7s		
conda-forge/linux-64		44.4MB /	51.4MB @
9.5MB/s	4.6s		
conda-forge/noarch		16.8MB /	24.9MB @
3.6MB/s	4.6s[+] 4.8s		
conda-forge/linux-64		45.4MB /	51.4MB @
9.5MB/s	4.7s		
conda-forge/noarch		17.3MB /	24.9MB @
3.6MB/s	4.7s[+] 4.9s		
conda-forge/linux-64		46.1MB /	51.4MB @
9.4MB/s	4.8s		
conda-forge/noarch		17.6MB /	24.9MB @
3.6MB/s	4.8s[+] 5.0s		
conda-forge/linux-64		47.2MB /	51.4MB @
9.5MB/s	4.9s		
conda-forge/noarch		17.9MB /	24.9MB @
3.6MB/s	4.9s[+] 5.1s		
conda-forge/linux-64		48.2MB /	51.4MB @
9.5MB/s	5.0s		
conda-forge/noarch		18.2MB /	24.9MB @
3.6MB/s	5.0s[+] 5.2s		
conda-forge/linux-64		48.7MB /	51.4MB @
9.5MB/s	5.1s		
conda-forge/noarch		18.4MB /	24.9MB @
3.6MB/s	5.1s[+] 5.3s		
conda-forge/linux-64		49.8MB /	51.4MB @
9.5MB/s	5.2s		
conda-forge/noarch		18.6MB /	24.9MB @
3.5MB/s	5.2s[+] 5.4s		
conda-forge/linux-64		50.9MB /	51.4MB @
9.5MB/s	5.3s		
conda-forge/noarch		18.9MB /	24.9MB @
3.5MB/s	5.3s[+] 5.5s		
conda-forge/linux-64		51.4MB /	51.4MB @
9.5MB/s	5.4s		
conda-forge/noarch		19.0MB /	24.9MB @
3.5MB/s	5.4s[+] 5.6s		
conda-forge/linux-64		51.4MB /	51.4MB @
9.5MB/s	5.5s		
conda-forge/noarch		19.0MB /	24.9MB @
3.5MB/s	5.5s	conda-forge/linux-64	
51.4MB @	9.5MB/s	5.6s	

```
[+] 5.7s
conda-forge/noarch          19.9MB /  24.9MB @
3.5MB/s  5.6s[+] 5.8s
conda-forge/noarch          20.2MB /  24.9MB @
3.5MB/s  5.7s[+] 5.9s
conda-forge/noarch          20.5MB /  24.9MB @
3.5MB/s  5.8s[+] 6.0s
conda-forge/noarch          20.9MB /  24.9MB @
3.5MB/s  5.9s[+] 6.1s
conda-forge/noarch          21.4MB /  24.9MB @
3.5MB/s  6.0s[+] 6.2s
conda-forge/noarch          22.0MB /  24.9MB @
3.6MB/s  6.1s[+] 6.3s
conda-forge/noarch          22.7MB /  24.9MB @
3.6MB/s  6.2s[+] 6.4s
conda-forge/noarch          23.5MB /  24.9MB @
3.7MB/s  6.3s[+] 6.5s
conda-forge/noarch          24.6MB /  24.9MB @
3.8MB/s  6.4s[+] 6.6s
conda-forge/noarch          24.6MB /  24.9MB @
3.8MB/s  6.5sconda-forge/noarch
24.9MB @  3.8MB/s  6.6s
```

Pinned packages:

```
- python=3.11
```

Transaction

Prefix: /opt/conda/envs/nsc

All requested packages already installed

Transaction starting

```
[+] 0.0s
```

Transaction finished

```
[5]: # Profile the function. Do not profile the profiler (See table on slide 30)
from mandelbrot import compute_mandelbrot as mandelbrot_naive
from mandelbrot import compute_mandelbrot_numpy as mandelbrot_numpy
```

```
[6]: %load_ext line_profiler
```

```
[7]: %lprun -f mandelbrot_naive mandelbrot_naive(-2, 1, -1.5, 1.5, 512, 512, 100)
```

Timer unit: 1e-09 s

Total time: 2.59478 s

File: /ncs/mandelbrot.py

Function: compute_mandelbrot at line 39

Line #	Hits	Time	Per Hit	% Time	Line Contents
39					def compute_mandelbrot(40
41					x_min: float, 42
43					x_max: float, 44
45					y_min: float, 46
47					y_max: float, 48
49					width: int, 50
51					height: int, 52
53					max_iter: int 54
55					) -> list[list[int]]: 56
57	1	179637.0	179637.0	0.0	x = np.linspace(x_min, 58
59					x_max, width) 60
61	1	52749.0	52749.0	0.0	y = np.linspace(y_min, 62
63					y_max, height) 64
65					img = [] 66
67					for i in range(height): 68
69					img_row = [] 70
71	262656	25100764.0	95.6	1.0	for j in range(width): 72
73	262144	2569129024.0	9800.4	99.0	img_row.append(mandelbrot_point(x[j] + 1j * y[i], max_iter)) 74
75	512	151314.0	295.5	0.0	img.append(img_row) 76
77					return img

Load the line profiler into ipython, available via the magic command lprun.

We can see that appending and computing the mandelbrot point is very expensive in terms of time, taking 99% of the total running time.

****Done?*** Commit → continue to profiling report**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit cf632815d5ff453831b97b0f40034955dc9a3ef0 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 21:32:01 2026 +0000
```

103: milestone 2, half done

Rule:** Use `line_profiler` to find *which line* is the bottleneck. Once you know, **remove** `@profile` and benchmark with plain `perf_counter`

Support your answers with numbers from your profiling output — copy-pasting terminal output directly into your mini-report is fine.

Add a profiling section to your MP1 mini-report. In your own words, answer:

1. cProfile on naive vs NumPy:** How many functions appear in each profile? What does this difference tell you about where the work actually happens?*

```
[8]: # Mostly from slide 27

from mandelbrot import compute_mandelbrot as mandelbrot_naive
from mandelbrot import compute_mandelbrot_numpy as mandelbrot_numpy

cProfile.run('mandelbrot_naive(-2, 1, -1.5, 1.5, 512, 512, 100)',
             'naive_profile.prof')

cProfile.run('mandelbrot_numpy(-2, 1, -1.5, 1.5, 512, 512, 100)',
             'numpy_profile.prof')

for name in ('naive_profile.prof', 'numpy_profile.prof'):
    stats = pstats.Stats(name)
    print(f"{name}: {len(stats.stats)}")
```

naive_profile.prof: 20

numpy_profile.prof: 42

It appears that numpy has more than twice the number of functions involved. I assume that this is due to numpy being more fragmented, where functions are only good at one thing.

2. line_profiler on naive:** Which lines dominate runtime? What fraction of total time is spent in the inner loop?*

```
[9]: %lprun -f mandelbrot_naive mandelbrot_naive(-2, 1, -1.5, 1.5, 512, 512, 100) #_
      ↪ same as before
```

Timer unit: 1e-09 s

Total time: 2.32989 s

File: /ncs/mandelbrot.py

Function: compute_mandelbrot at line 39

Line #	Hits	Time	Per Hit	% Time	Line Contents
39					def compute_mandelbrot(
40					x_min: float,
41					x_max: float,
42					y_min: float,
43					y_max: float,

```

44                                     width: int,
45                                     height: int,
46                                     max_iter: int
47                                     ) -> list[list[int]]:
48             1      149611.0 149611.0      0.0      x = np.linspace(x_min,
x_max, width)
49             1      51707.0 51707.0      0.0      y = np.linspace(y_min,
y_max, height)
50
51             1      221.0    221.0      0.0      img = []
52
53             513      57258.0    111.6      0.0      for i in range(height):
54             512      92187.0    180.1      0.0          img_row = []
55             262656    22888010.0    87.1      1.0          for j in range(width):
56             262144    2306500838.0    8798.6    99.0
img_row.append(mandelbrot_point(x[j] + 1j * y[i], max_iter))
57             512      150716.0    294.4      0.0          img.append(img_row)
58
59             1      100.0    100.0      0.0      return img

```

We can see that line 55 is *the* dominant runtime.

The inner loop is line 55-56, spending 100%.

3. Based on your profiling results: why is NumPy faster than naive Python?

Numpy moves the inner loop into compiled C code (broadcasting), which is much faster than pure python.

4. What would you need to change to make the naive version faster? (hint: what does line profiler tell you about the inner loop?)

`mandelbrot_point` and python `.append()` should be reworked. For example, preallocating the grid/image and inlining the point computation, perhaps even compiling it.

****Done? Commit → move on to Milestone 3 (Numba)****

```

(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit b63cc9b52486ab03b0c56e946d2b5e27b165f847 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date:   Mon Mar 2 21:38:44 2026 +0000

```

l03: milestone 2 done

1.3.3 ****Milestone 3: Implement Numba @njit and benchmark****

```

[10]: from numba import njit

# From slide 36. Approach A (where I have filled in the gaps in the hybrid)

```

```

@njit
def mandelbrot_point_numba(c, max_iter=100):
    z = 0j
    for n in range(max_iter):
        if z.real*z.real+z.imag*z.imag > 4.0:
            return n
        z = z*z + c
    return max_iter

def mandelbrot_hybrid(xmin, xmax, ymin, ymax, width, height, max_iter=100):
    x = np.linspace(xmin, xmax, width)
    y = np.linspace(ymin, ymax, height)
    result = np.zeros((height, width), dtype=np.int32)

    # outer loops still in Python
    for i in range(height):
        for j in range(width):
            c = x[j] + 1j * y[i]
            result[i, j] = mandelbrot_point_numba(c, max_iter)

```

```

[11]: # from slide 37. Approach B
from numba import njit
import numpy as np

@njit                                     # <-- entire function compiled !
def mandelbrot_naive_numba(xmin, xmax, ymin, ymax, width, height, max_iter=100):
    """Fully JIT - compiled Mandelbrot --- structure identical to naive."""
    x = np.linspace(xmin, xmax, width)
    y = np.linspace(ymin, ymax, height)
    result = np.zeros((height, width), dtype=np.int32)

    for i in range(height):               # compiled loop
        for j in range(width):            # compiled loop
            c = x[j] + 1j * y[i]
            z = 0j                         # complex literal: type inference works!
            n = 0
            while n < max_iter and (z.real*z.real + z.imag*z.imag) <= 4.0:
                z = z*z + c
                n += 1
            result[i, j] = n
    return result

# Warm - up (triggers compilation --- don 't time this!)
_ = mandelbrot_naive_numba(-2, 1, -1.5, 1.5, 64, 64)

```

****Your Task:** Implement both approaches and measure the difference.**

```
[12]: # From slide 38
import time
import statistics

def bench(fn, *args, runs=5):
    fn(*args) # extra warm-up
    times = []
    for _ in range(runs):
        t0 = time.perf_counter()
        fn(*args)
        times.append(time.perf_counter() - t0)
    return statistics.median(times)

# Warm up (triggers JIT compilation -- exclude from timing)
_ = mandelbrot_hybrid(-2, 1, -1.5, 1.5, 64, 64)
_ = mandelbrot_naive_numba(-2, 1, -1.5, 1.5, 64, 64)

t_hybrid = bench(mandelbrot_hybrid, -2, 1, -1.5, 1.5, 1024, 1024)
t_full = bench(mandelbrot_naive_numba, -2, 1, -1.5, 1.5, 1024, 1024)

print(f"Hybrid:           {t_hybrid:.3f}s")
print(f"Fully compiled: {t_full:.3f}s")
print(f"Ratio:             {t_hybrid/t_full:.1f}x")
```

```
Hybrid:           1.509s
Fully compiled: 0.045s
Ratio:            33.3x
```

Since we are provided with the code, I assume that this is the code I *must* use.

****Done?** Commit → log in Performance Tracker**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 97144fb7739f7214faf838257d394995d9b084f4 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 21:42:59 2026 +0000
```

103: milestone 3 almost done

```
[13]: # From slide 39, but with max iter included
import time
import statistics
```



```
def bench(fn, *args, runs=5):
    fn(*args) # warm-up
    times = []
    for _ in range(runs):
        t0 = time.perf_counter()
        fn(*args)
        times.append(time.perf_counter() - t0)
    return statistics.median(times)

width, height = 1024, 1024
args = (-2, 1, -1.5, 1.5, width, height, 100) # include max iter

t_naive = bench(mandelbrot_naive, *args)
t_numpy = bench(mandelbrot_numpy, *args)
t_numba = bench(mandelbrot_naive_numba, *args)

print(f"Naive: {t_naive:.3f}s")
print(f"NumPy: {t_numpy:.3f}s ({t_naive/t_numpy:.1f}x)")
print(f"Numba: {t_numba:.3f}s ({t_naive/t_numba:.1f}x)")
```

Naive: 4.113s

NumPy: 0.573s (7.2x)

Numba: 0.048s (86.6x)

****Done? Add timings and speedup table (naive / NumPy / Numba) to your mini-report → log in Performance Tracker → commit**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit eec7018d9c7270f98d237026c39807f494a13604 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 21:47:43 2026 +0000
```

103: milestone 3 done

1.3.4 ****Milestone 4:** Data type optimization (float32 vs float64)**

Modify your Numba version to accept a dtype parameter:

```
[14]: try:
    # From slide 42
    from numba import njit
    import numpy as np
    import time

    @njit
    def mandelbrot_numba_typed(xmin, xmax, ymin, ymax,
                               width, height, max_iter=100, dtype=np.float64):
```

```

x = np.linspace(xmin, xmax, width).astype(dtype)
y = np.linspace(ymin, ymax, height).astype(dtype)
result = np.zeros((height, width), dtype=np.int32)
for i in range(height):
    for j in range(width):
        c = x[j] + 1j * y[i]
        result[i, j] = mandelbrot_point_numba(c, max_iter)
return result

for dtype in [np.float16, np.float32, np.float64]:
    t0 = time.perf_counter()
    mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=dtype)
    print(f"{dtype.__name__}: {time.perf_counter()-t0:.3f}s")

except Exception:
    import traceback
    traceback.print_exc()

```

```

Traceback (most recent call last):
  File "/tmp/ipykernel_1321/793518995.py", line 21, in <module>
    mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=dtype)
  File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/dispatcher.py", line 443, in _compile_for_args
    raise e
  File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/dispatcher.py", line 376, in _compile_for_args
    return_val = self.compile(tuple(argtypes))
    ~~~~~
  File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/dispatcher.py", line 908, in compile
    cres = self._compiler.compile(args, return_type)
    ~~~~~
  File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/dispatcher.py", line 80, in compile
    status, retval = self._compile_cached(args, return_type)
    ~~~~~
  File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/dispatcher.py", line 94, in _compile_cached
    retval = self._compile_core(args, return_type)
    ~~~~~
  File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/dispatcher.py", line 107, in _compile_core
    cres = compiler.compile_extra(self.targetdescr.typing_context,
    ~~~~~
  File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler.py", line 739, in compile_extra
    return pipeline.compile_extra(func)
    ~~~~~

```

```

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler.py", line 439, in compile_extra
    return self._compile_bytecode()
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler.py", line 505, in _compile_bytecode
    return self._compile_core()
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler.py", line 481, in _compile_core
    raise e

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler.py", line 473, in _compile_core
    pm.run(self.state)

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler_machinery.py", line 363, in run
    raise e

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler_machinery.py", line 356, in run
    self._runPass(idx, pass_inst, state)

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler_lock.py", line 35, in _acquire_compile_lock
    return func(*args, **kwargs)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler_machinery.py", line 311, in _runPass
    mutated |= check(pss.run_pass, internal_state)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/compiler_machinery.py", line 272, in check
    mangled = func(compiler_state)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/typed_passes.py", line 472, in run_pass
    lower.lower()

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 193, in lower
    self.lower_normal_function(self.fndesc)

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 232, in lower_normal_function
    entry_block_tail = self.lower_function_body()
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 262, in lower_function_body
    self.lower_block(block)

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 276, in lower_block
    self.lower_inst(inst)

```

```

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 462, in lower_inst
    val = self.lower_assign(ty, inst)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 674, in lower_assign
    return self.lower_expr(ty, value)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 1268, in lower_expr
    res = self.lower_call(resty, expr)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 939, in lower_call
    res = self._lower_call_normal(fnty, expr, signature)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/lowering.py", line 1239, in _lower_call_normal
    res = impl(self.builder, argvals, self.loc)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-packages/numba/core/base.py",
line 1196, in __call__
    res = self._imp(self._context, builder, self._sig, args, loc=loc)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-packages/numba/core/base.py",
line 1226, in wrapper
    return fn(*args, **kwargs)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-packages/numba/np/arrayobj.py",
line 5219, in array_astype
    ret = _empty_nd_impl(context, builder, rettype, shapes)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-packages/numba/np/arrayobj.py",
line 4263, in _empty_nd_impl
    ary = arycls(context, builder)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-packages/numba/core/cgutils.py",
line 103, in __init__
    self._datamodel = self._context.data_model_manager[self._fe_type]
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/manager.py", line 43, in __getitem__
    return self.lookup(fetype)
    ~~~~~

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/manager.py", line 37, in lookup
    model = self._cache[fetype] = handler(self, fetype)
    ~~~~~

```

```

File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/models.py", line 885, in __init__
    super(ArrayModel, self).__init__(dmm, fe_type, members)
File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/models.py", line 524, in __init__
    self._models = tuple([self._dmm.lookup(t) for t in self._members])
    ~~~~~
File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/models.py", line 524, in <listcomp>
    self._models = tuple([self._dmm.lookup(t) for t in self._members])
    ~~~~~
File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/manager.py", line 37, in lookup
    model = self._cache[fetype] = handler(self, fetype)
    ~~~~~
File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/models.py", line 377, in __init__
    self._pointee_model = dmm.lookup(fe_type.dtype)
    ~~~~~
File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/manager.py", line 37, in lookup
    model = self._cache[fetype] = handler(self, fetype)
    ~~~~~
File "/opt/conda/envs/nsc/lib/python3.11/site-
packages/numba/core/datamodel/models.py", line 370, in __init__
    raise NotImplementedError(fe_type)
NotImplementedError: float16

```

It seems like float16 is not supported/implemented (yet), so exclude it.

```

[15]: # From slide 42 without np.float16
import numpy as np
import time

for dtype in [np.float32, np.float64]:
    t0 = time.perf_counter()
    mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=dtype)
    print(f"{dtype.__name__}: {time.perf_counter()-t0:.3f}s")

```

float32: 0.226s

float64: 0.208s

Compare accuracy and generate side-by-side images:

```

[16]: # From slide 43, without float16
import matplotlib.pyplot as plt
import numpy as np

r32 = mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=np.float32)

```

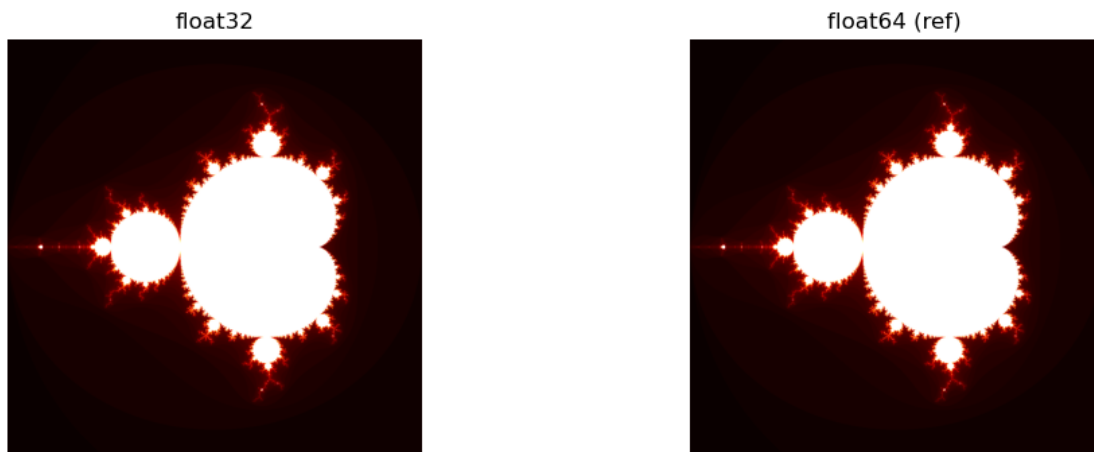
```

r64 = mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=np.float64)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, result, title in zip(axes, [r32, r64],
                              ['float32', 'float64 (ref)']):
    ax.imshow(result, cmap='hot')
    ax.set_title(title)
    ax.axis('off')
    plt.savefig('precision_comparison.png', dpi=150)
print(f"Max diff float32 vs float64: {np.abs(r32-r64).max()}")

```

Max diff float32 vs float64: 33



Run your experiments and observe:

- **Speed:** Does float32 actually run faster than float64 on your hardware? By how much? Is float16 faster or slower — and why might that surprise you?

```

[17]: t_f32 = bench(mandelbrot_numba_typed,
                  -2, 1, -1.5, 1.5, 1024, 1024, 100, np.float32, runs=100)
t_f64 = bench(mandelbrot_numba_typed,
              -2, 1, -1.5, 1.5, 1024, 1024, 100, np.float64, runs=100)

print(f"Using float32 (median): {t_f32:.3f}s")
print(f"Using float64 (median): {t_f64:.3f}s")

```

Using float32 (median): 0.045s

Using float64 (median): 0.045s

They are exactly the same. It seems that my hardware (CPU) does not support float16 in hardware, instead converting (perhaps at compile time) to float64.

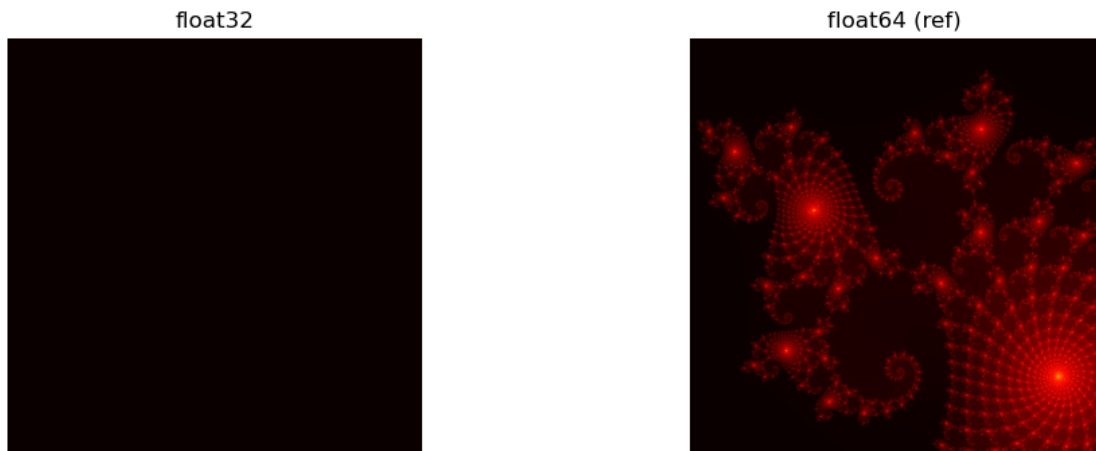
- **Visual quality:** Zoom in on a detailed region of the Mandelbrot set. Can you see artefacts with float16? What about float32?

I am using the region *Deep Seahorse Spiral* (parameters from last lecture). My computer does not support float16 with numba (yet).

```
[18]: r32_seahorse = mandelbrot_numba_typed(-0.7487667139, -0.7487667078, 0.
      ↪1236408449, 0.1236408510,
      1024, 1024, 2000, np.float32)
r64_seahorse = mandelbrot_numba_typed(-0.7487667139, -0.7487667078, 0.
      ↪1236408449, 0.1236408510,
      1024, 1024, 2000, np.float64)

# copied mostly from slide 43
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, result, title in zip(axes, [r32_seahorse, r64_seahorse],
                              ['float32', 'float64 (ref)']):
    ax.imshow(result, cmap='hot')
    ax.set_title(title)
    ax.axis('off')
print(
    f"Max diff float32 vs float64: {np.abs(r32_seahorse-r64_seahorse).max()}")
```

Max diff float32 vs float64: 945



- ****Recommendation:** Based on what you observe, which precision would you choose for production use, and why?*

For detailed regions, such as the deep seahorse spiral, one should choose float64. For *production use*, precision is often not that important and the massive memory savings (about half the usage) is often worth it, although it results in lower detailed images.

Hint: The answer is not always obvious — modern CPUs handle different precisions differently, and the memory bandwidth argument does not always dominate.

Add a data type section to your MP1 mini-report. Include:

- Your measured runtimes for float16, float32, float64

Done. The runtime was measured above in a previous exercise.

- Side-by-side images (or a zoomed crop showing where artefacts appear)

Done. Already included in this document/notebook.

- A brief recommendation: which precision would you use and why?

Done. Already included.

No specific format required — a few sentences and your images is enough. The goal is to show you understand the precision/performance trade-off.

Noted!

****Done? Add and commit work and results.**

```
(mikkel@pella)-[~/Sync/AAU/Semester 8/github/nsc-mikkel]
$ git log -n 1
commit 3b984f5362d7b7564bc2311b9cc1185742b694e7 (HEAD -> main)
Author: mikkel-coder <mikkel.ks.sorensen@gmail.com>
Date: Mon Mar 2 21:51:55 2026 +0000
```

103: milestone 4 done

1.3.5 ****Milestone 5 (extension):** Parallel Numba with prange**

****Experiment 1:** @jit vs. @njit**

- Replace @njit with @jit

```
[19]: from numba import jit
      # From slide 42, but using jit over njit

      @jit
      def mandelbrot_numba_typed_jit(xmin, xmax, ymin, ymax,
                                     width, height, max_iter=100, dtype=np.float64):
          x = np.linspace(xmin, xmax, width).astype(dtype)
          y = np.linspace(ymin, ymax, height).astype(dtype)
          result = np.zeros((height, width), dtype=np.int32)
          for i in range(height):
              for j in range(width):
                  c = x[j] + 1j * y[i]
                  result[i, j] = mandelbrot_point_numba(c, max_iter)
          return result
```

- Measure performance difference

```
[20]: t_jit = bench(mandelbrot_numba_typed_jit, -2, 1, -1.5,
                   1.5, 1024, 1024, 100, np.float32, runs=100)
```



```
t_njit = bench(mandelbrot_numba_typed, -2, 1, -1.5, 1.5, 1024,
               1024, 100, np.float32, runs=100) # uses njit

print(f"Using @jit (median): {t_jit:.3f}s")
print(f"Using @njit (median): {t_njit:.3f}s")
```

Using @jit (median): 0.045s
Using @njit (median): 0.046s

- Usually @njit is faster

Since numba is able to fully compile the @jit, then there should be no difference. Let's check.

```
[21]: # Check if they have the same signature
jit_signatures = mandelbrot_numba_typed_jit.signatures
njit_signatures = mandelbrot_numba_typed.signatures

print(f"{jit_signatures=}")
print(f"{njit_signatures=}")

jit_signatures=[(int64, int64, float64, float64, int64, int64, int64,
class(float32))]
njit_signatures=[(int64, int64, float64, float64, int64, int64,
omitted(default=100), class(float32)), (int64, int64, float64, float64, int64,
int64, omitted(default=100), class(float64)), (int64, int64, float64, float64,
int64, int64, int64, class(float32)), (int64, int64, float64, float64, int64,
int64, int64, class(float64)), (float64, float64, float64, float64, int64,
int64, int64, class(float32)), (float64, float64, float64, float64, int64,
int64, int64, class(float64))]
```

```
[22]: import difflib

# Check if the asm code is the same
njit_signature = mandelbrot_numba_typed.signatures[0]
jit_signature = mandelbrot_numba_typed_jit.signatures[0]

njit_asm = mandelbrot_numba_typed.inspect_asm(njit_signature)
jit_asm = mandelbrot_numba_typed_jit.inspect_asm(jit_signature)

# https://docs.python.org/3/library/difflib.html
diff = difflib.unified_diff(
    njit_asm.split(),
    jit_asm.split(),
    lineterm=''
)

# print only the first like 20 lines
for line in list(diff)[:20]:
    print(line)
```

```

---
+++
@@ -37,12 +37,12 @@
.section
.ltext,"axl",@progbits
.globl
-_ZN8__main__22mandelbrot_numba_typedB3v15B38c8tJTIEFIjxB2IKSgI4CrvQC1QZ6FczSBAA
_3dExxddxx26omitted_28default_3d100_2918class_28float32_29
+_ZN8__main__26mandelbrot_numba_typed_jitB3v21B38c8tJTIEFIjxB2IKSgI4CrvQC1QZ6Fcz
SBAA_3dExxddxxx18class_28float32_29
.p2align
4
.type
-_ZN8__main__22mandelbrot_numba_typedB3v15B38c8tJTIEFIjxB2IKSgI4CrvQC1QZ6FczSBAA
_3dExxddxx26omitted_28default_3d100_2918class_28float32_29,@function
-_ZN8__main__22mandelbrot_numba_typedB3v15B38c8tJTIEFIjxB2IKSgI4CrvQC1QZ6FczSBAA
_3dExxddxx26omitted_28default_3d100_2918class_28float32_29:
+_ZN8__main__26mandelbrot_numba_typed_jitB3v21B38c8tJTIEFIjxB2IKSgI4CrvQC1QZ6Fcz
SBAA_3dExxddxxx18class_28float32_29,@function
+_ZN8__main__26mandelbrot_numba_typed_jitB3v21B38c8tJTIEFIjxB2IKSgI4CrvQC1QZ6Fcz
SBAA_3dExxddxxx18class_28float32_29:
.cfi_startproc
pushq
%rbp
@@ -69,10 +69,10 @@
.cfi_def_cfa_offset

```

They appear logically the same but are still seen as two independent different functions.

****Experiment 2:** Parallel Numba (advanced)**

```

[23]: # From slide 46 with code filled in by me
from numba import njit, prange

@njit(parallel=True)
def mandelbrot_numba_parallel(xmin, xmax, ymin, ymax, width, height,
    ↪max_iter=100):
    x = np.linspace(xmin, xmax, width)
    y = np.linspace(ymin, ymax, height)
    result = np.zeros((height, width), dtype=np.int32)

    for i in prange(height):    # Parallel!
        for j in range(width):
            c = x[j] + 1j * y[i]
            z = 0j
            n = 0
            while n < max_iter and (z.real*z.real + z.imag*z.imag) <= 4.0:

```

```

        z = z*z + c
        n += 1
    result[i, j] = n
return result

```

- Try parallel=True and prange

```

[24]: import numba as nb

t_parallel = bench(mandelbrot_numba_parallel,
                  -2, 1, -1.5, 1.5, 1024, 1024, 100, runs=100)

print(f"Using {nb.get_num_threads()} threads")
print(f"Numba parallel (median): {t_parallel:.3f}s")

```

Using 16 threads

Numba parallel (median): 0.014s

- May give 2-4× speedup on multi-core

```

[25]: t_serial = bench(mandelbrot_naive_numba,
                    -2, 1, -1.5, 1.5, 1024, 1024, 100, runs=100)
speedup = t_serial/t_parallel

print(f"Speed up using multiple threads: {speedup:.2f}x")

```

Speed up using multiple threads: 3.42x

******Experiment 3:** Scaling with grid size****

- Test 512^2 , 1024^2 , 4096^2 (or larger)

```

[26]: grid_sizes = [512, 1024, 4096]

```

- Plot grid size vs. time for all implementations

```

[27]: import json
from multiprocessing import Pool
from pathlib import Path
from mandelbrot import compute_mandelbrot as mandelbrot_naive
from mandelbrot import compute_mandelbrot_numpy as mandelbrot_numpy

implementations: list[callable] = [
    mandelbrot_naive,
    mandelbrot_numpy,
    mandelbrot_hybrid,
    mandelbrot_naive_numba,
    mandelbrot_numba_typed,
    mandelbrot_numba_typed_jit,
    # mandelbrot_numba_parallel # unsafe to compute in parallel in GNU OpenMP

```

```

]

def benchmark_implementation(task):
    print(task)
    impl, grid, runs = task
    time = bench(impl, -2, 1, -1.5, 1.5, grid, grid, 100, runs=runs)
    return (impl.__name__, grid, time)

def main(file: Path):
    benchmarking_tasks = [
        # func, grid, benchmark runs
        # 1 run or else this is going to take hours
        (impl, grid, 1) for grid in grid_sizes for impl in implementations
    ]
    with Pool() as p:
        results = p.map(benchmark_implementation, benchmarking_tasks)

    results_json = [{"impl": impl, "grid": grid, "time": time}
                     for impl, grid, time in results]
    # Also run mandelbrot_numba_parallel
    for grid in grid_sizes:
        time = bench(mandelbrot_numba_parallel, -2, 1, -
                     1.5, 1.5, grid, grid, 100, runs=1)
        results_json.append(
            {"impl": mandelbrot_numba_parallel.__name__, "grid": grid, "time":
↳time})

    with file.open("w") as fp:
        json.dump(results_json, fp)

results_file = Path("103_mandelbrot_set.json")
if not results_file.exists():
    print(f"{results_file} does not exists. Compute results... This will take
↳multiple minutes!")
    main(results_file)

```

103_mandelbrot_set.json does not exists. Compute results... This will take multiple minutes!

(<function mandelbrot_hybrid at 0x7f868f4ed940>, 512, 1)(<function compute_mandelbrot at 0x7f868f4ee660>, 512, 1)(<function compute_mandelbrot_numpy at 0x7f868f4ee700>, 512, 1)

(CPUDispatcher(<function mandelbrot_naive_numba at 0x7f85ff9754e0>), 512, 1)

```
(<function compute_mandelbrot_numpy at 0x7f868f4ee700>, 1024, 1)(<function
mandelbrot_hybrid at 0x7f868f4ed940>, 1024, 1)(<function compute_mandelbrot at
0x7f868f4ee660>, 1024, 1)(CPUDispatcher(<function mandelbrot_numba_typed at
0x7f85ff9754e0>), 512, 1)(CPUDispatcher(<function mandelbrot_numba_typed_jit at
0x7f868f4d77e0>), 512, 1)
```

```
(CPUDispatcher(<function mandelbrot_numba_typed at 0x7f868f4d77e0>), 1024,
1)(CPUDispatcher(<function mandelbrot_naive_numba at 0x7f868f4d77e0>), 1024,
1)(<function compute_mandelbrot at 0x7f868f4ee660>, 4096, 1)
```

```
(CPUDispatcher(<function mandelbrot_numba_typed_jit at 0x7f868f4d77e0>), 1024,
1)(<function mandelbrot_hybrid at 0x7f868f4ed940>, 4096, 1)
```

```
(<function compute_mandelbrot_numpy at 0x7f868f4ee700>, 4096, 1)
```

```
(CPUDispatcher(<function mandelbrot_naive_numba at 0x7f85ff40d6c0>), 4096, 1)
(CPUDispatcher(<function mandelbrot_numba_typed at 0x7f85ff40d760>), 4096, 1)
(CPUDispatcher(<function mandelbrot_numba_typed_jit at 0x7f85ff40d760>), 4096,
1)
```

```
[28]: with results_file.open("r") as fp:
        results = json.load(fp)

        for impl in results:
            print(impl)
```

```
{'impl': 'compute_mandelbrot', 'grid': 512, 'time': 1.4571719119994668}
{'impl': 'compute_mandelbrot_numpy', 'grid': 512, 'time': 0.3295421479997458}
{'impl': 'mandelbrot_hybrid', 'grid': 512, 'time': 0.9000440439995145}
{'impl': 'mandelbrot_naive_numba', 'grid': 512, 'time': 0.02468749400031811}
{'impl': 'mandelbrot_numba_typed', 'grid': 512, 'time': 0.017184722999445512}
{'impl': 'mandelbrot_numba_typed_jit', 'grid': 512, 'time':
0.016940755000177887}
{'impl': 'compute_mandelbrot', 'grid': 1024, 'time': 4.786946068999896}
{'impl': 'compute_mandelbrot_numpy', 'grid': 1024, 'time': 0.9029469560009602}
{'impl': 'mandelbrot_hybrid', 'grid': 1024, 'time': 1.9509156629992503}
{'impl': 'mandelbrot_naive_numba', 'grid': 1024, 'time': 0.07499045300028229}
{'impl': 'mandelbrot_numba_typed', 'grid': 1024, 'time': 0.07352167499993811}
{'impl': 'mandelbrot_numba_typed_jit', 'grid': 1024, 'time':
0.06801035600074101}
{'impl': 'compute_mandelbrot', 'grid': 4096, 'time': 68.36290053800076}
{'impl': 'compute_mandelbrot_numpy', 'grid': 4096, 'time': 12.724589266001203}
{'impl': 'mandelbrot_hybrid', 'grid': 4096, 'time': 27.424995904000752}
{'impl': 'mandelbrot_naive_numba', 'grid': 4096, 'time': 1.0789505940010713}
{'impl': 'mandelbrot_numba_typed', 'grid': 4096, 'time': 1.0662354139985837}
```

```
{'impl': 'mandelbrot_numba_typed_jit', 'grid': 4096, 'time': 1.0467227490007645}
{'impl': 'mandelbrot_numba_parallel', 'grid': 512, 'time': 0.002598298999146209}
{'impl': 'mandelbrot_numba_parallel', 'grid': 1024, 'time':
0.010538250000536209}
{'impl': 'mandelbrot_numba_parallel', 'grid': 4096, 'time': 0.1769641610007966}
```

```
[29]: # AI assisted
implementations: list[str] = sorted({entry["impl"] for entry in results})

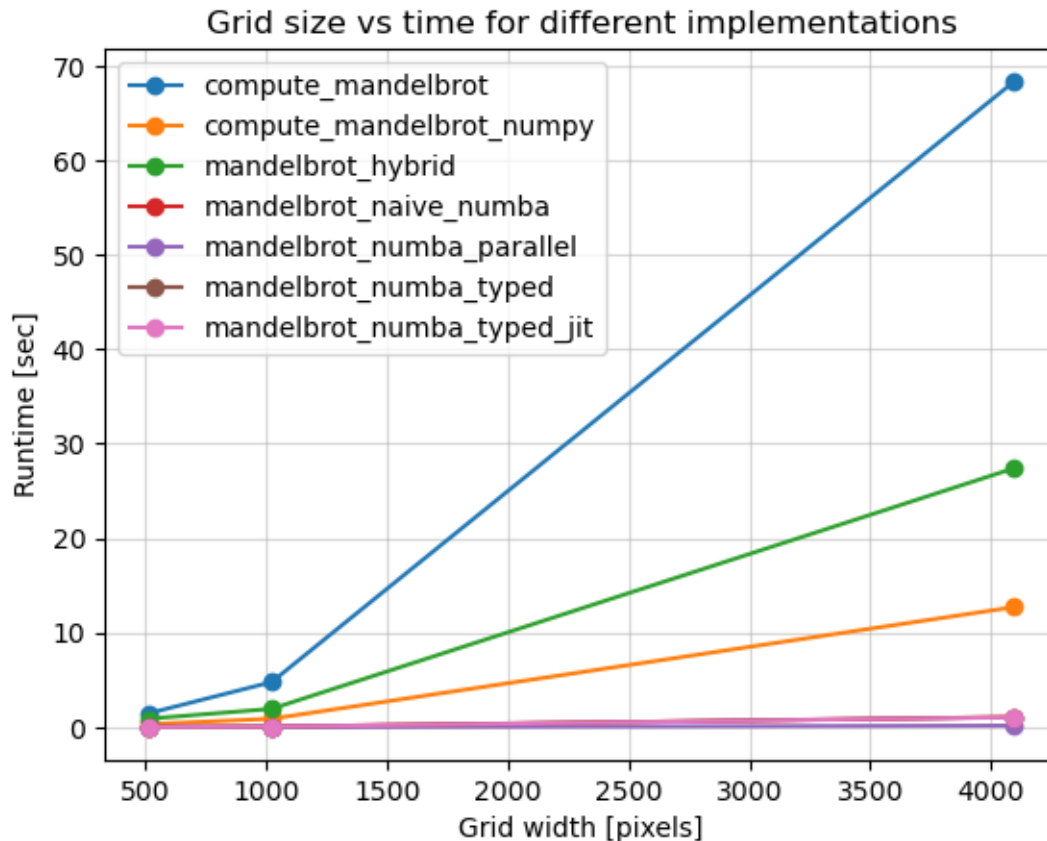
for impl in implementations:
    # Filter entries for this implementation
    data: list[dict] = [entry for entry in results if entry["impl"] == impl]

    # Sort by grid size
    data = sorted(data, key=lambda x: x["grid"])

    grids = [entry["grid"] for entry in data]
    times = [entry["time"] for entry in data]

    plt.plot(grids, times, marker="o", label=impl)

plt.title("Grid size vs time for different implementations")
plt.xlabel("Grid width [pixels]")
plt.ylabel("Runtime [sec]")
plt.grid(alpha=0.5)
plt.legend()
plt.show()
```



- Does speedup scale with problem size?

```
[30]: # AI assisted
# lookup: (impl, grid) -> time
lookup = {(r["impl"], r["grid"]): r["time"] for r in results}
grids = sorted({r["grid"] for r in results})

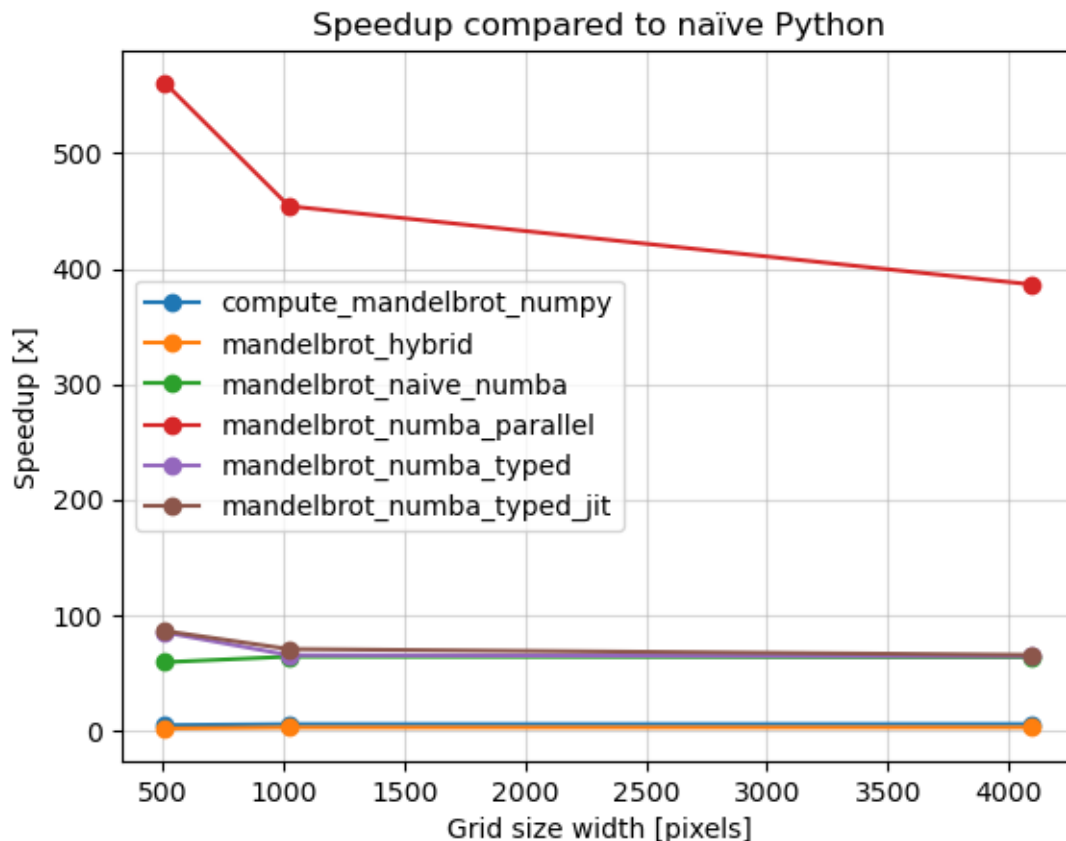
# Plot speedup
for impl in implementations:
    if impl == "compute_mandelbrot":
        continue # Baseline speedup

    speedups = []
    valid_grids = []

    for grid in grids:
        t_base = lookup[("compute_mandelbrot", grid)]
        t_impl = lookup[(impl, grid)]
        speedups.append(t_base / t_impl)
        valid_grids.append(grid)
```

```
plt.plot(valid_grids, speedups, marker="o", label=impl)

plt.title("Speedup compared to naïve Python")
plt.xlabel("Grid size width [pixels]")
plt.ylabel("Speedup [x]")
plt.grid(alpha=0.5)
plt.legend()
plt.show()
```



It appears so, but the parallel behaves differently with its speedups. I suspect that this is due to scheduling overhead, but still a massive speedup!

1.4 Reflection

In this mini project, combined with the lectures one, two, and three, I was able to conduct *almost* every single exercise clearly documented in both Jupyter Notebook and PDF format. Lecture one focused on setting and configuring a reproducible environment, where I extended it using a devcontainer for better developer user experience and a highly reproducible environment.

In lecture two, I locked in a naive Mandelbrot Python runtime of 4.113s with NumPy of 0.578s

resulting in a speedup of $6.85\times$. I also conducted a memory usage profiling between Python and NumPy, Python's usage slowly growing over time as compared to NumPy's single large memory allocation.

In lecture three, I confirmed my suspicions with line profiling that 99% of my naive python implementation runtime was spent appending and computing a single point. I also implemented a compiled version using Numba, resulting in a runtime of $0.048s$, a speedup from naive Python of $86.6\times$. In the last lecture's extensions, I plotted the speedup of all implementations so far, with the parallel having the greatest speedup of about $560\times$.

Implementation	Time	Speedup
Naive	4.113s	N/A
NumPy	0.578s	6.85x
Numba	0.048s	86.6x